AI / ML / DL / NLP Interview Prep: Zero to Hero

System design · ML · Deep learning · NLP & LLMs · Computer vision · RL · Time series · Recommenders · Speech · Graph ML · GenAI & agents

Statistics · Coding · SQL & data engineering · Cloud · MLOps · Security · Product sense · Leadership · 20 industries

With "what if this happens?" scenario answers for every topic

Contents

1. 01. System Design Fundamentals (Zero → Solid)
2. 02. ML & LLM Systems: How They're Actually Built
3. 03. The Debugging Framework (How to Answer "What If X Happens?")
4. 04. 50 "What If…?" Scenarios with Model Answers
5. 05. Case Studies: Full Interview Answers
6. 06. Interview Playbook & Cheat Sheet
7. 07. Machine Learning Fundamentals (Theory Round)
8. 08. Deep Learning (Theory + Training Debugging)
9. 09. NLP & LLMs (Theory + Scenarios)
10. 10. Computer Vision
11. 11. Statistics, Probability & A/B Testing
12. 12. ML Coding Round (Implement From Scratch)
13. 13. MLOps & Production ML (Q&A)
14. 14. DSA Coding Round (for AI/ML roles)
15. 15. Behavioral & Project Deep-Dive Round
16. 16. Reinforcement Learning & Bandits
17. 17. Time Series & Forecasting
18. 18. Recommender Systems (Deep Dive)
19. 19. Speech & Audio AI
20. 20. Graph Machine Learning
21. 21. Generative & Multimodal AI
22. 22. AI Agents
23. 23. Industry Playbooks: AI/ML Interviews in Every Industry
24. 24. Classic System Design Cases (For Any Engineer)
25. 25. Data Engineering & SQL
26. 26. Cloud & Infrastructure for ML
27. 27. Security, Privacy & Responsible AI
28. 28. AI Product Sense
29. 29. Senior, Staff & Leadership Rounds (AI/ML)

AI / ML / DL / NLP Interview Prep: Zero to Hero

A complete prep course for **AI Engineer, ML Engineer, Applied Scientist, Data Scientist, NLP / CV / LLM / Speech / RL Engineer, MLOps, Data Engineer and Backend/Software Engineer** interviews, in **every industry** (healthcare, finance, e-commerce, ads, autonomous driving, manufacturing, and 14 more).

Prefer one file? Download the full course as a PDF: AI-ML-Interview-Prep.pdf

Most candidates don't fail because they lack knowledge. They fail because when the interviewer says **"What if this happens?"** they have no *structured way to think out loud*. Every file here includes those scenario questions with model answers.

Step 1: Know which rounds you'll face

Interview round	What they test	Study file(s)
ML theory / fundamentals	bias-variance, metrics, algorithms, overfitting, imbalance	07-ml-fundamentals.md
Deep learning	backprop, optimizers, CNN/RNN/Transformers, training debugging	08-deep-learning.md
NLP / LLM	tokenization, embeddings, BERT/GPT, RLHF, LoRA, prompting, hallucination	09-nlp-and-llms.md
Computer vision	detection, segmentation, ViT, CLIP, mAP, edge deployment	10-computer-vision.md
Statistics & A/B testing	probability, Bayes, p-values, experiment design	11-statistics-and-ab-testing.md
ML coding	implement algorithms from scratch (NumPy/PyTorch)	12-ml-coding.md
DSA coding	LeetCode-style problems	14-dsa-coding-round.md
ML / LLM system design	design RAG, recommender, fraud system, LLM platform	01, 02, 05
"What if X happens?" / debugging	production failures, inconsistent outputs, drift, outages	03 (method) + 04 (50 scenarios) + "what-if" sections in 07-13
MLOps / production	CI/CD for ML, serving, monitoring, versioning	13-mlops.md
Project deep dive & behavioral	your own projects, STAR stories	15-behavioral-and-project-deep-dive.md
Interview day strategy	45-min script, phrases, cheat sheet	06-interview-playbook.md
Reinforcement learning & bandits	MDPs, Q-learning, PPO, exploration, RLHF	16-reinforcement-learning.md
Time series & forecasting	ARIMA, backtesting, demand forecasting, anomalies	17-time-series-and-forecasting.md

Interview round	What they test	Study file(s)
Recommender systems	collaborative filtering, two-tower, ranking, bias, cold start	18-recommender-systems.md
Speech & audio	ASR, TTS, WER, voice agents	19-speech-and-audio.md
Graph ML	GNNs, knowledge graphs, GraphRAG, fraud rings	20-graph-ml.md
Generative & multimodal AI	diffusion, vision-language models, multimodal RAG	21-generative-and-multimodal-ai.md
AI agents	tool use, planning, memory, agent evaluation & platforms	22-ai-agents.md
Industry / domain rounds	how AI is used and questioned in 20 industries	23-industry-playbooks.md
General system design (any engineer)	news feed, video streaming, ride-hailing, payments, Google Docs…	24-classic-system-design-cases.md
Data engineering & SQL	modeling, pipelines, Spark, 20+ SQL problems	25-data-engineering-and-sql.md
Cloud & infrastructure	AWS/GCP/Azure, Kubernetes, GPUs, cost	26-cloud-and-infrastructure-for-ml.md
Security, privacy & responsible AI	attacks on ML, privacy law, fairness, governance	27-security-privacy-responsible-ai.md
AI product sense	framing problems, metrics, ROI, AI UX	28-ai-product-sense.md
Senior / staff / lead rounds	strategy, leadership, hiring, incidents	29-senior-and-leadership.md

---

Step 2: Learn the one method for all "what if" questions

> **C**larify → **L**ayer map → **H**ypotheses (ranked) → **I**solate (how to prove it) → **F**ix (now + long-term) → **P**revent (monitoring/evals)

Read 03-debugging-framework.md **first**. It works for RAG, ML models, training bugs, data pipelines, and infrastructure.

Step 3: Follow a plan

10-week full plan (from zero)

Week	Theory (1-1.5 h/day)	Practice (1 h/day)
1	07 ML fundamentals	14 DSA patterns 1-2
2	11 Statistics + 07 what-ifs	12 ML coding §1-5

Week	Theory (1-1.5 h/day)	Practice (1 h/day)
3	08 Deep learning	14 DSA patterns 3-5
4	09 NLP & LLMs	12 ML coding §6-12
5	10 Computer vision (skip if not CV role) + 03 framework	14 DSA patterns 6-9
6	01 System design fundamentals	04 scenarios A-B (out loud)
7	02 ML & LLM systems + 13 MLOps	04 scenarios C-F
8	05 Case studies 1-4	14 DSA patterns 10-12
9	05 Case studies 5-11 + 15 project prep	2 full mock interviews
10	06 playbook, revise weak areas	4 mock interviews (mixed rounds)

Weeks 11-14: specialize (pick what fits your target job)

Week	Track A: AI specialist	Track B: Software / data	Track C: Senior roles
11	18 Recommenders or 19 Speech or 16 RL	24 Classic cases 1-7	28 Product sense
12	21 Generative & 22 Agents	24 Classic cases 8-20	29 Leadership
13	17 Time series or 20 Graph ML	25 Data engineering & SQL	27 Security & responsible AI
14	23 Your industry + 27	26 Cloud & infra	23 Your industry + mocks

2-week emergency plan (interview soon)

Days	Focus
1-2	03 framework + 04 scenarios (your domain sections)
3-4	07 + 08 Q&A (read questions, answer out loud)
5-6	09 (or 10 for CV) + 11
7-8	02 + 05 case studies 1, 4, 5
9-10	12 ML coding (top 6) + DSA patterns 1, 3, 7
11-12	15: write 2 project deep dives + 6 STAR stories
13-14	Mocks + 06 cheat sheet

By role: what to prioritize

Role	Highest priority files
AI / LLM Engineer	03, 04, 02, 09, 05, 13, 15
ML Engineer	07, 08, 12, 13, 02, 04, 05, 14
Data Scientist / Applied Scientist	07, 11, 12, 08, 15, 05 (#4-5)
NLP Engineer	09, 08, 12, 04 (A-B), 02
CV Engineer	10, 08, 12, 13, 05 (#7-8)
MLOps Engineer	13, 26, 02, 01, 04 (C-E), 05 (#3)

Role	Highest priority files
Recommender / Ranking Engineer	18, 07, 11, 05 (#4, #6), 23 (e-commerce, ads, media)
Speech / Voice AI Engineer	19, 08, 09, 22, 12
RL / Robotics Engineer	16, 08, 10, 23 (autonomous vehicles)
Forecasting / Quant / Supply-chain DS	17, 07, 11, 25, 23 (finance, logistics, energy)
GenAI / Agents Engineer	22, 21, 09, 02, 04, 27, 28
Data Engineer	25, 26, 01, 24, 13
Backend / Software Engineer	01, 24, 05 (#9-11), 14, 26
Senior / Staff / Lead / Manager	29, 28, 06, 15, plus your role's files

Interviewing in a specific industry?

Read the general files for your role, then your industry in 23-industry-playbooks.md. Its last table tells you which other chapters matter most for that industry.

Step 4: Practice the right way

1. **Say answers out loud.** Reading feels like learning; interviews test speaking.
2. Cover the answer → answer in 60 s (theory) or 3 min (scenario) → compare → mark misses → redo in 2 days.
3. Record mock interviews and watch them back.
4. Ask an AI or a friend to play interviewer: "*Ask me [ML theory / LLM system design / what-if] questions one by one, push back with follow-ups, then score my answer for structure, depth, and trade-offs.*"

All files

#	File
01	System design fundamentals
02	ML & LLM systems
03	Debugging framework (what-if method)
04	50 what-if scenarios
05	Case studies
06	Interview playbook & cheat sheet
07	ML fundamentals
08	Deep learning
09	NLP & LLMs
10	Computer vision
11	Statistics & A/B testing

#	File
12	ML coding from scratch
13	MLOps
14	DSA coding round
15	Behavioral & project deep dive
16	Reinforcement learning & bandits
17	Time series & forecasting
18	Recommender systems
19	Speech & audio
20	Graph ML
21	Generative & multimodal AI
22	AI agents
23	Industry playbooks (20 industries)
24	Classic system design cases
25	Data engineering & SQL
26	Cloud & infrastructure for ML
27	Security, privacy & responsible AI
28	AI product sense
29	Senior & leadership rounds

Rebuild the PDF

```
pip install markdown pygments
python scripts/build_pdf.py      # needs Chrome or Edge installed
```

Contributing

Found a mistake, or want to add a scenario question you were asked in a real interview? Open an issue or a pull request. New "what if" scenarios should follow the same format: **causes (ranked) → how to isolate → fix (now / long-term) → prevent.**

If this helped you, give the repo a star so others can find it.

License

Content is licensed under CC BY 4.0: free to share and adapt, with credit.

01. System Design Fundamentals (Zero → Solid)

Even ML/AI interviews expect these basics. Every AI system is still a web system underneath.

Contents

1. The big picture
2. Networking & APIs
3. Scaling: load balancers, caching, CDN
4. Databases
5. Async processing: queues & streams
6. Distributed systems concepts
7. Reliability, monitoring, security
8. Back-of-the-envelope estimation

Part 1: The big picture

System design = deciding which components you need and how they talk, so the system meets its requirements at scale.

Two kinds of requirements: - **Functional**: what it does ("users can upload documents and ask questions"). - **Non-functional**: how well it works (scale, latency, availability, consistency, cost, security).

The growth story (memorize this: it's the skeleton of every design)

```
Stage 1:  User → [Single server: app + DB]                    (works for ~100s of users)
Stage 2:  User → [App server] → [Database]                    (separate concerns)
Stage 3:  User → [Load balancer] → [App 1..N] → [Database]    (horizontal scaling)
Stage 4:  ... + [Cache] in front of DB                        (fast reads)
Stage 5:  ... + [CDN] for static files                        (global speed)
Stage 6:  ... + DB replicas (reads) + sharding (writes)       (data scale)
Stage 7:  ... + [Queue] → [Workers] for slow jobs             (async)
Stage 8:  ... + multiple regions, monitoring, autoscaling     (global + reliable)
```

The 5 qualities you trade off

Quality	Question	Typical tool
Scalability	Handle 10× users?	horizontal scaling, sharding, queues
Latency	How fast?	caching, CDN, indexes, async
Availability	Stays up when parts fail?	replication, redundancy, failover
Consistency	Everyone sees the same correct data?	single leader, transactions, quorum
Cost	Affordable?	right-sizing, caching, batch vs real-time

Part 2: Networking & APIs

What happens when you type a URL: DNS resolves name → IP → TCP connection (+ TLS handshake for HTTPS) → HTTP request → load balancer → server → response.

Concept	One-liner
IP / DNS	IP = address of a machine; DNS = phone book from name to IP
TCP	Reliable, ordered delivery (web, APIs, DBs)
UDP	Fast, no guarantees (video calls, gaming, DNS)
HTTP/HTTPS	Request/response protocol; HTTPS = encrypted via TLS
HTTP/2, HTTP/3	Multiplexing many requests on one connection; HTTP/3 over QUIC (UDP)

API styles

Style	Use when	Example
REST	Default for public/CRUD APIs	`GET /documents/42`
GraphQL	Clients need flexible queries, many entity types	mobile apps
gRPC	Fast internal service-to-service calls (binary, typed)	model server calls
WebSocket	Two-way real-time	chat, live collaboration
SSE (server-sent events)	Server → client streaming	**streaming LLM tokens**
Long polling	Simple near-real-time fallback	notifications
Webhooks	Notify another system when something happens	"payment completed"

Good API design rules

- Nouns for resources, HTTP verbs for actions (`POST /chats/{id}/messages`).
- **Pagination** (cursor-based for large lists), **versioning** (`/v1/`), **idempotency keys** for POST that must not run twice (payments), proper status codes (200, 201, 400, 401, 403, 404, 429, 500, 503).
- **Rate limiting** per user/API key.

Part 3: Scaling: load balancers, caching, CDN

Vertical vs horizontal scaling

- **Vertical** = bigger machine. Simple, but has a ceiling and is a single point of failure.
- **Horizontal** = more machines. Needs **stateless** servers (sessions in Redis/DB, not memory).

Load balancer

Spreads requests across servers, removes unhealthy ones (health checks). - **L4** (TCP level, fast) vs **L7** (HTTP level, can route by URL/header, e.g., `/api/llm` → GPU pool). - Algorithms: round-robin, least connections,

weighted, consistent hashing (sticky by key). - Related: **reverse proxy** (Nginx), **API gateway** (auth, rate limiting, routing, logging in one place).

Caching

Store results of expensive operations in fast memory (Redis/Memcached).

Pattern	How it works	Use
Cache-aside (most common)	App checks cache → miss → read DB → write cache	general reads
Write-through	Write to cache and DB together	data read right after write
Write-back	Write to cache, flush to DB later	high write rate, risk of loss
TTL	Entries expire after time	everything

- **Eviction:** LRU (least recently used) is the default answer.
- **Problems you should mention:**
- *Stale data* → TTL + invalidate on write.
- *Thundering herd / cache stampede* → many requests miss at once and hit DB → use locks / request coalescing / staggered TTLs.
- *Hot key* → one key gets huge traffic → replicate key, local cache.
- **AI-specific:** exact-match cache for prompts, **semantic cache** (embed query, return cached answer if similarity > threshold), embedding cache, **KV/prefix cache** in LLM servers.

CDN

Copies static files (images, JS, videos, model files) to servers worldwide. Users download from nearest location.

Rate limiting algorithms

- **Token bucket** (most common): bucket refills at rate R, each request takes a token; allows bursts.
- **Leaky bucket**: smooth constant output.
- **Fixed window / sliding window counter**: count requests per time window.
- Store counters in Redis for distributed limits. For LLMs, limit **tokens per minute**, not just requests.

Part 4: Databases

SQL vs NoSQL

	SQL (Postgres, MySQL)	NoSQL
Model	Tables, relations, joins	Key-value (Redis, DynamoDB), document (MongoDB), wide-column (Cassandra), graph (Neo4j)
Strength	Transactions (ACID), complex queries, correctness	Massive scale, flexible schema, high write throughput

	SQL (Postgres, MySQL)	NoSQL
Pick when	Payments, users, orders, metadata	Chat messages, event logs, sessions, feeds

Interview default: "I'll start with Postgres for metadata because we need consistency and joins; I'd move high-volume append-only data like chat messages to Cassandra/DynamoDB."

Specialized stores you should name: - **Object storage** (S3/GCS): files, images, model weights, datasets. Cheap and huge. - **Vector DB** (Pinecone, Weaviate, Milvus, Qdrant, pgvector): embeddings + similarity search. - **Search engine** (Elasticsearch/OpenSearch): full-text keyword search, inverted index, BM25. - **Data warehouse / lakehouse** (BigQuery, Snowflake, Databricks): analytics, training data. - **Time-series DB**: metrics. **Feature store** (Feast, Tecton): ML features.

ACID vs BASE

- **ACID**: Atomic, Consistent, Isolated, Durable (bank transfers).
- **BASE**: Basically Available, Soft state, Eventually consistent (likes counter).

Indexes

Like a book index. It makes reads fast, writes slightly slower. B-tree (most SQL), LSM-tree (write-heavy: Cassandra, RocksDB). For vectors: **HNSW** (graph, fast, memory-heavy), **IVF** (clusters), **PQ** (compression).

Replication

Copies of data on multiple machines. - **Leader-follower:** writes go to leader, reads from followers. Followers may lag → **replication lag** (user writes, then reads old data). Fix: read-your-own-writes from leader. - **Multi-leader:** writes in multiple regions; must resolve conflicts. - **Leaderless (quorum):** write to W nodes, read from R nodes; if **W + R > N** you read latest data.

Sharding (partitioning)

Split data across machines. - **By hash of key** (even spread): `shard = hash(user_id) % N`. - **By range** (good for range queries, risk of hot spots). - **By tenant** (common in B2B/RAG, gives isolation per customer). - **Consistent hashing:** keys and servers on a ring; adding a server only moves ~1/N keys. (Used in caches, Cassandra, DynamoDB.) - Problems: hot shards, cross-shard joins, resharding.

Denormalization

Duplicate data to avoid joins for fast reads (e.g., store author name inside post). Trade-off: harder updates.

Part 5: Async processing: queues & streams

Why: don't make users wait for slow work (embedding a 500-page PDF, sending emails, training jobs).

```
API → [Queue] → Workers (scale independently) → result in DB → notify user (webhook / polling / websocket)
```

Tool	Type	Use
RabbitMQ / SQS	Message queue (task consumed once)	background jobs
Kafka / Kinesis / Pub/Sub	Log/stream (many consumers, replayable)	event streaming, feature pipelines, logs

- **Delivery guarantees:** at-most-once, **at-least-once (default)**, exactly-once (hard). With at-least-once, make consumers **idempotent**.
- **Dead-letter queue (DLQ):** messages that fail repeatedly go aside for inspection.
- **Backpressure:** when consumers can't keep up, the queue grows; autoscale workers or slow producers.
- **Batch vs stream processing:** Spark (batch, hourly/daily) vs Flink/Kafka Streams (real-time). ML: batch features vs real-time features.

Architecture styles

- **Monolith**: one deployable. Start here. Simple.
- **Microservices**: separate services per domain; independent scaling & deploys; cost: network calls, distributed debugging.
- **Event-driven**: services react to events on Kafka.
- **CQRS**: separate write model and read model.
- **Saga**: distributed transaction as a sequence of local steps with compensating actions.

Part 6: Distributed systems concepts

Concept	Plain explanation
CAP theorem	During a network **P**artition you must choose **C**onsistency (reject requests) or **A**vailability (serve possibly stale data). Payments → CP. Social feed → AP.
PACELC	Even without partition: trade-off between **L**atency and **C**onsistency.
Eventual consistency	All replicas converge eventually; reads may be stale briefly.
Strong consistency	Every read sees latest write. Slower.
Consensus (Raft/Paxos)	Nodes agree on a value/leader despite failures (etcd, ZooKeeper).
Leader election	Pick one node to coordinate (e.g., one scheduler for training jobs).
Idempotency	Doing an operation twice = same result as once. You need this whenever you retry.
Retries + exponential backoff + jitter	Retry after 1s, 2s, 4s (+ random) to avoid overwhelming a recovering service.
Circuit breaker	Stop calling a failing service for a while; return fallback.
Bulkhead	Isolate resources so one failure doesn't sink everything (separate pools).
Timeouts	Every network call needs one.
Unique IDs	UUID (random), Snowflake (time + machine + sequence, sortable).
Bloom filter	Tiny structure answering "definitely not in set / maybe in set" (crawler seen-URLs).
Heartbeats / gossip	Detect dead nodes / spread cluster state.

Concept	Plain explanation
2PC	Two-phase commit across DBs. Consistent, but slow and blocking.

Part 7: Reliability, monitoring, security

Availability numbers

Availability	Downtime/year
99%	3.65 days
99.9%	8.8 hours
99.99%	52 minutes
99.999%	5 minutes

- **No single point of failure:** redundancy everywhere (multiple servers, replicas, zones).
- **Failover:** active-passive or active-active.
- **SLI / SLO / SLA:** indicator (p95 latency), objective (p95 < 2s for 99% of days), agreement (contract with penalties).

Observability (3 pillars)

- **Metrics**: numbers over time (QPS, latency p50/p95/p99, error rate, GPU utilization, token usage).
- **Logs**: events with details.
- **Traces**: one request followed across all services (OpenTelemetry). **For LLM apps, trace every step: retrieval, prompt, model call, tokens, cost.**

Deployments

- **Blue-green:** two environments, switch traffic.
- **Canary:** send 1-5% of traffic to new version, watch metrics, then expand.
- **Shadow:** new version receives copy of traffic, its outputs are not shown to users (very useful for ML models).
- **Feature flags:** turn features on/off without deploy.

Security basics

- **AuthN** (who are you: OAuth2/OIDC, JWT) vs **AuthZ** (what can you do: RBAC/ABAC).
- Encryption in transit (TLS) and at rest.
- Secrets in a vault, never in code/prompts.
- Least privilege, audit logs, input validation, multi-tenant isolation.

Part 8: Back-of-the-envelope estimation

Numbers to memorize

1 day	≈ 86,400 s ≈ **10^5 s**
1M requests/day	≈ **12 QPS**
100M requests/day	≈ 1,200 QPS
Peak traffic	≈ 2-3× average
Memory read	~100 ns
SSD read	~100 µs
Same-datacenter network round trip	~0.5 ms
Cross-continent round trip	~150 ms
Redis get	~1 ms
DB indexed query	~5-50 ms
LLM first token	~200-800 ms
LLM generation	~30-100 tokens/s per stream
1 token	≈ 0.75 English words ≈ 4 chars
Embedding (float32, 1536 dims)	1536 × 4 B ≈ **6 KB**
7B model in FP16	≈ 14 GB GPU memory (2 bytes/param)

Formula

```
QPS          = daily_requests / 86,400          (× 2-3 for peak)
Storage      = items × size × retention
Bandwidth    = QPS × response_size
Servers      = peak QPS / QPS_per_server
```

Worked example: RAG for a company with 10,000 employees

- Each asks 10 questions/day → 100k/day → **~1.2 QPS avg, ~5 QPS peak**. (Small! Don't over-engineer.)
- Corpus: 1M documents × 10 chunks = **10M chunks** × 6 KB = **60 GB vectors** (+ HNSW overhead ~1.5×) → fits on one vector DB node with replicas; or compress with PQ.
- LLM tokens: 100k × (3k input + 300 output) = 330M tokens/day → cost estimate from price per million tokens.
- Conclusion: the bottleneck is **LLM cost and latency**, not QPS → focus on caching, routing, context size.

> **Interview tip:** Always say what the numbers *mean* for the design ("5 QPS means a single service with 2 replicas for HA is enough; the real concern is cost").

02. ML & LLM Systems: How They're Actually Built

The interviewer for an AI/ML role wants to see you think **beyond the model**: data, serving, evaluation, monitoring, cost.

> **Keep this in mind:** in production ML, the model is ~10% of the system. The other 90% is data pipelines, serving, monitoring and feedback loops.

Contents

1. The ML lifecycle
2. Data & features
3. Training infrastructure
4. Model serving
5. Deployment & experimentation
6. Monitoring ML systems
7. LLM systems: RAG in depth
8. LLM serving & optimization
9. Agents & tool use
10. Evaluation of LLM systems
11. Safety, security & governance
12. Decision tables (when to use what)

---

1. The ML lifecycle

```
Problem framing → Data collection → Labeling → Feature engineering → Training → Evaluation
       ↑                                                                          │
       └── Monitoring ← Serving ← Deployment (canary/shadow) ← Model registry ←──┘
                 │
                 └── feedback / new labels → retraining
```

Problem framing questions (ask these in every ML design interview): - What is the business goal? What is the ML objective (proxy)? Are they aligned? - Is ML even needed? (Rules/heuristics baseline first.) - Online (real-time) or batch prediction? Latency budget? - Where do labels come from? How delayed are they? - What's the cost of a false positive vs false negative?

2. Data & features

- **Feature store** = one place to define features, used by both training (offline store, historical, point-in-time correct) and serving (online store, low latency, e.g., Redis). **Main reason: prevents train/serve skew.**

- **Point-in-time correctness:** when building training rows, only use feature values that existed *at that time*. Otherwise you get leakage.
- **Batch features** (computed daily: "user's purchases last 30 days") vs **real-time features** (streaming: "transactions in last 5 minutes").
- **Data validation:** schema, null rates, ranges, distribution checks before training.
- **Data versioning:** DVC, Delta Lake/Iceberg time travel.
- **Labeling:** human labelers, weak supervision, implicit feedback (clicks), LLM-assisted labeling + human review.

3. Training infrastructure

- Orchestration: Airflow / Kubeflow / Prefect / SageMaker pipelines.
- Experiment tracking: MLflow, Weights & Biases.
- Distributed training: **data parallel** (copy model, split data; DDP), **model/tensor parallel** (split model across GPUs), **pipeline parallel**, **FSDP/ZeRO** (shard optimizer state + params). Mixed precision (bf16).
- Checkpointing (resume after spot instance preemption).
- **Model registry:** versioned artifacts + metadata + stage (staging/production).

4. Model serving

Mode	When	Example
Batch	Predictions can be precomputed	nightly recommendations, churn scores
Online (real-time)	Need fresh prediction per request	fraud check at payment
Streaming	Predict on events as they arrive	anomaly detection on logs
On-device / edge	Privacy, offline, low latency	keyboard prediction

Serving stack: model server (Triton, TorchServe, TF Serving, BentoML, vLLM for LLMs) behind an API, autoscaling, **dynamic batching** (group requests to use GPU efficiently), model caching. Optimization: quantization, distillation, pruning, ONNX/TensorRT compilation.

Two-stage pattern (search/recs/ads): cheap **candidate generation** (millions → 1000s, e.g., ANN on embeddings) → expensive **ranking** model (1000s → 100) → **re-ranking** with business rules (diversity, freshness).

5. Deployment & experimentation

- **Shadow mode** → **canary** → **A/B test** → full rollout. Keep previous model warm for instant rollback.
- **A/B testing:** randomize by user, pick primary metric + guardrail metrics (latency, errors, revenue), power analysis for sample size, beware novelty effect and peeking.
- **Multi-armed bandits** when you want to explore + exploit continuously.
- **Champion / challenger** for automated retraining.

6. Monitoring ML systems

Layer	What to monitor
System	latency, throughput, errors, GPU/CPU/memory
Data	missing values, schema, feature distributions (**data drift**: PSI, KS test)
Model	prediction distribution, confidence, **performance when labels arrive** (concept drift)
Business	conversion, fraud losses, user satisfaction
Alert → investigate → retrain/rollback. Retraining triggers: schedule, drift threshold, performance drop.	

---

7. LLM systems: RAG in depth

RAG (Retrieval-Augmented Generation) = fetch relevant knowledge, put it in the prompt, let the LLM answer grounded in it. Use RAG when knowledge changes, is private, or needs citations. (Fine-tuning teaches *style/format/behavior*, not fresh facts.)

Offline: ingestion pipeline

```
Sources (PDF, Confluence, DB, web) → Connectors → Parse (OCR, tables, layout)
 → Clean (remove boilerplate, dedupe) → Chunk → Enrich metadata (title, section, date, ACL,
tenant)
 → Embed → Upsert into vector index (+ keyword index) → Version the index
```

Chunking choices: | Strategy | Notes | |----------|-------| | Fixed size (e.g., 500 tokens, 50 overlap) | Simple baseline | | Recursive / by structure (headings, paragraphs) | Better coherence, usually the best default | | Semantic chunking | Split where topic changes | | Parent-child / small-to-big | Retrieve small precise chunk, send bigger parent section to LLM | | Per table row / per FAQ item | Structured content | Add **contextual headers** to each chunk: "Doc: HR Policy 2025 > Section: Leave > ...". This improves retrieval a lot.

Online: query pipeline

```
Query → guardrails → (rewrite / expand / decompose / HyDE) → embed
 → hybrid retrieval (vector + BM25) with metadata/ACL pre-filter → merge (RRF)
 → rerank (cross-encoder) → top-k → build context (order, dedupe, token budget)
 → prompt (instructions + context + question + cite sources) → LLM → output guardrails +
citation check → stream to user
```

Key techniques to name: - **Hybrid search**: vectors capture meaning, BM25 captures exact words (product codes, names). Combine with **Reciprocal Rank Fusion**. - **Reranking**: cross-encoder scores (query, chunk) pairs jointly; big precision boost. - **Query rewriting**: turn "what about for contractors?" into a standalone query using chat history. - **HyDE**: generate a hypothetical answer, embed it, search with that. - **Multi-query**: generate several phrasings, union results. - **Metadata filtering**: tenant, permissions, date,

doc type. Pre-filter for security. - **GraphRAG / knowledge graphs**: for entity-relationship and multi-hop questions. - **Agentic RAG**: LLM decides when/what to retrieve iteratively.

Vector DB essentials

- Similarity: cosine / dot product / L2 (normalize vectors for cosine).
- ANN indexes: **HNSW** (high recall, fast, RAM heavy), **IVF** (cluster then search nearby clusters), **PQ** (compress vectors, less memory, lower recall). ANN = *approximate* → results can vary slightly. This matters for consistency questions.
- Scaling: sharding, replicas, namespaces per tenant, quantization.
- Choice: pgvector (already on Postgres, < ~10M vectors), managed (Pinecone), self-hosted (Milvus, Qdrant, Weaviate), Elasticsearch/OpenSearch (if you need hybrid in one system).

8. LLM serving & optimization

Technique	What it does
Streaming (SSE)	Show tokens as they are generated, so the wait feels shorter
Continuous batching (vLLM, TGI)	New requests join the batch mid-generation → much higher GPU throughput
PagedAttention / KV cache	Efficient memory for attention cache
Prefix / prompt caching	Reuse computation for repeated system prompts/context
Quantization (INT8/INT4, AWQ, GPTQ)	Smaller memory, faster, slight quality loss
Speculative decoding	Small draft model proposes tokens, big model verifies
Distillation	Train small model to mimic big one
Model routing / cascade	Cheap model first; escalate hard queries to big model
Semantic cache	Return cached answer for similar question
LLM gateway	One internal API: auth, rate limits, quotas per team, fallback providers, logging, cost tracking

Key metrics: TTFT (time to first token), TPOT/ITL (time per output token), throughput (tokens/s), cost per 1k requests. **Memory rule of thumb:** params × bytes (FP16 = 2 B) + KV cache. 70B FP16 ≈ 140 GB → multiple GPUs (tensor parallel) or quantize.

API vs self-hosted: | API (closed models) | Self-hosted (open models) | |---|---| | Best quality, zero infra, fast start | Data control, cost at high volume, customizable | | Per-token cost, rate limits, vendor changes | GPU ops burden, scaling, lower top-end quality |

9. Agents & tool use

Agent loop: **LLM plans → calls a tool → observes result → repeats → final answer.** Design concerns: tool schemas & descriptions, max steps/cost limits, state/memory, error handling, **human-in-the-loop for risky actions**, sandboxing code execution, observability of each step, evaluation of trajectories (did it pick

the right tools?), permissions per tool (least privilege). Patterns: ReAct, plan-and-execute, router + specialist agents, multi-agent (use only when you need it; more agents means more failure points).

10. Evaluation of LLM systems

Level	Metrics / methods
Retrieval	recall@k, precision@k, MRR, nDCG (needs labeled query→doc pairs)
Generation	faithfulness/groundedness, answer relevance, correctness vs reference, citation accuracy
Consistency	agreement rate across repeated runs / paraphrased questions
Safety	jailbreak success rate, toxicity, PII leakage
Online	thumbs up/down, resolution rate, escalation rate, retention, latency, cost
Methods: golden datasets (built from real queries), LLM-as-judge (with rubric, validated vs humans), human review of samples, A/B tests, **regression evals in CI on every prompt/model change**.	

11. Safety, security & governance

- Input guardrails: prompt-injection detection, topic restriction, PII detection.
- Output guardrails: toxicity, PII, policy, schema validation, groundedness.
- **Indirect prompt injection** via retrieved docs/web/email. Treat all retrieved content as untrusted.
- Multi-tenant isolation (separate namespaces, ACL pre-filter, scoped caches).
- Data retention, audit logs, model cards, compliance (GDPR right-to-be-forgotten → must delete from index **and** caches **and** logs; hard for fine-tuned weights → another reason to prefer RAG).

12. Decision tables

Prompting vs RAG vs fine-tuning | Need | Use | |------|-----| | Fresh/private facts, citations | **RAG** | | Specific output style/format, domain jargon, smaller cheaper model | **Fine-tuning** (often LoRA) | | Quick iteration, general task | **Prompt engineering** first, always | | Both knowledge + behavior | RAG + fine-tuning |

Real-time vs batch prediction | Real-time | Batch | |-----------|-------| | Depends on request-time context (fraud at checkout) | Inputs known in advance (daily recs) | | Harder: latency, online features | Easier, cheaper, can use big models |

Which metric? | Problem | Metric | |---------|--------| | Imbalanced classification | precision/recall, PR-AUC, F1 | | Ranking | nDCG, MRR, recall@k | | Regression | MAE / RMSE | | Business | revenue, retention, CTR (with guardrails) |

03. The Debugging Framework (How to Answer "What If X Happens?")

This is the most important file. Interviewers ask scenario questions because they want to see **how you think**, not whether you memorized the answer. A calm, structured answer with an imperfect conclusion is better than a long list of random guesses.

The framework: C.L.H.I.F.P.

> **C**larify → **L**ayer map → **H**ypotheses → **I**solate (measure) → **F**ix (now + long-term) → **P**revent

Step	What you do	What you say
1. Clarify	Pin down the symptom. Who, when, how often, since when, what changed?	"Before I jump in: is this for *all* users or some? Started suddenly or gradually? Did we deploy anything recently?"
2. Layer map	Draw the request path. Every layer is a suspect.	"Let me walk the request path: query → rewrite → embed → retrieve → rerank → prompt → LLM → post-process."
3. Hypotheses	List causes per layer, **ranked by likelihood**.	"The most likely causes, in order, are: sampling temperature, retrieval non-determinism, and index version mismatch."
4. Isolate	How you'd *prove* which one it is, cheapest test first.	"I'd replay the same query 20 times and log retrieved chunk IDs. If chunks are identical but answers differ → it's generation. If chunks differ → it's retrieval."
5. Fix	Short-term mitigation + long-term fix.	"Short-term: temperature 0 and pin the model version. Long-term: versioned index with atomic swaps."
6. Prevent	Monitoring, evals, alerts, tests so it doesn't recur.	"I'd add a consistency eval to CI and alert when answer agreement drops below 90%."

Memory trick: "*Can Large Hippos Itch For Pizza?*"

Step 2 in detail: the layer maps

You must be able to draw these from memory. When asked any scenario, **draw the relevant one first**.

RAG / LLM application

```
User
 │
 ▼
[API gateway / auth / rate limit]
 │
```

```
 ▼
[Conversation memory]  ← chat history, user profile
 │
 ▼
[Query understanding]  ← rewrite, expand, classify intent, guardrails (input)
 │
 ▼
[Embedding model]      ← query → vector
 │
 ▼
[Retriever]            ← vector DB (ANN) + keyword (BM25) + metadata/ACL filters
 │
 ▼
[Reranker]             ← cross-encoder, top-k selection
 │
 ▼
[Context builder]      ← order chunks, truncate to token budget, add citations
 │
 ▼
[Prompt template]      ← system prompt + context + question
 │
 ▼
[LLM]                  ← model version, temperature, max tokens, provider
 │
 ▼
[Post-processing]      ← guardrails (output), format, citation check
 │
 ▼
[Cache]  [Logs/traces]  [Feedback]
```

Offline side (where many bugs live):

```
Source docs → parse/OCR → clean → chunk → embed → upsert to vector index → (index version)
```

Classic ML model in production

```
Raw data → ETL → Feature pipeline → Feature store (offline + online)
                                          │                  │
                                    Training job       Serving API ← request
                                          │                  │
                                    Model registry ──→ Model server → prediction → logging →
monitoring
```

Generic web service

```
Client → DNS → CDN → Load balancer → App servers → Cache → Database
                                          │
                                       Queue → Workers
```

Step 3 in detail: universal causes checklist

When you're stuck, run through these. One of them is almost always the answer.

Category	Questions to ask yourself
Change	What was deployed / changed? Code, model, prompt, data, config, dependency, provider?
Data	Did the input data change (drift, schema, nulls, new source, stale)?
Randomness	Is anything non-deterministic (sampling, ANN search, GPU math, random seeds, ties)?
State	Is there per-user state (history, cache, personalization, permissions)?
Versions	Can different replicas serve different versions (model, index, prompt, A/B test)?
Capacity	Is something overloaded (timeouts, rate limits, queue backlog, memory, GPU)?
Dependencies	Is a downstream service slow/failing (vector DB, LLM provider, DB)?
Train/serve skew	Is production computing something differently than training/evaluation did?
Feedback loops	Is the model influencing its own future training data?

Step 4 in detail: isolation techniques

Technique	When to use
Replay the same input N times, log every intermediate output	Inconsistency, non-determinism
Diff a good request vs. bad request trace	"Works for some users, not others"
Bisect: freeze all layers except one	Any multi-layer pipeline
Timeline correlation: overlay metric drop with deploy/data events	Sudden degradation
Slice metrics by segment (region, device, user type, doc type, language)	"Accuracy dropped"
Golden set: fixed eval questions with known answers	Quality regressions
Shadow / canary: run new version alongside old	Before shipping a fix

A good line to use in interviews:

> "I'd find the *first layer where the output differs* between the good and bad case. That's where the bug lives."

Step 5: always give TWO fixes

Interviewers want to hear both:

- **Mitigation (stop the bleeding, minutes-hours):** rollback, feature flag off, fallback model, cache, rate limit, raise timeout, serve "I don't know".
- **Root-cause fix (days-weeks):** re-architecture, retraining, new pipeline, versioning, data contracts.

Step 6: prevention vocabulary

Use these words: **evals in CI**, **golden datasets**, **canary deploys**, **shadow traffic**, **drift monitoring**, **SLOs + alerts**, **tracing (request ID through every layer)**, **data contracts / schema validation**, **versioning (model, prompt, index, data)**, **runbooks**, **post-mortem**.

Full worked example: the question you were asked

> **"Your RAG app has the same information, but gives different users different answers, and sometimes no answer at all. What do you do?"**

Speak it like this (≈4 minutes)

1. Clarify (30 sec)

> "A few questions first. Is it the *exact same question text*, or similar questions? Different answers in wording, or in facts? Does the same user get different answers when they retry, or only different users? And 'no answer': is that an 'I don't know', an error, or a timeout? Did anything change recently, like the model, prompt or documents?"

(Interviewer will give you some answers. If they say "you decide", state your assumptions.)

2. Layer map (30 sec): draw the RAG pipeline above.

3. Hypotheses, ranked (1 min)

Different answers to the same question: 1. **Generation randomness**: temperature > 0, no seed. Most common cause. 2. **Per-user context**: chat history, user profile, or name/date injected into the prompt makes prompts differ. 3. **Permission / metadata filters**: users have different access rights (ACLs), so they retrieve different documents. *This may be correct behavior!* 4. **Retrieval non-determinism**: approximate nearest-neighbour search, ties in scores, different replica returns slightly different top-k, chunk order changes. 5. **Version skew**: replicas have different index versions (mid-reindex) or different model/prompt versions (A/B test, canary, provider silently updated model). 6. **Conflicting source docs**: two documents say different things (old policy vs new); whichever gets retrieved wins. 7. **Query rewriting**: an LLM rewrites the query non-deterministically, so retrieval differs. 8. **Caching**: some users hit a stale cached answer, others get a fresh one.

Sometimes no answer: 1. **Retrieval returns nothing above the similarity threshold** → model correctly says "I don't know" (threshold too strict, or query phrasing differs from document vocabulary). 2. **Context overflow / truncation**: long chat history pushes the retrieved chunks out of the token budget. 3. **Guardrail false positives**: input/output filters block legitimate answers. 4. **Timeouts / rate limits** from vector DB or LLM provider → app swallows the error and returns empty. 5. **Chunking bug**: the answer is split across chunks, or the doc failed to parse (PDF table, scanned image) and never got indexed.

4. Isolate (1 min)

> "I'd pick a few failing questions and **replay each one 20 times**, logging a full trace: rewritten query, retrieved chunk IDs and scores, final prompt hash, model version, and the answer. - If **chunk IDs differ** between runs → the problem is in retrieval (non-deterministic ANN, index version, query rewrite). - If **chunk IDs are the same but answers differ** → it's generation (temperature, prompt differences). - If **the prompt hashes differ** for different users → it's per-user context (history, profile, ACLs). - For 'no answer' cases I'd check: did retrieval return 0 chunks? Was there an error or timeout? Did a guardrail fire?"

5. Fix (1 min)

Cause	Fix
Sampling	temperature = 0 (or low), fixed seed where supported, pin exact model version (`model-2025-xx-xx`, not `latest`)
Retrieval non-determinism	deterministic tie-break (sort by score, then doc_id); exact search for small corpora; hybrid search (BM25 + vectors) for stability
Version skew	**versioned indexes** + atomic alias swap (build new index fully, then flip pointer); pin model/prompt version per request and log it
Conflicting docs	dedupe, add `effective_date` / `source_of_truth` metadata, prefer latest, remove superseded docs
Per-user context	limit/ summarize history; keep volatile data (timestamp) out of prompt unless needed
ACLs	that's expected, but make the answer say "based on documents you have access to"
No retrieval hits	tune threshold on an eval set; query expansion / HyDE; hybrid search; better chunking
Overflow	token budgeting: reserve fixed space for retrieved context; summarize history
Errors	retries with backoff, fallback model, explicit error messages instead of empty answers
Cache	invalidate cache on index update (include index version in cache key)

6. Prevent (30 sec)

> "I'd add a **consistency eval**: run each golden question 5-10 times and measure answer agreement. Alert if it drops below a threshold. Add **retrieval metrics** (recall@k on a labeled set) and **groundedness** checks to CI. Log the model, prompt and index version on every request so any answer is reproducible. And monitor the rate of 'no answer' responses per day. A spike tells me retrieval or the provider broke."

Why this answer works: it shows you know the full pipeline, you rank causes, you know how to *prove* the cause, and you think about production.

Phrases that buy you thinking time (and sound senior)

- "Let me first make sure I understand the symptom…"
- "Let me walk through the request path, because every layer is a suspect."
- "The cheapest thing to check first is…"
- "I'd want to separate *retrieval* problems from *generation* problems."
- "This could actually be correct behavior if…, so I'd confirm the expected behavior first."
- "Short-term I'd mitigate by…, and the long-term fix is…"
- "To make sure this never happens silently again, I'd monitor…"
- "The trade-off here is… (latency vs quality / cost vs accuracy / freshness vs consistency)."

What NOT to do

- Don't jump straight to "I'd fine-tune the model." (Most expensive fix, rarely the cause.)
- Don't list 15 causes with no order. Rank them.
- Don't say "I'd check the logs" without saying **what** you'd look for in them.
- Don't forget the short-term mitigation. Production is burning *now*.
- Don't stay silent while thinking. Think out loud.

04. 50 "What If...?" Scenarios with Model Answers

Every answer follows the framework from 03-debugging-framework.md: **Causes (ranked) → How to isolate → Fix (now / long-term) → Prevent.**

How to practice: read only the question, cover the rest, speak for 3 minutes, then compare. Mark the ones you missed and redo them 2 days later.

Contents

- A. RAG systems (1-12)
- B. LLM apps & agents (13-22)
- C. ML models in production (23-32)
- D. Data & pipelines (33-38)
- E. Serving, infra & scale (39-46)
- F. Recommenders, search & ranking (47-50)

A. RAG systems

1. Same question, different answers for different users (or on retry)

Full worked answer in 03-debugging-framework.md. **Short version:** temperature/sampling → per-user prompt differences (history, ACLs) → non-deterministic retrieval (ANN, ties, query rewrite) → version skew (index/model/prompt) → conflicting docs → cache. **Isolate:** replay N times, log chunk IDs + prompt hash + model version. Same chunks + different answers = generation. Different chunks = retrieval. **Fix:** temp 0 + pinned model, deterministic tie-breaks, versioned index with atomic swap, dedupe conflicting docs, version in cache key. **Prevent:** consistency eval (agreement over N runs), log versions per request.

2. RAG says "I don't know" even though the answer is in the documents

Causes (ranked): 1. Document never indexed properly: PDF parse failure, scanned image without OCR, table flattened into garbage. 2. Chunking split the answer across chunks, or chunk too big so the embedding is diluted. 3. Vocabulary mismatch. User says "PTO", doc says "paid leave". Pure vector search misses exact terms, codes, IDs. 4. Similarity threshold too strict / top-k too small. 5. Metadata filter wrong (wrong tenant, date filter, ACL). 6. Answer retrieved but ranked low → cut off by reranker or context budget. 7. Prompt too strict ("only answer if 100% sure").

Isolate: search the vector DB directly for the doc → is it there? Inspect the chunk text. Run the query and print top-50 with scores. Where does the right chunk rank? If not in top-50 → embedding/chunking. In top-50 but not top-5 → reranking/k. **Fix:** better parser/OCR, chunk with overlap & respect headings, **hybrid search (BM25 + vector)**, query expansion/rewrite, raise k then rerank, tune threshold on labeled set. **Prevent:** retrieval eval set with **recall@k**, ingestion validation (empty-chunk checks, parse-success rate).

3. RAG gives confident wrong answers (hallucinations)

Causes: retrieved chunks are irrelevant but model answers anyway; model uses its own pretrained knowledge; outdated docs; prompt doesn't require grounding; conflicting chunks. **Isolate:** check if the answer text is supported by retrieved chunks (groundedness / faithfulness check). Classify failures: *retrieval failure* (right doc not retrieved) vs *generation failure* (right doc retrieved, model ignored it). **Fix:** prompt "answer only from context, cite sources, say I don't know otherwise"; require citations and verify cited chunk contains the claim; reranker + relevance threshold; smaller context with higher precision; post-hoc NLI/LLM-judge faithfulness check before returning. **Prevent:** faithfulness metric in evals (e.g., RAGAS-style), user feedback thumbs-down loop, hallucination rate dashboard.

4. Answers are outdated after the source docs were updated

Causes: ingestion pipeline not triggered / failing silently; old chunks not deleted (upsert adds new, old stays → both retrieved); answer cache not invalidated; embedding batch job lags. **Isolate:** compare doc `last_modified` vs. index `ingested_at`; search index for both old and new versions of the doc. **Fix:** change-data-capture (event on doc update) → re-ingest; **delete-by-doc_id then insert** (idempotent upserts keyed by doc_id + chunk_no); cache key includes index version; prefer newest via `effective_date` metadata. **Prevent:** freshness SLO ("95% of doc updates searchable in < 15 min"), alert on ingestion lag.

5. Retrieval quality dropped after switching embedding model

Cause: almost always, **query embedded with new model, documents still embedded with old model** (different vector spaces) or partially re-embedded index. Also: different normalization (cosine vs dot product), different max token length truncating chunks. **Fix:** re-embed entire corpus into a **new index**, test offline, then swap atomically. Never mix models in one index. Store `embedding_model_version` as index metadata and assert at query time. **Prevent:** retrieval eval gate before swap.

6. RAG is too slow (p95 = 8 s)

Isolate first: trace per stage. Typical budget: embed 20 ms, retrieve 30-100 ms, rerank 100-300 ms, LLM first token 300-800 ms, full generation 1-5 s. **Fixes by stage:** - LLM: **stream tokens** (perceived latency), smaller/faster model for simple queries (routing), shorter context, limit output tokens, prompt caching, speculative decoding/self-hosted vLLM. - Retrieval: ANN index (HNSW) tuning, fewer filters post-hoc, colocate DB. - Rerank: rerank top-20 not top-100, smaller cross-encoder. - Query rewrite: skip it for short clear queries, or run in parallel. - Cache: exact-match + **semantic cache** for frequent questions. - Parallelize independent calls (retrieve from 2 sources concurrently). **Trade-off:** latency vs quality. Measure quality impact of each cut on eval set.

7. Users see documents they shouldn't have access to

Severity: this is a security incident, so say that out loud. Mitigate immediately (disable feature or restrict to safe corpus). **Causes:** ACL filtering done *after* retrieval in the app (bug) or not at all; ACL metadata stale (permission revoked but index not updated); shared cache across users returns another user's answer; prompt injection asking to "show all docs". **Fix:** enforce **pre-retrieval filtering** in the vector DB with tenant/group IDs; per-tenant namespaces/indexes for strong isolation; cache key includes user/permission scope; sync ACL changes on events. **Prevent:** automated tests "user A queries doc B → must not appear", audit logs.

8. Works great in demo, bad in production

Causes: eval set doesn't represent real queries (demo questions were written by the team who know the docs); real users write short, vague, misspelled, multi-language queries; production corpus 100× bigger → more distractors. **Fix:** sample real production queries → label → new eval set; add query rewriting/clarifying questions; hybrid search; reranker; slice metrics by query type. **Prevent:** continuous eval on sampled production traffic.

9. Answer quality is fine for short docs, bad for long docs (100+ page manuals)

Causes: fixed-size chunks lose structure; tables/sections split; "lost in the middle" (LLMs ignore middle of long context). **Fix:** structure-aware chunking (by headings), parent-child retrieval (retrieve small chunk, give parent section), add doc title/section path to each chunk, summarise sections, put most relevant chunks at start/end of context.

10. Multi-hop questions fail ("Which manager approved the project that had the biggest budget?")

Cause: single retrieval can't fetch both facts. **Fix:** query decomposition (LLM splits into sub-questions), iterative/agentic retrieval, knowledge graph for entity relations, or SQL tool for structured data. **Trade-off:** more LLM calls = more latency and cost.

11. Cost of RAG app exploded this month

Isolate: cost per request × requests; break down tokens: input (context!) vs output; which feature/user/tenant. **Causes:** context size grew (k raised, longer chunks, chat history unbounded); retries loop; one tenant / bot abusing; agent loops; more expensive model in use. **Fix:** token budget per request, summarise history, model routing (cheap model for easy queries), semantic cache, prompt caching, per-user rate limits & quotas, max-iterations on agents. **Prevent:** cost per request dashboard + budget alerts.

12. Different answers in English vs Hindi (or other language) for the same question

Causes: embedding model weak at cross-lingual retrieval; docs only in English; LLM weaker in that language. **Fix:** multilingual embedding model; translate query → retrieve in doc language → answer in user's language; eval per language.

B. LLM apps & agents

13. The model suddenly got worse, and we didn't change anything

Causes: provider updated the model behind an alias (`gpt-x-latest`); our data/docs changed; user traffic mix changed (new customer, new feature); a dependency (tokenizer, SDK default params) changed; system prompt edited via config by another team. **Isolate:** rerun golden eval set now vs. stored results; check model version in response headers/logs; diff prompt/config history. **Fix:** pin dated model versions; version prompts in git; rollback. **Prevent:** daily scheduled eval run against production config; alert on score drop.

14. The model sometimes returns invalid JSON and breaks the downstream service

Fix (layered): use structured output / JSON mode / function calling with schema; validate with a schema (Pydantic); on failure → retry with the error message ("fix this JSON"); fallback default; temperature 0; keep schema small. **Prevent:** parse-failure rate metric; contract tests.

15. Prompt injection: a user made the bot reveal its system prompt / ignore rules

Explain: direct injection (user input) vs **indirect injection** (malicious text inside retrieved doc/web page/email). **Mitigate:** never trust LLM output for authorization; least-privilege tools (read-only by default, human approval for actions like send/delete/pay); separate system vs. untrusted content clearly; input/output classifiers; don't put secrets in prompts; allow-listed tool arguments. **Key line:** "*I assume the model WILL be tricked, so I design the system so a tricked model can't do damage.*"

16. Agent gets stuck in a loop calling the same tool

Causes: tool returns error the model doesn't understand; ambiguous tool descriptions; no stopping criterion. **Fix:** max-steps + max-cost limits, detect repeated identical calls, clear error messages from tools, better tool descriptions, planner/executor split, return partial result + escalate to human.

17. LLM provider is down / rate limiting us (429s)

Mitigate: retries with exponential backoff + jitter; **fallback to a second provider/model** via an LLM gateway; queue non-urgent requests; graceful degradation ("Answers delayed"). **Long-term:** multi-provider gateway, quota management per feature, circuit breaker, self-hosted backup model for critical paths.

18. Chatbot forgets earlier parts of a long conversation

Cause: history truncated to fit context window. **Fix:** rolling summary of older turns + last N turns verbatim; store facts in memory store (vector or key-value) and retrieve relevant ones; per-session state in Redis.

19. How do you know your LLM feature is actually good? (evaluation)

Answer layers: (1) offline golden set with references → exact-match/semantic similarity, LLM-as-judge with rubric (calibrated against human labels), (2) RAG-specific: context recall/precision, faithfulness, answer relevance, (3) online: thumbs up/down, resolution rate, escalation-to-human rate, A/B tests, (4) safety: red-team set, toxicity, PII leakage. **Watch out:** LLM judges are biased (position, verbosity, self-preference) → randomize order, validate against humans.

20. Outputs contain PII (emails, phone numbers) of other customers

Mitigate: output PII filter now. **Root causes:** PII in indexed docs, or in fine-tuning data (model memorized it), or shared cache/memory across users. **Fix:** PII redaction at ingestion, tenant isolation, cache scoping, data retention policies, remove from training data and retrain.

21. Fine-tuned model is great on our eval but users say it's worse at general tasks

Cause: catastrophic forgetting / overfitting to narrow data; eval set too narrow; eval leak (test examples in train). **Fix:** mix general data in fine-tuning, LoRA with lower rank/lr, fewer epochs, evaluate on general benchmark too, deduplicate train/test. Ask: "Did we even need fine-tuning, or would RAG/prompting work?"

22. Responses are too long / verbose and users complain

Fix: explicit length instruction + examples, `max_tokens`, output format constraints, eval metric for length, post-processing summarizer as last resort.

C. ML models in production

23. Model accuracy was 95% offline but 70% in production

Causes (ranked): 1. **Train/serve skew**: features computed differently online vs offline (different code, units, defaults for nulls, timezones). 2. **Data leakage** in training: a feature that wouldn't be available at prediction time (e.g., "refund_issued" when predicting fraud). Offline score was fake. 3. **Distribution shift**: production population differs from training data. 4. Wrong split: random split on time-series data (future leaked into training). 5. Preprocessing mismatch: different tokenizer/scaler version, label encoding order. **Isolate:** log production features → recompute the same rows offline → compare feature-by-feature. Check feature importance for suspiciously strong features. **Fix:** single feature definition shared by training & serving (**feature store**), time-based splits, point-in-time correct joins. **Prevent:** skew monitoring, shadow deployment before launch.

24. Model performance slowly degrades over months

Cause: data drift (input distribution changes) or **concept drift** (relationship between input and label changes, e.g., fraud patterns evolve). **Detect:** monitor input feature distributions (PSI, KL divergence), prediction distribution, and actual performance when labels arrive (often delayed!). **Fix:** scheduled retraining or drift-triggered retraining, sliding window training data, online learning for fast-moving domains.

25. Model's predictions suddenly all the same (e.g., everything "not fraud")

Causes: upstream feature broken → all nulls → default value; a feature service down and returning fallback; model file corrupted/wrong version deployed; input scaling bug. **Isolate:** check prediction distribution & feature null rates around the time it started; correlate with deploys. **Fix:** rollback; fail loudly (alert) instead of silently defaulting. **Prevent:** input validation, alerts on prediction distribution, canary deploys.

26. Labels arrive 30 days late (e.g., loan default, chargebacks). How do you monitor?

Answer: use proxy signals until labels arrive: input drift, prediction drift, early indicators (first payment missed), human-reviewed sample. Then backfill real performance when labels land.

27. Fraud model: 99.9% accuracy but catches no fraud

Cause: class imbalance. Predicting "not fraud" always is 99.9% accurate. **Fix:** use precision/recall, PR-AUC, recall at fixed false positive rate; class weights, resampling, threshold tuning based on business cost (cost of missed fraud vs. cost of blocking good user).

28. New model won the offline evaluation but lost the A/B test

Causes: offline metric ≠ business metric; offline data biased by old model (it only saw items the old model showed, which is **selection bias / feedback loop**); latency increase hurt engagement; novelty effects; A/B test underpowered. **Fix:** align offline metric to online goal, counterfactual/off-policy evaluation, check guardrail metrics (latency), run test longer/with power analysis.

29. Model is biased against a user group

Isolate: slice metrics by group (FPR, FNR per group). **Causes:** under-representation in training data, proxy features (zip code ≈ race), biased historical labels. **Fix:** rebalance/collect data, remove proxies, fairness constraints, per-group thresholds (with legal review), human review for high-stakes.

30. Retraining pipeline produced a worse model and it auto-deployed

Fix: never auto-deploy without gates: eval against current champion on fixed holdout + recent data, sanity checks (prediction distribution), canary with automatic rollback. Model registry with champion/challenger.

31. Can't reproduce last month's model

Need: version everything: code (git), data snapshot (DVC/Delta/lakehouse time travel), features, hyperparameters, random seeds, library versions (Docker image), experiment tracking (MLflow/W&B).

32. Model works in batch, but real-time version is different

Cause: batch features use full-day aggregates; real-time uses partial windows; different code paths. **Fix:** same feature definitions (feature store, streaming computation), point-in-time features, compare outputs on the same entities.

D. Data & pipelines

33. A daily pipeline silently produced half the usual rows

Fix/prevent: data quality checks (row counts vs expected range, null rates, schema, freshness) that **fail the pipeline** (Great Expectations / dbt tests), alerting, idempotent reruns.

34. Upstream team changed a column type and broke our model

Fix: data contracts (schema agreed and validated at boundary), schema registry, version columns instead of mutating, CI tests on upstream changes.

35. Duplicate events in Kafka caused double-counting

Explain: most queues give *at-least-once* delivery. **Fix:** idempotent consumers (dedupe by event_id with TTL store), idempotent writes (upsert by key), exactly-once semantics where supported.

36. Training data has labeling errors

Fix: inter-annotator agreement, label review on disagreements, confident learning (find examples the model strongly disagrees with), gold questions for labelers.

37. Backfill job overloaded the production database

Fix: read from replica/warehouse not primary, rate-limit batch jobs, run off-peak, separate OLTP and analytics.

38. Embedding / ingestion job takes 3 days for the full corpus

Fix: parallelize with workers on a queue, batch embedding calls, GPU batching, incremental updates only (hash content → skip unchanged), checkpoint & resume.

E. Serving, infra & scale

39. Model latency p99 spikes every few minutes

Causes: garbage collection pauses, autoscaling cold starts (model loading), noisy neighbour on GPU, batch jobs sharing the cluster, large inputs, cache expiry thundering herd. **Fix:** keep warm replicas / min instances, preload models, dynamic batching with max wait, separate pools for batch vs online, timeout + fallback.

40. Traffic goes 10× during a product launch

Plan: horizontal autoscaling on stateless servers, queue + backpressure, rate limiting & priority tiers, cache hot results, pre-warm capacity, graceful degradation (smaller model, cached answers), load test beforehand.

41. GPU cost is too high for LLM inference

Fix: continuous batching (vLLM/TGI), quantization (INT8/INT4), smaller distilled model, routing easy queries to small model, KV cache/prefix caching, spot instances for batch workloads, autoscale to zero for low traffic, right-size GPUs.

42. One replica returns different predictions than others

Cause: mid-rollout with mixed versions, stale model file on one node, different hardware/library version, local cache. **Fix:** immutable versioned model artifacts, readiness checks that verify model hash, version in response header.

43. Rolled out a new model, error rate jumped: what now?

Answer: rollback first (that's why we canary + keep previous version warm), then debug from logs offline. "Mitigate first, investigate second."

44. Vector DB is the bottleneck at 50M vectors

Fix: shard by tenant/hash, tune HNSW (M, ef), use IVF-PQ/quantization to cut memory, metadata pre-filter partitions, replicas for read throughput, tiered storage (hot/cold).

45. A downstream feature service is slow, making all predictions slow

Fix: timeouts + defaults for non-critical features, circuit breaker, cache features, precompute batch features, async calls in parallel.

46. Users in another region experience high latency

Fix: deploy in multiple regions, route by geo-DNS, replicate read data/indexes, edge cache, smaller models at edge. Trade-off: consistency of data across regions.

F. Recommenders, search & ranking

47. Recommendations show the same popular items to everyone

Cause: popularity bias + feedback loop; cold-start fallback used too often; candidate generation too narrow. **Fix:** diversity/novelty in re-ranking, exploration (bandits, epsilon-greedy), personalization features, debias training with exposure weighting.

48. New users / new items get bad recommendations (cold start)

Fix: content-based features (item text/image embeddings), onboarding questions, popularity within segment, contextual features (location, device), explore budget for new items.

49. Search results are relevant but click-through is dropping

Isolate: UI change? latency? new content mix? query distribution shift? **Fix:** check position bias in training data, freshness features, A/B test ranker, analyze zero-click queries.

50. Users found the recommender is gaming engagement (clickbait rising)

Cause: optimizing a proxy (clicks) not the true goal (satisfaction). **Fix:** multi-objective ranking (dwell time, return visits, hides/reports as negatives), quality classifiers, long-term holdout metrics.

Final drill: 10 rapid-fire questions (answer in 30 seconds each)

1. Same chunks retrieved, different answers → *generation randomness / prompt diff*

2. Different chunks retrieved for the same query → *ANN non-determinism, query rewrite, index version, filters*
3. Great offline, bad online → *train/serve skew, leakage, drift*
4. Slow degradation → *drift → monitor + retrain*
5. Sudden drop → *something changed → correlate with deploys/data*
6. All predictions identical → *broken feature upstream*
7. Provider down → *retry, fallback provider, circuit breaker*
8. Answer is outdated → *ingestion lag, old chunks not deleted, cache*
9. Data leak between users → *security incident, pre-filter ACLs, scoped cache*
10. Cost spike → *token breakdown, context growth, loops, abuse → budgets + routing + cache*

05. Case Studies: Full Interview Answers

Each case follows the same 7-step structure you should use in the interview (see 06-interview-playbook.md):

> **1. Requirements → 2. Estimates → 3. API → 4. High-level design → 5. Data model → 6. Deep dives → 7. What-ifs & trade-offs**

AI/ML cases

1. Enterprise RAG assistant (multi-tenant) (most asked)
2. Customer-support AI agent
3. LLM gateway / internal LLM platform
4. Recommendation system (video/product feed)
5. Real-time fraud detection
6. Semantic search for e-commerce
7. Content moderation system
8. Document AI pipeline (invoices/forms extraction)

Classic cases (also asked to AI engineers)

9. URL shortener
10. Chat application
11. Rate limiter

---

1. Enterprise RAG assistant (multi-tenant)

> "Design a ChatGPT-like assistant that answers questions over each company's internal documents."

1. Requirements - Functional: upload/connect docs (PDF, Confluence, Drive), ask questions in chat, answers with citations, follow-up questions, respect document permissions, admin sees usage. - Non-functional: answers grounded (low hallucination), p95 time-to-first-token < 2 s, strict tenant isolation, docs searchable within 15 min of update, 99.9% availability, cost control.

2. Estimates: 500 companies × 2,000 users × 10 questions/day = 10M/day ≈ 115 QPS avg, ~350 peak. 500 tenants × 200k docs × 20 chunks = 2B chunks → 2B × 3 KB (768-dim fp32) ≈ 6 TB → shard + quantize. Tokens: 10M × 4k ≈ 40B tokens/day → **cost is the #1 concern** → caching, routing.

3. API

```
POST /v1/sources                    connect a source / upload file → returns job_id
GET  /v1/jobs/{id}                  ingestion status
POST /v1/conversations              create chat
```

```
POST /v1/conversations/{id}/messages { "text": "..." } → SSE stream of tokens + citations
POST /v1/messages/{id}/feedback      { "rating": -1, "comment": "..." }
```

4. High-level design

```
                      ┌──────────────── INGESTION (async) ────────────────┐
Sources/connectors → │ Queue → Parser/OCR → Chunker → Embedder (GPU batch) │ → Vector DB
(sharded by tenant)
  (webhooks / CDC)   │                                  → Keyword index    │ → Object store
(raw docs)
                      └─────────────────────────────────────────────────────┘ → Metadata DB
(Postgres: docs, ACLs, versions)

User → API gateway (auth, tenant, rate limit) → Chat service
          Chat service → Conversation store (history)
                       → Query rewriter (small LLM)
                       → Retriever (hybrid, ACL pre-filter by tenant + groups)
                       → Reranker
                       → Context builder (token budget)
                       → LLM gateway (routing, fallback, caching) → LLM providers / self-hosted
vLLM
                       → Output guardrails + citation verification
                       → SSE stream to user
          All steps → Tracing/logging (versions, chunk IDs, tokens, cost) → Eval & monitoring
```

5. Data model - `documents(doc_id, tenant_id, source, title, version, updated_at, acl_groups[], status)` - `chunks(chunk_id, doc_id, tenant_id, text, section_path, embedding, acl_groups[], index_version)` - `conversations(conv_id, user_id, tenant_id)` / `messages(msg_id, conv_id, role, text, citations, model_version, prompt_version, index_version, latency, tokens)`

6. Deep dives - **Tenant isolation:** namespace per tenant in vector DB (or index per large tenant); every query has mandatory `tenant_id` filter enforced server-side; ACL groups as pre-filter; cache keys include tenant + permission scope. - **Freshness:** connectors push change events → re-ingest only changed docs (content hash) → delete old chunks by doc_id → insert new. Full re-embeds (model change) build a new index version and swap alias. - **Quality:** hybrid search + rerank; contextual chunk headers; "answer only from context with citations"; verify citations; say "I don't know" when retrieval confidence is low. - **Latency:** streaming, parallel retrieval, rerank only top-30, small model for query rewrite, prompt caching. - **Cost:** semantic cache for repeated questions per tenant, model routing (simple → small model), context budget, per-tenant quotas. - **Evaluation:** per-tenant golden sets from real queries; retrieval recall@k, faithfulness, consistency; thumbs-down sampling to human review; CI eval gate on prompt/model changes.

7. What-ifs → see scenarios 1-12 in 04-what-if-scenarios.md. Be ready for: inconsistent answers, no answer, data leakage between tenants, outdated answers, embedding model migration, provider outage.

---

2. Customer-support AI agent

> "Design an AI agent that resolves support tickets: answers questions, checks order status, issues refunds."

- **Requirements:** chat + email channels, answer FAQs (RAG over help center), take actions via tools (order lookup, refund ≤ $50), hand off to human, audit trail. Non-functional: safe actions, < 3 s responses, measurable resolution rate.
- **Design:**

```
Channel adapters → Conversation service → Intent classifier/router
   ├─ FAQ → RAG pipeline
   ├─ Action → Agent (LLM + tools: get_order, create_refund, update_address)
   │            Tools behind a Tool API with authZ (acts AS the customer, not admin), limits,
idempotency keys
   │            Risky actions (refund > $50) → human approval queue
   └─ Escalation → Human agent console (with AI summary of conversation)
State: conversation store + ticket system;  Observability: trace every tool call
```

- **Deep dives:** idempotent refunds (idempotency key per ticket), policy checks done in code not in prompt, max-steps limit, prompt-injection defense (user can't make it refund $5,000 because the tool enforces limits), evaluation via simulated customers + replay of historical tickets, metrics: resolution rate, CSAT, escalation rate, wrong-action rate.
- **What-ifs:** agent loops (#16), injection (#15), tool API down (fallback to human), duplicate refund (idempotency).

---

3. LLM gateway / internal LLM platform

> "Many teams in our company want to use LLMs. Design a platform."

- **Requirements:** one API for many models/providers, auth per team, quotas & cost attribution, logging/redaction, fallback, caching, eval & prompt management.
- **Design:**

```
Teams' apps → LLM Gateway (stateless, autoscaled)
                ├─ AuthN/Z (API keys per team), PII redaction
                ├─ Rate limiter (tokens/min per team, Redis token bucket)
                ├─ Router (model by policy/cost/latency; fallback on 429/5xx; circuit breaker)
                ├─ Cache (exact + semantic)
                ├─ Providers: external APIs, self-hosted vLLM cluster (GPU autoscaling)
                └─ Async logging → Kafka → warehouse (usage, cost per team) + trace store
Prompt registry (versioned prompts) | Eval service (run suites on new model/prompt versions)
```

- **Trade-offs:** central gateway adds a hop (~ms) and is a critical dependency → deploy multi-AZ, keep it thin. Streaming support required end to end.

---

4. Recommendation system

> "Design the home feed recommendations for a video app with 100M users."

- **Requirements:** personalized feed, fresh content, < 200 ms, optimize long-term engagement (watch time, not just clicks), diversity.
- **Estimates:** 100M DAU × 20 feed loads = 2B/day ≈ 23k QPS avg, ~70k peak. 10M candidate videos.
- **Design (multi-stage funnel):**

```
Request → Candidate generation (≈1000 items, parallel sources):
              ├─ Two-tower model: user embedding → ANN search over video embeddings
              ├─ Collaborative filtering / co-watch
              ├─ Trending / fresh / followed creators
          → Ranking model (≈1000 → 100): deep model with user, item, context features; multi-
task (P(click), P(watch>50%), P(like), P(hide))
          → Re-ranking (business rules): diversity, freshness, dedupe, remove watched, policy
filters
          → Feed
Features: online feature store (Redis) ← streaming (Kafka/Flink: recent watches) + batch (Spark
daily)
Training: logged impressions + engagement → daily/continuous training → registry → canary → A/B
test
```

- **Deep dives:** cold start (#48), popularity bias & exploration (#47), position bias in logged data, multi-objective scoring (`score = w1·P(watch) + w2·P(like) − w3·P(hide)`), precomputing for inactive users (batch) vs real-time for active.
- **What-ifs:** offline win/online loss (#28), clickbait (#50), feature pipeline lag → stale recommendations.

---

5. Real-time fraud detection

> "Score every card transaction for fraud in < 100 ms."

- **Requirements:** score at authorization time, block/allow/review decision, explainability for analysts, adapt to new fraud patterns. 10k TPS peak.
- **Design:**

```
Payment service → Fraud API (p99 < 100ms)
     ├─ Rules engine (hard blocks: blacklisted card, impossible velocity)
     ├─ Feature fetch: online store (Redis): real-time aggregates (txns in last 5 min, new
device?, distance from last txn)
     │      ↑ Streaming pipeline (Kafka → Flink) updates aggregates per card/user/merchant
     ├─ Model (GBDT e.g. XGBoost: fast, strong on tabular; maybe + graph features for fraud
rings)
     └─ Decision: score → thresholds (allow / step-up auth (OTP) / block / manual review queue)
Labels: chargebacks (delayed 30-90 days), analyst reviews → training data (point-in-time
features)
```

- **Deep dives:** class imbalance (#27), delayed labels (#26), threshold set by business cost, **fail-open vs fail-closed** if fraud service is down (usually fail-open with rules-only fallback for small amounts; state the trade-off), adversarial drift (#24), explainability (SHAP for analysts).

---

6. Semantic search for e-commerce

> "Users type 'comfortable running shoes for flat feet'. Return relevant products."

- **Design:** query understanding (spell-fix, category/attribute extraction with an LLM or classifier: category=running shoes, feature=arch support) → **hybrid retrieval** (BM25 on title/attributes + vector search on product embeddings) with filters (in stock, price, size) → learning-to-rank model (features: text relevance, popularity, CTR, conversion, price, personalization) → re-rank (diversity, sponsored slots).
- Indexing: product catalog changes → CDC → update search index (Elasticsearch/OpenSearch with vector support).
- Evaluation: nDCG on labeled query-product pairs, online CTR / add-to-cart / zero-result rate.
- What-ifs: zero results (#2 analog: query relaxation, fallbacks), position bias (#49), new products cold start (#48).

---

7. Content moderation system

> "Detect harmful posts (text + images) on a social platform with 50M posts/day."

- **Design:**

```
Post created → sync fast check (hash match against known-bad content, cheap text classifier) →
publish or hold
             → async deeper models via queue (multimodal classifiers / LLM with policy prompt)
             → score per policy category → auto-remove (high confidence) | human review queue
(uncertain) | allow
User reports → priority boost into review queue
Human decisions → labels → retraining; appeals flow
```

- **Deep dives:** precision vs recall per category (CSAM: maximize recall; satire: precision), cost (cheap model filters, expensive model only for uncertain), multilingual, adversarial evasion (leetspeak, text in images → OCR), reviewer wellbeing & throughput, latency of the async path.

---

8. Document AI pipeline

> "Extract fields from 1M invoices/month (vendor, date, total, line items)."

- **Design:** upload → object storage → queue → classify doc type → OCR/layout model (or multimodal LLM) → field extraction (structured output with JSON schema) → validation rules (totals sum, date

formats, vendor in master data) → confidence score → auto-approve or human review UI → corrections stored as training data → ERP via API.
- **Estimates:** 1M/month ≈ 0.4/s average, so throughput is easy; focus on accuracy, cost per document, and review workload.
- **What-ifs:** new invoice layout breaks extraction (drift → human review rises → retrain/few-shot examples), scanned low-quality docs (pre-processing, OCR confidence), duplicate invoices (hash + business key dedupe), LLM returns invalid JSON (#14).

---

9. URL shortener

- **Req:** shorten URL, redirect fast, 100M new URLs/month, read-heavy (100:1).
- **Estimates:** writes ≈ 40/s, reads ≈ 4k/s; 5 years × 1.2B × 500 B ≈ 3 TB.
- **Design:** `POST /shorten` → ID generator (Snowflake or counter ranges per server) → **base62 encode** (7 chars = 62^7 ≈ 3.5T) → store `code → long_url` in key-value DB (DynamoDB/Cassandra). `GET /{code}` → cache (Redis) → DB → **301/302 redirect**. Analytics via async events to Kafka.
- **Trade-offs:** hash vs counter (collisions vs coordination), 301 (cached by browser, less analytics) vs 302.

10. Chat application

- **Req:** 1:1 & group chat, online status, delivery receipts, history.
- **Design:** clients hold **WebSocket** connections to chat servers; connection registry (Redis: user → server); message → chat server → persist (Cassandra, partitioned by conversation_id, sorted by time) → route to recipient's server via pub/sub; offline users → push notification; message IDs time-sortable (Snowflake) for ordering.
- **Deep dives:** at-least-once delivery + client dedupe, group fan-out, presence via heartbeats, end-to-end encryption.

11. Rate limiter

- **Design:** token bucket per (user, endpoint) stored in Redis with atomic Lua script; at API gateway; return 429 + `Retry-After`. For LLM: limit tokens/min and concurrent requests.
- **Trade-offs:** centralized Redis (accurate, extra hop) vs local in-memory (fast, approximate); fail-open if Redis down.

06. Interview Playbook & Cheat Sheet

A. The 45-minute "Design X" script

Time	Step	What you say / do
0-5 min	**Requirements**	"Let me clarify scope." Functional (3-5 features), non-functional (scale, latency, accuracy, cost, privacy). Write them down. Ask: *who are the users, how many, what matters most?*
5-8 min	**Estimates**	QPS, storage, tokens/cost. Then **say what it means**: "So cost, not QPS, is the bottleneck."
8-12 min	**API**	3-5 endpoints. Mention streaming for LLMs.
12-25 min	**High-level design**	Draw boxes & arrows. Walk one request end to end. For ML: show **offline** (data, training/ingestion) and **online** (serving) paths separately.
25-38 min	**Deep dives**	Pick 2-3 hardest parts (or ask: "Which part would you like me to go deeper on?"). Data model, retrieval/ranking, scaling, evaluation.
38-45 min	**What-ifs, trade-offs, monitoring**	Failure modes, bottlenecks, how you'd evaluate & monitor, what you'd do with more time.

B. The "What if X happens?" script

Use **C.L.H.I.F.P.** from 03-debugging-framework.md: 1. **Clarify** the symptom (who / when / since when / what changed). 2. **Layer map**: draw the pipeline. 3. **Hypotheses** ranked by likelihood. 4. **Isolate**: cheapest test first; find the first layer where output differs. 5. **Fix**: mitigation now + root-cause fix later. 6. **Prevent**: evals, monitoring, alerts, versioning.

C. What interviewers actually score

Points in your favor: - You asked clarifying questions before designing. - You stated assumptions out loud. - You drove the conversation (didn't wait to be led). - You explained **trade-offs** ("I chose X over Y because..., the cost is..."). - You covered evaluation and monitoring (very important for ML roles). - You started simple, then scaled ("for v1 I'd use pgvector; at 100M vectors I'd move to..."). - You handled pushback calmly ("Good point. In that case I'd change...").

Points against you: - Jumping into a model architecture before understanding the problem. - Listing buzzwords without reasons. - Long silences. - Over-engineering a system that only gets 5 QPS. - Ignoring the cost and latency of LLM calls.

D. Trade-off sentences (memorize the shape)

- "**X gives us** lower latency, **at the cost of** staleness, **which is acceptable here because** ..."
- Latency vs quality: bigger model / reranker / more context = better but slower and costlier.

- Freshness vs cost: real-time ingestion vs hourly batch.
- Consistency vs availability: payments → consistency; feeds → availability.
- Precision vs recall: depends on the cost of each error type.
- Build vs buy: managed vector DB/LLM API (speed) vs self-host (control, cost at scale).
- RAG vs fine-tune: knowledge vs behavior.
- Determinism vs creativity: temperature 0 for factual Q&A, higher for brainstorming.

E. One-page cheat sheet

Building blocks: LB · API gateway · cache (Redis) · CDN · SQL · NoSQL · object storage · queue (SQS/Rabbit) · stream (Kafka) · search (Elastic) · vector DB · feature store · model registry · model server · LLM gateway · monitoring/tracing.

Scale moves: stateless + horizontal · cache · read replicas · shard · async queue · CDN · batch on GPU · autoscale · multi-region.

Reliability moves: redundancy · timeouts · retries with backoff + jitter · circuit breaker · fallback/degradation · idempotency · canary + rollback · rate limits · DLQ.

ML moves: feature store (no skew) · point-in-time joins · time-based splits · shadow → canary → A/B · drift monitoring · champion/challenger · version code+data+model.

LLM/RAG moves: hybrid search · rerank · good chunking + contextual headers · query rewrite · ACL pre-filter · citations · "I don't know" threshold · temp 0 + pinned model for consistency · versioned index + alias swap · semantic/prompt cache · model routing · streaming · token budgets · guardrails · evals in CI (recall@k, faithfulness, consistency).

Numbers: 1M/day ≈ 12 QPS · 1 day ≈ 10^5 s · 1 token ≈ 4 chars · 1536-d fp32 vector ≈ 6 KB · 7B fp16 ≈ 14 GB · Redis ~1 ms · LLM TTFT ~0.5 s · 99.9% = 8.8 h/yr down.

F. Common questions to prepare answers for

1. Design a RAG system for internal documents. *(05 #1)*
2. Design a chatbot/agent for customer support. *(05 #2)*
3. How would you evaluate an LLM application? *(02 §10)*
4. How do you reduce hallucinations? *(04 #3)*
5. RAG vs fine-tuning: when to use each? *(02 §12)*
6. How do you reduce LLM latency / cost? *(04 #6, #11, 02 §8)*
7. Design a recommendation system. *(05 #4)*
8. Design fraud detection. *(05 #5)*
9. Model works offline but not in production. Why? *(04 #23)*
10. How do you detect and handle drift? *(04 #24)*
11. How do you deploy a new model safely? *(02 §5, 04 #43)*
12. Same question, different answers. Why? *(03 worked example)*
13. How to handle multi-tenant data isolation? *(04 #7)*
14. How to serve a 70B model? *(02 §8, 04 #41)*
15. How do you defend against prompt injection? *(04 #15)*

G. Your 2-week mock interview plan

Day	Task
1, 3, 5, 7, 9, 11, 13	One full 45-min design (rotate 05 cases), **recorded**. Watch back: Did I clarify? Estimate? Trade-offs? Monitoring?
2, 4, 6, 8, 10, 12, 14	8 scenario questions from 04, 3 min each, out loud, timed
Daily	10 min: redraw the RAG layer map and ML lifecycle from memory

Practice partners: a friend, a peer engineer, or ask an AI to play interviewer: "*Act as a senior ML interviewer. Ask me a system design question, then push back with 'what if' failure scenarios. Score me on structure, trade-offs, and depth.*"

H. If you blank out in the interview

Say: "*Let me take a step back and walk through the request path.*" Then draw the pipeline. Your layer map will give you the next thing to say.

07. Machine Learning Fundamentals (Theory Round)

Format: **Q → short answer you can say in 30-60 seconds.** Then "what if" scenarios at the end. Practice: cover the answer, say it out loud, compare.

Contents

1. Core concepts
2. Bias, variance, regularization
3. Linear & logistic regression
4. Trees & ensembles
5. Other classic algorithms
6. Unsupervised learning
7. Evaluation & metrics
8. Data preparation & feature engineering
9. What-if scenarios (ML)

---

1. Core concepts

Q: Supervised vs unsupervised vs self-supervised vs reinforcement learning? Supervised: learn from labeled (x, y). Unsupervised: find structure without labels (clustering, dimensionality reduction). Self-supervised: create labels from the data itself (predict next token, masked word). This is how LLMs are pretrained. RL: an agent learns by trial and error from rewards.

Q: Classification vs regression? Predict a category vs a continuous number.

Q: Parametric vs non-parametric? Parametric has a fixed number of parameters (linear regression). Non-parametric grows with data (kNN, decision trees can).

Q: Generative vs discriminative? Discriminative models P(y|x) (logistic regression). Generative models P(x, y) or P(x) (Naive Bayes, GPT, diffusion).

Q: What is a loss function vs cost vs objective? Loss: error for one example. Cost: average over dataset. Objective: what we optimize (cost + regularization).

Q: What is gradient descent? Iteratively move parameters opposite to the gradient of the loss: `w = w − lr · ∇L`. Batch (all data), stochastic (one sample), mini-batch (common).

Q: Why train / validation / test split? Train to fit, validation to tune hyperparameters/early stop, test to estimate final performance once. Using test for tuning = leakage → optimistic results.

Q: What is cross-validation? Split data into k folds; train on k−1, validate on 1, rotate; average. Gives a more reliable estimate with small data. Use **stratified** k-fold for imbalance, **time-series split** for temporal data, **group k-fold** when same user appears multiple times.

Q: Curse of dimensionality? In high dimensions data becomes sparse and distances become similar, so distance-based methods (kNN) degrade and you need far more data.

Q: No Free Lunch theorem? No single algorithm is best for all problems. Choose based on your data and constraints.

2. Bias, variance, regularization

Q: Bias-variance trade-off? Bias = error from too-simple assumptions (underfitting). Variance = error from sensitivity to training data (overfitting). Total error = bias^2 + variance + noise. More complexity → lower bias, higher variance.

Q: How do you detect overfitting / underfitting? | | Train error | Validation error | |---|---|---| | Underfitting | high | high | | Overfitting | low | high (big gap) | | Good fit | low | low, small gap |

Q: How to fix overfitting? More data, data augmentation, simpler model, regularization (L1/L2, dropout), early stopping, feature selection, ensembling/bagging, cross-validation for tuning.

Q: How to fix underfitting? More complex model, more/better features, less regularization, train longer.

Q: L1 vs L2 regularization? L1 (Lasso) adds |w| → pushes weights to exactly zero → feature selection. L2 (Ridge) adds w^2 → shrinks weights smoothly, handles correlated features. Elastic Net = both.

Q: Why does L1 create sparsity? Its penalty has a constant gradient (corner at zero in the constraint diamond), so the optimum often lands on an axis where weights are exactly zero.

3. Linear & logistic regression

Q: Assumptions of linear regression? Linearity, independent errors, homoscedasticity (constant error variance), normally distributed errors (for inference), no strong multicollinearity.

Q: What is multicollinearity and why does it matter? Features highly correlated → unstable, uninterpretable coefficients. Detect with VIF; fix with removal, PCA, or Ridge.

Q: How does logistic regression work? Linear score → sigmoid → probability. Trained by minimizing **log loss (binary cross-entropy)**, which is maximum likelihood. Decision boundary is linear.

Q: Why not MSE for logistic regression? With sigmoid, MSE is non-convex and gives weak gradients; log loss is convex and penalizes confident wrong predictions strongly.

Q: R^2 meaning? Fraction of target variance explained by the model. Can be negative on test data if the model is worse than predicting the mean.

4. Trees & ensembles

Q: How does a decision tree split? Choose the feature/threshold that maximizes impurity reduction: Gini or entropy (information gain) for classification, variance/MSE for regression.

Q: Bagging vs boosting? - **Bagging** (Random Forest): train many models in parallel on bootstrap samples, average them → **reduces variance**. - **Boosting** (AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost): train models sequentially, each fixes previous errors → **reduces bias**. More prone to overfitting; needs tuning (learning rate, depth, n_estimators, early stopping).

Q: Why does Random Forest use random feature subsets? To decorrelate trees, so averaging reduces variance more.

Q: Why is XGBoost/LightGBM so popular for tabular data? Strong accuracy, handles missing values, regularization, fast (histogram-based splits), little preprocessing (no scaling needed). Usually beats deep learning on tabular data.

Q: Any caveats with feature importance in trees? Impurity-based importance is biased toward high-cardinality features. Use permutation importance or SHAP.

Q: Stacking? Train a meta-model on predictions of several base models (use out-of-fold predictions to avoid leakage).

5. Other classic algorithms

Q: kNN? Predict using the k closest training points. Needs feature scaling; slow at prediction; suffers in high dimensions. Small k → overfit, large k → underfit.

Q: Naive Bayes? Bayes' rule assuming features are conditionally independent given the class. Fast, good baseline for text (spam). Use Laplace smoothing for unseen words.

Q: SVM? Find the hyperplane with maximum margin. Soft margin (C) allows errors. **Kernel trick** computes similarity in a higher-dimensional space without explicitly mapping (RBF, polynomial).

Q: When would you choose a simple model over a complex one? Small data, need interpretability (finance/health regulations), low latency, strong baseline, easier maintenance.

6. Unsupervised learning

Q: K-means? Pick k centroids, assign points to nearest, recompute centroids, repeat. Assumes spherical, similar-sized clusters; sensitive to init (use k-means++) and scaling. Choose k by elbow method / silhouette score.

Q: K-means vs DBSCAN vs hierarchical? DBSCAN finds arbitrary shapes and noise points, no k needed, but sensitive to eps. Hierarchical builds a dendrogram, good for small data.

Q: PCA? Projects data onto directions of maximum variance (eigenvectors of covariance matrix). Used for compression, visualization, removing correlation. Requires scaling. Linear only.

Q: t-SNE / UMAP? Non-linear dimensionality reduction for visualization. Distances between clusters in t-SNE are not meaningful.

Q: Anomaly detection methods? Statistical (z-score), Isolation Forest, One-class SVM, autoencoders (high reconstruction error = anomaly), density-based.

7. Evaluation & metrics

Q: Confusion matrix terms? TP, FP, TN, FN. - **Precision** = TP / (TP+FP). "Of what I flagged, how much was right?" Care about it when false alarms are costly (spam filter). - **Recall** = TP / (TP+FN). "Of all positives, how many did I catch?" Care about it when misses are costly (cancer, fraud). - **F1** = harmonic mean of precision and recall. - **Accuracy** misleading on imbalanced data.

Q: ROC-AUC vs PR-AUC? ROC plots TPR vs FPR across thresholds; AUC = probability a random positive ranks above a random negative. On heavy imbalance, ROC looks better than it really is, so **PR-AUC** tells you more.

Q: How to choose a threshold? Based on business cost: minimize cost = FP·cost_FP + FN·cost_FN, or fix one constraint ("recall ≥ 95%") and maximize the other.

Q: Calibration? Predicted probability 0.8 should be right 80% of the time. Check with reliability diagram; fix with Platt scaling or isotonic regression. Matters when probabilities are used for decisions/pricing.

Q: Regression metrics? MAE (robust to outliers, interpretable), RMSE (penalizes big errors), MAPE (percent, breaks near zero), R^2.

Q: Ranking metrics? Precision@k, Recall@k, MRR, MAP, nDCG (accounts for position and graded relevance).

Q: Offline metric vs online metric? Offline (AUC on test set) is a proxy. Online (CTR, revenue, retention via A/B test) is what matters. They can disagree.

8. Data preparation & feature engineering

Q: How do you handle missing values? First ask *why* missing (random or meaningful?). Options: drop, impute (mean/median/mode, model-based, KNN), add "is_missing" indicator, use models that handle NaN (XGBoost).

Q: How to handle categorical features? One-hot (low cardinality), ordinal (ordered), target encoding (high cardinality; use out-of-fold encoding to avoid leakage), hashing, learned embeddings.

Q: When do you need feature scaling? Distance/gradient-based models: kNN, SVM, k-means, PCA, linear/logistic regression with regularization, neural nets. Not needed for trees. Fit the scaler on **train only**.

Q: How to handle imbalanced data? Use proper metrics (PR-AUC, recall@precision); class weights; resampling (undersample majority, oversample minority, SMOTE, applied only to training data); threshold tuning; anomaly detection framing; collect more minority data.

Q: What is data leakage? Examples? Information in training that wouldn't be available at prediction time. Examples: scaling on full dataset before split; target encoding without folds; future data in time series; duplicate rows across train/test; a feature created after the outcome (e.g., "account_closed" predicting churn).

Q: Outliers? Investigate first (error or real?). Cap/winsorize, transform (log), robust models/losses (Huber, MAE), or remove if errors.

Q: Feature selection methods? Filter (correlation, mutual info), wrapper (recursive feature elimination), embedded (L1, tree importance), plus domain knowledge.

9. What-if scenarios

Your model has 98% training accuracy but 70% validation accuracy. What do you do?

Overfitting. Check leakage/split first (duplicates? distribution difference between train and val?). Then: regularization, simpler model, more data/augmentation, early stopping, cross-validation, feature selection. Plot learning curves: if val improves with more data → collect data.

Both training and validation accuracy are low.

Underfitting or bad data. Check labels quality, features informativeness, bugs (labels shuffled? wrong target column?). Try a more complex model, better features, less regularization. Compare to a simple baseline (predict majority class). If you can't beat it, the problem is the data, not the model.

Validation score is suspiciously high (99%).

Suspect **leakage** first: target-derived features, duplicates across splits, future info, ID columns correlated with target. Check the top feature importances. If one feature dominates everything, that's a red flag.

Model performs well on validation but poorly on test.

Overfit to validation through heavy tuning, or val/test come from different distributions. Use cross-validation, a fresh holdout, check distributions.

You have only 500 labeled examples.

Simple models + strong regularization, cross-validation, transfer learning / pretrained embeddings, data augmentation, active learning (label the most uncertain), weak supervision, semi-supervised learning, LLM-assisted labeling with human verification.

You have 1% positive class (fraud).

See §8 imbalance. Mention: never report accuracy; tune threshold on business cost; stratified splits.

Features are highly correlated.

Fine for prediction with trees; problematic for interpreting linear coefficients. Use regularization (Ridge), PCA, or drop redundant ones.

Model is accurate but stakeholders don't trust it.

Explainability: SHAP/LIME, feature importance, partial dependence; show examples; compare to simple interpretable baseline; calibrate probabilities; involve domain experts in error analysis.

Model works for most users but fails for one segment.

Slice-based error analysis; check data representation; add segment features or separate model; fairness considerations.

Training takes too long.

Sample data for iteration, fewer features, use LightGBM (fast), early stopping, distributed training, better hardware, cache preprocessing.

Labels are noisy.

Estimate noise (relabel a sample), consensus labeling, robust losses, remove examples model confidently disagrees with (confident learning), more regularization.

Data distribution changed after deployment.

Drift → see 04 #24.

08. Deep Learning (Theory + Training Debugging)

Contents

1. Neural network basics
2. Optimization
3. Regularization & normalization
4. Architectures: CNN, RNN, Transformer
5. Training at scale
6. Generative models
7. What-if: training debugging scenarios (very commonly asked)

---

1. Neural network basics

Q: What is a neuron / perceptron? Weighted sum of inputs + bias, passed through a non-linear activation.

Q: Why do we need non-linear activations? Without them, stacked linear layers collapse into a single linear function, so the network couldn't learn non-linear patterns.

Q: Common activations? | Activation | Notes | |---|---| | Sigmoid | 0 to 1, saturates → vanishing gradients; used for binary outputs | | Tanh | −1 to 1, zero-centered, still saturates | | ReLU | max(0,x); fast, no saturation for x>0; can "die" (always 0) | | Leaky ReLU / PReLU | small slope for x<0 → fixes dying ReLU | | GELU / SiLU (Swish) | smooth; used in Transformers/LLMs | | Softmax | turns logits into a probability distribution (multi-class output) |

Q: Explain backpropagation. Forward pass computes outputs and loss. Backward pass applies the chain rule layer by layer from output to input to compute the gradient of the loss w.r.t. every weight. The optimizer then updates the weights.

Q: Vanishing / exploding gradients? Gradients multiplied through many layers shrink to ~0 (early layers stop learning) or blow up (NaN). Fixes: ReLU-family activations, good initialization (He/Xavier), normalization (BatchNorm/LayerNorm), residual connections, gradient clipping, LSTM/GRU gates for RNNs.

Q: Why not initialize all weights to zero? All neurons would compute the same thing and get the same gradients (symmetry), so they never differentiate. Use Xavier/Glorot (tanh/sigmoid) or He/Kaiming (ReLU).

Q: Loss functions? MSE/MAE (regression), binary cross-entropy (binary), categorical cross-entropy (multi-class), focal loss (imbalance, detection), contrastive / triplet / InfoNCE (embeddings), KL divergence (distillation, VAEs).

Q: Why cross-entropy with softmax? Gradient becomes simply (predicted − true), stable and strong; corresponds to maximum likelihood.

2. Optimization

Q: SGD vs Momentum vs Adam? - SGD: noisy updates, may generalize well, needs tuned LR. - Momentum: accumulates velocity → faster, smoother through ravines. - RMSProp: per-parameter LR scaled by recent gradient magnitude. - **Adam**: momentum + RMSProp + bias correction; default choice. **AdamW**: decoupled weight decay, the standard for Transformers.

Q: Learning rate is the most important hyperparameter. How do you pick it? LR range test (increase LR, watch loss), then use **warmup + decay** schedule (cosine, step, linear decay). Too high → diverge/NaN; too low → slow/stuck.

Q: Why warmup? Early in training Adam's statistics are unreliable and weights random; large updates can destabilize (especially Transformers). Warmup ramps LR up gradually.

Q: Batch size effects? Larger batch: faster per epoch on GPUs, smoother gradients, may generalize slightly worse, needs larger LR (linear scaling rule). Limited by memory → gradient accumulation simulates large batches.

Q: Local minima in deep learning? In high dimensions most critical points are saddle points; SGD noise and momentum help escape. Many minima are similarly good.

3. Regularization & normalization

Q: Dropout? Randomly zero activations during training (rate p) → prevents co-adaptation, acts like ensemble. Off at inference (activations scaled accordingly).

Q: BatchNorm vs LayerNorm? BatchNorm normalizes each feature across the batch. It works well for CNNs, depends on batch size, and behaves differently in train vs eval (running stats). LayerNorm normalizes across the features of each sample. It doesn't depend on the batch and is used in Transformers/RNNs. RMSNorm: simpler LayerNorm used in modern LLMs.

Q: Other regularizers? Weight decay, data augmentation, early stopping, label smoothing, mixup/cutmix, stochastic depth.

Q: Why use residual (skip) connections? `output = F(x) + x` gives gradients a direct path backward → enables very deep networks (ResNet, Transformers) and makes it easy to learn identity.

4. Architectures

CNNs

Q: Why convolutions for images? Local connectivity + weight sharing → far fewer parameters, translation equivariance, hierarchical features (edges → textures → objects). **Q: Output size formula?** `(W - K + 2P) / S + 1`. **Q: Pooling?** Downsamples, adds small translation invariance. Max vs average. Global average pooling before classifier. **Q: 1×1 convolution?** Changes channel count cheaply, mixes channels (used in Inception/ResNet bottlenecks). **Q: Receptive field?** Region of the input that affects one output unit; grows with depth, dilation, stride. **Q: Famous models?** LeNet → AlexNet → VGG (deep 3×3) →

GoogLeNet/Inception → ResNet (skip connections) → EfficientNet (compound scaling) → ViT (Transformers on patches) → ConvNeXt.

RNNs

Q: RNN problem? Sequential (no parallelism), vanishing gradients over long sequences. **Q: LSTM vs GRU?** LSTM: cell state + input/forget/output gates → long-term memory. GRU: fewer gates (update/reset), faster, often similar performance.

Transformers

Q: Explain self-attention. Each token creates a Query, Key, Value. Attention weights = softmax($Q \cdot K^T / \sqrt{d}$), which says how much each token should look at every other token. Output = weighted sum of Values. Captures long-range dependencies in parallel. **Q: Why divide by √d?** Dot products grow with dimension → softmax saturates → tiny gradients. Scaling keeps variance stable. **Q: Multi-head attention?** Several attention heads in parallel, each learns different relationships (syntax, coreference…), concatenated. **Q: Positional encoding?** Attention is order-agnostic, so we add position info: sinusoidal, learned, or **RoPE** (rotary, used in modern LLMs), ALiBi. **Q: Encoder vs decoder vs encoder-decoder?** Encoder-only (BERT): bidirectional, understanding tasks. Decoder-only (GPT, Llama): causal mask, generation. Encoder-decoder (T5, original Transformer): translation/summarization. **Q: Complexity of attention?** $O(n^2)$ in sequence length (time + memory). Solutions: FlashAttention (IO-aware exact), sparse/sliding-window attention, linear attention, KV caching for inference. **Q: Transformer block?** Multi-head attention → add & norm → feed-forward (MLP) → add & norm. Modern: pre-norm, RMSNorm, SwiGLU, RoPE, GQA.

5. Training at scale

- **Mixed precision** (fp16/bf16): faster, less memory; bf16 avoids overflow (no loss scaling needed); fp16 needs loss scaling.
- **Gradient accumulation:** simulate large batch.
- **Gradient checkpointing:** recompute activations in backward → less memory, ~30% slower.
- **Data parallel (DDP), FSDP/ZeRO, tensor parallel, pipeline parallel**: see 02 §3.
- **Transfer learning / fine-tuning:** freeze early layers, train head, then unfreeze with low LR. **PEFT/LoRA**: train small low-rank adapters instead of full weights.
- **Knowledge distillation:** small student learns from teacher's soft probabilities (with temperature).
- **Quantization / pruning** for inference.

6. Generative models

Q: Autoencoder vs VAE? Autoencoder compresses and reconstructs. VAE learns a probabilistic latent space (mean + variance) with KL regularization → can sample new data; uses reparameterization trick. **Q: GAN?** Generator vs discriminator in a min-max game. Problems: mode collapse, unstable training. **Q: Diffusion models?** Learn to reverse a gradual noising process. Start from noise and denoise step by step. Stable, high quality (Stable Diffusion = diffusion in latent space + text conditioning via cross-attention, classifier-free guidance). Slower sampling (fixes: fewer steps, distillation). **Q: Autoregressive models?** Generate one token at a time conditioned on previous (GPT).

---

7. What-if: training debugging scenarios

General debugging recipe (say this first):

> "Start small: overfit a single batch. If the model can't drive loss to ~0 on 1 batch, there's a bug, not a modelling problem. Then check data → model → loss → optimizer → evaluation, one at a time."

Loss is NaN / Inf

Causes: LR too high; exploding gradients; log(0) or division by zero in custom loss; fp16 overflow; bad input data (NaN in features, corrupted images); sqrt/exp of large values. **Fix:** lower LR, gradient clipping, add epsilon to log/division, use bf16 or loss scaling, check data for NaN (`torch.isnan(x).any()`), use `torch.autograd.detect_anomaly()`, use numerically stable built-ins (`CrossEntropyLoss` on logits, not softmax + log).

Loss doesn't decrease at all

Causes: LR too low/high; forgot `optimizer.zero_grad()` or `optimizer.step()`; frozen layers by mistake; labels misaligned with inputs (shuffled separately); wrong loss for task (softmax applied twice); dead ReLUs; data not normalized; bug in data pipeline (all same image). **Fix:** overfit one batch, visualize inputs + labels, check gradients are non-zero per layer, LR range test.

Loss decreases, then suddenly spikes

Causes: LR too high after warmup, bad batch (outlier/corrupt data), exploding gradients, numerical instability in fp16. **Fix:** gradient clipping, lower peak LR, longer warmup, skip/inspect the bad batch, bf16, restart from checkpoint.

Training loss goes down, validation loss goes up

Overfitting. Early stopping, more data/augmentation, dropout, weight decay, smaller model, mixup. Also check val pipeline matches train preprocessing.

Validation is better than training

Usually fine: dropout/augmentation are active only during training, train loss averaged over the epoch while val measured at the end. Also check val set isn't too easy/small or leaked.

Model predicts the same class for every input

Class imbalance (use weights/focal loss), LR too high causing collapse, dead network, labels broken, final activation wrong.

GPU out of memory (OOM)

Reduce batch size + gradient accumulation; mixed precision; gradient checkpointing; shorter sequences; smaller model / LoRA; free unused tensors (`del`, `torch.no_grad()` in eval; if you forget it, activations are

kept in memory); avoid accumulating loss tensors with graph (`loss.item()`); FSDP/ZeRO offload.

Training is very slow / GPU utilization low (30%)

Data loading bottleneck → more `num_workers`, `pin_memory`, prefetching, preprocess offline, faster storage formats; too-small batch; CPU-GPU sync calls (`.item()` every step); use mixed precision, `torch.compile`, larger batch.

Results differ every run

Set seeds (python, numpy, torch, CUDA), deterministic algorithms (`torch.use_deterministic_algorithms`), fixed data order; some GPU ops are inherently non-deterministic. Report mean ± std over multiple seeds.

Model works in notebook but bad after deployment

`model.eval()` not called (dropout/BatchNorm in training mode!); different preprocessing (normalization constants, resize, tokenizer version); different input format (RGB vs BGR, channel order); quantization changed accuracy.

Fine-tuning a pretrained model makes it worse

LR too high (destroys pretrained weights → use 10-100× smaller LR), catastrophic forgetting, too few examples (freeze more layers, use LoRA), mismatched preprocessing vs pretraining.

Accuracy plateaus at a certain level

Check label noise ceiling (human agreement), error analysis on mistakes, more/better data often beats architecture changes, LR schedule (decay), bigger model if underfitting.

BatchNorm issues with batch size 1-2

Statistics unreliable → use GroupNorm/LayerNorm, freeze BN stats, or larger batch via accumulation (doesn't fix BN) / sync BN across GPUs.

09. NLP & LLMs (Theory + Scenarios)

Contents

1. Classic NLP
2. Tokenization
3. Embeddings
4. Transformers & pretrained models
5. How LLMs are trained
6. Using LLMs: prompting, decoding, fine-tuning
7. LLM limitations & evaluation
8. What-if scenarios (NLP/LLM)

(For RAG and LLM system design, see 02 and 04.)

---

1. Classic NLP

Q: Text preprocessing steps? Lowercasing, tokenization, removing punctuation/stopwords, stemming (crude chop: "running"→"run") vs lemmatization (dictionary form: "better"→"good"), normalization. *Note:* for Transformers you mostly skip this and use the model's own tokenizer on raw text.

Q: Bag of Words vs TF-IDF? BoW counts words. TF-IDF = term frequency × inverse document frequency → down-weights common words. Both ignore word order and meaning.

Q: BM25? Improved TF-IDF ranking function (term frequency saturation + document length normalization). Still the strong baseline for keyword search.

Q: n-grams? Sequences of n words. They capture some local order ("not good").

Q: Classic NLP tasks? Text classification, sentiment, NER (named entity recognition), POS tagging, parsing, machine translation, summarization (extractive vs abstractive), question answering, topic modeling (LDA), coreference resolution.

Q: What is NER and how to build it today? Labeling spans like PERSON/ORG/DATE. Token classification with BIO tags using a fine-tuned BERT-style model; or LLM with structured output for low-data cases.

Q: Language model perplexity? exp(average negative log-likelihood per token). Lower = model less "surprised". Comparable only with the same tokenizer.

2. Tokenization

Q: Word vs character vs subword tokenization? Word: huge vocab, out-of-vocabulary problem. Character: long sequences. **Subword** (BPE, WordPiece, Unigram/SentencePiece): frequent words whole, rare words split into pieces → no OOV, compact.

Q: How does BPE work? Start with characters, repeatedly merge the most frequent adjacent pair into a new token until vocab size reached.

Q: Why do tokenizers matter in interviews / production? - Cost and context limits are in tokens. - Non-English languages often use more tokens per word → higher cost, worse performance. - Numbers and code tokenized oddly → arithmetic/spelling weaknesses. - **Mismatched tokenizer between training and serving = garbage outputs.**

3. Embeddings

Q: Word2Vec? Learns word vectors by predicting context words (Skip-gram) or the word from context (CBOW). Similar words → close vectors; analogies (king − man + woman ≈ queen). GloVe: from co-occurrence matrix. FastText: subword n-grams → handles rare words.

Q: Static vs contextual embeddings? Static: one vector per word ("bank" same everywhere). Contextual (BERT, etc.): vector depends on the sentence.

Q: Sentence embeddings? Models trained with **contrastive learning** (pull similar pairs together, push others apart). Examples: Sentence-BERT, E5, BGE, OpenAI/Cohere embeddings. Used for semantic search, clustering, RAG.

Q: Bi-encoder vs cross-encoder? Bi-encoder encodes query and document separately → precompute document vectors → fast retrieval. Cross-encoder reads query + document together → more accurate, slow → used for **reranking** top results.

Q: Cosine similarity vs dot product? Cosine ignores magnitude; equals dot product on normalized vectors. Use what the embedding model was trained with.

4. Transformers & pretrained models

(Attention mechanics in 08 §4.)

Q: BERT vs GPT? | BERT | GPT | |---|---| | Encoder-only, bidirectional | Decoder-only, left-to-right (causal mask) | | Pretrained with masked language modeling (MLM) (+NSP originally) | Pretrained with next-token prediction | | Best for understanding: classification, NER, embeddings | Best for generation, general-purpose LLMs |

Q: T5? Encoder-decoder, every task as text-to-text.

Q: Why did Transformers replace RNNs? Parallel training, better long-range dependencies, scale well with data and compute.

Q: What is the context window and what limits it? Max tokens the model can attend to. Limited by $O(n^2)$ attention cost, KV-cache memory, and positional encoding trained lengths (extended via RoPE scaling).

Q: KV cache? During generation, store Keys and Values of previous tokens so each new token only computes attention for itself → much faster decoding, but memory grows with sequence length × batch.

Q: Mixture of Experts (MoE)? Many expert FFNs, a router activates only a few per token → huge parameter count with lower compute per token. Challenges: load balancing, memory.

5. How LLMs are trained

```
1. Pretraining:              next-token prediction on trillions of tokens (web, books, code).
Learns language + knowledge.
2. SFT (instruction tuning): fine-tune on (instruction, good response) pairs. Learns to follow
instructions/chat format.
3. Preference alignment:     RLHF (reward model + PPO) or DPO (direct preference optimization)
on (chosen, rejected) pairs.
                             Learns helpful, harmless, honest behavior.
(+ reasoning training with RL on verifiable rewards, like math and code, for "thinking" models)
```

Q: RLHF in simple words? Humans compare two answers → train a reward model to predict preference → optimize the LLM with RL (PPO) to maximize reward, with a KL penalty to stay close to the original model (avoid reward hacking).

Q: DPO vs RLHF? DPO skips the reward model and RL loop; directly optimizes the model on preference pairs with a classification-like loss. Simpler, more stable.

Q: Scaling laws? Loss decreases predictably as a power law with model size, data, and compute. Chinchilla: for a compute budget, train smaller models on more data (~20 tokens per parameter) than earlier practice.

Q: Emergent abilities / in-context learning? Large models can learn a task from examples in the prompt without weight updates.

6. Using LLMs

Q: Decoding strategies? | Method | Behavior | |---|---| | Greedy | Always pick top token. Deterministic, can be repetitive | | Beam search | Keep top-k sequences. Good for translation, bland for chat | | Temperature | <1 sharper (more deterministic), >1 more random | | Top-k | sample from k most likely tokens | | Top-p (nucleus) | sample from smallest set with cumulative prob ≥ p | | Repetition/frequency penalty | discourage repeats |

Q: Prompting techniques? Clear instructions + role, few-shot examples, chain-of-thought / reasoning, output format specification (JSON schema), delimiters for untrusted input, self-consistency (sample several, vote), decomposition, ReAct (reason + tool use).

Q: Fine-tuning methods? - **Full fine-tuning:** all weights; expensive, risk of forgetting. - **PEFT:** LoRA (low-rank adapters on attention/MLP weights; trains <1% of params), **QLoRA** (LoRA on 4-bit quantized base → fine-tune big models on one GPU), prefix/prompt tuning, adapters. - **Q: How does LoRA work?** Freeze W; learn $\Delta W = B \cdot A$ with small rank r. At inference merge or swap adapters. Hyperparameters: rank r, alpha, target modules, LR.

Q: When fine-tune vs RAG vs prompt? See 02 §12. One-liner: "*RAG for knowledge, fine-tuning for behavior/format/cost, prompting first always.*"

Q: Structured output? JSON mode / function calling / constrained decoding (grammar) + schema validation + retry.

Q: Small vs large models? Small: cheaper, faster, can be fine-tuned and self-hosted, good for narrow tasks (classification, extraction). Large: better reasoning, generalization, fewer examples needed.

7. LLM limitations & evaluation

Q: Why do LLMs hallucinate? They're trained to produce plausible next tokens, not verified truths; knowledge gaps/cutoffs; training rewarded confident answers; no grounding. Mitigate: RAG with citations, "I don't know" option, lower temperature, verification step, tool use (calculator, search).

Q: Other limitations? Knowledge cutoff, context limits, lost-in-the-middle, sensitivity to prompt wording, weak arithmetic without tools, bias, prompt injection, cost/latency, non-determinism.

Q: How to evaluate LLMs? - Benchmarks (MMLU, GSM8K, HumanEval...) for general capability. Watch out for contamination (test data leaked into training). - **Task-specific eval set** built from real use. This matters most. - Metrics: exact match / F1 (QA), BLEU/ROUGE (translation/summarization; they correlate weakly with quality), BERTScore, pass@k (code), LLM-as-judge with rubrics, human eval, pairwise preference (win rate). - Safety/red-teaming, consistency, latency, cost.

8. What-if scenarios

Your text classifier works on English but fails on Hinglish / code-mixed text.

Data mismatch. Collect & label code-mixed samples, use multilingual model (XLM-R, mBERT, multilingual LLM), transliteration normalization, augmentation via translation, evaluate per language.

Sentiment model misclassifies sarcasm and negation ("not bad at all").

BoW/TF-IDF models ignore order → use contextual models (fine-tuned Transformer); add targeted training examples; error analysis with slices (negation, sarcasm).

Fine-tuned LLM outputs repeat the same phrase endlessly.

Decoding: add repetition penalty, adjust temperature/top-p, set max tokens, stop sequences. Training: check EOS token included in training data (a common bug: the model never learned to stop), data has duplicates, chat template mismatch.

Fine-tuned model ignores the instruction format after training.

Chat template mismatch between training and inference (system/user/assistant tags), loss computed on prompt tokens instead of only response tokens, too few epochs or LR too low.

LLM answers well in testing, but users report it is "dumb" for some queries.

Collect failing examples, categorize (missing knowledge → RAG; reasoning → bigger model / CoT; format → prompt/fine-tune; ambiguity → clarifying questions). Build eval set from real failures.

NER model misses new product names.

Out-of-distribution entities. Gazetteers/dictionaries, active learning on new data, regular retraining, LLM-based extraction with few-shot examples, character/subword features.

Embedding search returns semantically similar but wrong results ("cancel subscription" matches "subscribe").

Embeddings capture topic not intent/negation. Use hybrid with keyword, fine-tune embeddings with hard negatives, add a cross-encoder reranker, intent classifier.

Summarization model produces facts not in the source.

Abstractive hallucination. Faithfulness checks (NLI entailment, QA-based), constrain with extractive-first, lower temperature, fine-tune on faithful data, citations.

Context window is too small for a 300-page document.

Chunk + map-reduce summarization, RAG, hierarchical summarization, long-context model (trade-off: cost, lost-in-the-middle).

LLM gives different answer quality for the same prompt in different languages.

Tokenization inefficiency + less training data in that language. Translate-in/translate-out pipeline, multilingual eval, language-specific few-shot examples.

Model output contains toxic or biased content.

Input/output safety classifiers, system prompt policies, preference tuning with safety data, red-teaming, human review loops, logging and incident process.

Training an LLM: loss stops improving early.

Data quality/duplication, LR schedule, tokenizer issues, too-small model for data (capacity), check for data loading bugs (same shard repeated), monitor gradient norm.

10. Computer Vision

Contents

1. Core tasks
2. Key concepts & models
3. Metrics
4. Data & augmentation
5. Deployment
6. What-if scenarios (CV)

---

1. Core tasks

Task	Output	Typical models
Classification	one label per image	ResNet, EfficientNet, ViT, ConvNeXt
Object detection	boxes + labels	YOLO family, Faster R-CNN, DETR, RT-DETR
Semantic segmentation	class per pixel	U-Net, DeepLab, SegFormer
Instance segmentation	mask per object	Mask R-CNN, SAM (promptable)
Keypoints / pose	body/face points	HRNet, OpenPose
OCR	text from image	detection + recognition (CRNN), TrOCR, document AI models
Image similarity / retrieval	embedding	CLIP, DINOv2, metric learning
Generation	images	Diffusion models
Vision-language	captions, VQA	CLIP, multimodal LLMs
Video	actions, tracking	3D CNN, video transformers, SORT/DeepSORT/ByteTrack

2. Key concepts & models

Q: One-stage vs two-stage detectors? Two-stage (Faster R-CNN): region proposals then classify → more accurate, slower. One-stage (YOLO, SSD, RetinaNet): predict boxes directly → fast, real-time.

Q: Anchor boxes? Predefined box shapes the model adjusts. Anchor-free detectors (FCOS, newer YOLOs, DETR) predict directly.

Q: Non-Maximum Suppression (NMS)? Remove duplicate boxes: keep highest-score box, drop others with IoU above threshold. Too low threshold → misses overlapping objects (crowds).

Q: IoU? Intersection area / union area of predicted vs ground-truth box.

Q: U-Net? Encoder-decoder with skip connections that keep fine spatial detail. Standard for segmentation (especially medical).

Q: Vision Transformer (ViT)? Split image into patches, embed them as tokens, apply Transformer. Needs lots of data or pretraining; less built-in inductive bias than CNNs.

Q: CLIP? Image encoder + text encoder trained contrastively on image-text pairs → shared embedding space → zero-shot classification and image search with text.

Q: Transfer learning in CV? Start from ImageNet/foundation-model weights; replace head; fine-tune. Big gains when you have little data.

Q: Focal loss? Down-weights easy examples, focuses on hard ones. Fixes foreground/background imbalance in detection.

Q: Self-supervised learning in vision? SimCLR/MoCo (contrastive), MAE (masked patches), DINO. They learn features without labels.

3. Metrics

- Classification: accuracy, precision/recall/F1, top-5 accuracy, confusion matrix.
- Detection: **mAP** (mean Average Precision across classes) at IoU 0.5 or averaged over 0.5:0.95 (COCO).
- Segmentation: mIoU, Dice coefficient.
- Generation: FID, CLIP score, human eval.
- Production: latency (ms/frame), FPS, model size.

4. Data & augmentation

- Augmentations: flips, crops, rotation, color jitter, blur, noise, cutout, mixup, cutmix, mosaic (YOLO). **Must be label-preserving** (don't flip text/digits; for detection, transform boxes too).
- Labeling: bounding boxes / polygons tools (CVAT, Label Studio), label QA, active learning.
- Synthetic data for rare cases.
- Normalize with the same mean/std used in pretraining.

5. Deployment

- Edge (mobile, cameras): quantization (INT8), pruning, distillation, TensorRT/ONNX/CoreML/TFLite, smaller backbones (MobileNet, EfficientNet-Lite).
- Video pipelines: decode → sample frames (not every frame) → batch inference → tracking to smooth → events.
- Trade-off: accuracy vs FPS vs power.

6. What-if scenarios

Model works in the lab but fails on real camera images.

Domain shift: lighting, angle, blur, resolution, compression, camera sensor. Collect production samples, label, fine-tune; augmentation matching real conditions; check preprocessing (RGB vs BGR, resize method,

normalization).

Detector misses small objects.

Higher input resolution, feature pyramid (FPN), tiling/slicing large images (SAHI), anchor sizes tuned, more small-object training examples.

Too many false positives at night.

Night data under-represented → collect/augment low-light data, separate threshold or model for night, IR-aware preprocessing, hard negative mining.

Model is accurate but too slow for real-time on edge device.

Smaller backbone, quantization, pruning, distillation, lower resolution, TensorRT, skip frames + tracking between detections.

Model learned the background instead of the object ("cows on grass" problem).

Spurious correlation. Use Grad-CAM to see where the model looks, diversify backgrounds, augmentation (cutout, background replacement), counterfactual examples.

Medical images: only 200 labeled scans.

Transfer learning (from domain-specific pretrained models), heavy augmentation, self-supervised pretraining on unlabeled scans, cross-validation, expert-in-the-loop active learning, careful patient-level splits (no same patient in train & test!).

Overlapping objects in crowds are missed.

NMS threshold too aggressive → tune, Soft-NMS, or NMS-free detectors (DETR).

Accuracy dropped after a camera firmware update.

Input distribution changed (color profile, compression). Monitor input statistics (brightness, blur), detect drift, recalibrate/fine-tune.

OCR fails on handwritten or rotated text.

Deskew/rotation correction, specific handwriting models, fine-tune on domain data, multimodal LLM fallback with confidence checks.

11. Statistics, Probability & A/B Testing

Asked in ML engineer, data scientist and applied scientist interviews.

Contents

1. Probability
2. Distributions
3. Statistics & inference
4. A/B testing
5. Classic puzzle questions
6. What-if scenarios

---

1. Probability

- **Conditional:** $P(A|B) = P(A \cap B) / P(B)$.
- **Independence:** $P(A \cap B) = P(A) \cdot P(B)$.
- **Bayes' theorem:** $P(A|B) = P(B|A) \cdot P(A) / P(B)$.
- **Expectation** is linear: $E[X+Y] = E[X] + E[Y]$ (always, even if dependent).
- **Variance:** $Var(X) = E[X^2] - E[X]^2$. $Var(aX) = a^2 Var(X)$.

Classic Bayes question: Disease affects 1% of people. Test is 99% sensitive, 95% specific (5% false positive). You test positive. What's the chance you're sick? $P(sick|+) = (0.99 \cdot 0.01) / (0.99 \cdot 0.01 + 0.05 \cdot 0.99) = 0.0099 / 0.0594 \approx$ **16.7%**. Lesson: low base rates → most positives are false (same reason fraud alerts are mostly false).

2. Distributions

Distribution	Use
Bernoulli / Binomial	yes/no events, number of successes in n trials (clicks)
Poisson	counts per time interval (requests/minute)
Normal	sums/averages (Central Limit Theorem)
Exponential	time between events
Uniform	equal likelihood
Power law / long tail	popularity, word frequency

Central Limit Theorem: the mean of many independent samples is approximately normal, whatever the original distribution → why t-tests/z-tests work for averages.

3. Statistics & inference

Q: p-value? Probability of observing results at least this extreme **if the null hypothesis were true**. It is NOT the probability the null is true.

Q: Type I vs Type II error? Type I: false positive (reject a true null), rate = α (usually 0.05). Type II: false negative (miss a real effect), rate = β. **Power** = 1 − β (usually 0.8).

Q: Confidence interval? If we repeated the experiment many times, 95% of such intervals would contain the true value.

Q: Which test? t-test (compare means), z-test for proportions (conversion rates), chi-square (categorical independence), Mann-Whitney (non-normal), bootstrap (anything, by resampling).

Q: Correlation vs causation? Correlation can come from confounders or reverse causation. Causation needs randomized experiments or causal methods (diff-in-diff, instrumental variables, propensity matching).

Q: Simpson's paradox? A trend in each group reverses when groups are combined, due to a confounding variable (e.g., different group sizes). Always check segment-level results.

Q: MLE vs MAP? MLE maximizes likelihood of data. MAP adds a prior (= regularization: Gaussian prior ↔ L2, Laplace prior ↔ L1).

4. A/B testing

Steps: 1. Hypothesis + **primary metric** (e.g., conversion) + **guardrail metrics** (latency, revenue, errors). 2. Randomization unit (user, not request, so the same user never sees both). 3. **Sample size** via power analysis (baseline rate, minimum detectable effect, α, power). 4. Run for full weeks (weekday/weekend cycles); **don't peek** and stop early when significant. 5. Analyze: significance, confidence intervals, segments; check sanity (sample ratio mismatch). 6. Decide considering practical significance, not only statistical.

Pitfalls to mention: - **Peeking** → inflated false positives (use sequential testing if you must). - **Multiple comparisons** → many metrics/variants → Bonferroni or FDR correction. - **Novelty / primacy effects** → run longer. - **Network effects** (social, marketplaces) → cluster randomization. - **Sample Ratio Mismatch (SRM):** expected 50/50 but got 52/48 → bug in assignment; don't trust results. - Long-term vs short-term effects → holdout groups.

For ML models: offline evaluation → shadow → A/B test. Interleaving for ranking (faster, more sensitive).

5. Classic puzzle questions

- **Expected number of coin flips to get heads?** 2 (geometric, 1/p).
- **Birthday problem:** 23 people → >50% chance two share a birthday.
- **Monty Hall:** always switch. You win 2/3 of the time.
- **Two children, at least one is a boy. Probability both are boys?** 1/3.

- **Sample from a stream of unknown length uniformly?** Reservoir sampling: keep the i-th item with probability 1/i.
- **Generate a fair coin from a biased one?** Flip twice: HT → H, TH → T, else repeat.

6. What-if scenarios

A/B test shows +5% conversion, but the effect disappears after launch.

Novelty effect, peeking/false positive, seasonality, SRM bug, segment mix changed. Use holdouts, longer tests, pre-registered analysis.

Test result is significant for the overall population but negative for mobile users.

Simpson's paradox or genuine heterogeneity. Investigate segments; consider targeted rollout; be careful of multiple-comparison false findings.

You don't have enough traffic for an A/B test.

Increase MDE, use variance reduction (CUPED: use the pre-experiment metric as a covariate), run longer, use more sensitive metrics, interleaving, or Bayesian approaches.

Metric moved, but you suspect it's not from your change.

Check A/A test, seasonality, other concurrent experiments, logging changes, external events.

12. ML Coding Round (Implement From Scratch)

Interviewers ask you to code ML pieces using only NumPy (or PyTorch) in 30-45 minutes. **Practice:** type each one from memory until you can do it in < 15 minutes. Talk while coding: shapes, edge cases, complexity.

Contents

1. Linear regression (gradient descent)
2. Logistic regression
3. K-means
4. k-Nearest Neighbors
5. Metrics: precision, recall, F1, AUC
6. Softmax & cross-entropy (stable)
7. Scaled dot-product & multi-head attention
8. A 2-layer neural network with backprop
9. Decision tree split (Gini)
10. PyTorch training loop
11. Cosine similarity top-k search
12. BPE tokenizer merge step
13. Interview tips for coding rounds

---

1. Linear regression

```
import numpy as np

class LinearRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr, self.epochs = lr, epochs

    def fit(self, X, y):                        # X: (n, d), y: (n,)
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        for _ in range(self.epochs):
            y_hat = X @ self.w + self.b
            err = y_hat - y                     # (n,)
            dw = (2 / n) * X.T @ err            # gradient of MSE
            db = (2 / n) * err.sum()
            self.w -= self.lr * dw
            self.b -= self.lr * db
        return self

    def predict(self, X):
        return X @ self.w + self.b

# Closed form (normal equation): w = (XᵀX)⁻¹ Xᵀy  → O(d³), use np.linalg.lstsq for stability
```

2. Logistic regression

```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))   # clip avoids overflow

class LogisticRegression:
    def __init__(self, lr=0.1, epochs=1000, l2=0.0):
        self.lr, self.epochs, self.l2 = lr, epochs, l2

    def fit(self, X, y):
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        for _ in range(self.epochs):
            p = sigmoid(X @ self.w + self.b)
            dw = X.T @ (p - y) / n + self.l2 * self.w   # gradient of log loss (+ L2)
            db = (p - y).mean()
            self.w -= self.lr * dw
            self.b -= self.lr * db
        return self

    def predict_proba(self, X):
        return sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

def log_loss(y, p, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

3. K-means

```
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    centroids = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        # distances: (n, k)
        dists = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = dists.argmin(axis=1)
        new = np.array([X[labels == j].mean(axis=0) if np.any(labels == j) else centroids[j]
                        for j in range(k)])               # keep old centroid if cluster empty
        if np.allclose(new, centroids):
            break
        centroids = new
    return centroids, labels
# Mention: k-means++ init, scaling features, O(n·k·d) per iteration.
```

4. k-Nearest Neighbors

```
from collections import Counter

def knn_predict(X_train, y_train, X_test, k=3):
    preds = []
    for x in X_test:
```

```
        d = np.linalg.norm(X_train - x, axis=1)
        idx = np.argpartition(d, k)[:k]          # O(n) instead of full sort
        preds.append(Counter(y_train[idx]).most_common(1)[0][0])
    return np.array(preds)
```

5. Metrics

```
def precision_recall_f1(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return precision, recall, f1

def roc_auc(y_true, scores):
    # AUC = probability a random positive scores higher than a random negative
    order = np.argsort(scores)
    ranks = np.empty(len(scores)); ranks[order] = np.arange(1, len(scores) + 1)
    n_pos = y_true.sum(); n_neg = len(y_true) - n_pos
    return (ranks[y_true == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
    # (ties need average ranks; mention this in the interview)
```

6. Softmax & cross-entropy

```
def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)    # numerical stability: subtract max
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

def cross_entropy(logits, labels):             # logits (n, C), labels (n,) ints
    z = logits - logits.max(axis=1, keepdims=True)
    log_probs = z - np.log(np.exp(z).sum(axis=1, keepdims=True))   # log-softmax
    return -log_probs[np.arange(len(labels)), labels].mean()
# Gradient w.r.t. logits = softmax(logits) - one_hot(labels), divided by n
```

7. Attention

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q: (..., Lq, d), K: (..., Lk, d), V: (..., Lk, dv)
    d = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d)       # (..., Lq, Lk)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)          # mask=False → blocked
    weights = softmax(scores, axis=-1)
    return weights @ V, weights

def causal_mask(L):
    return np.tril(np.ones((L, L), dtype=bool))        # token i sees tokens ≤ i
```

```python
import torch, torch.nn as nn, torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x, causal=False):                # x: (B, L, D)
        B, L, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        # (B, L, D) -> (B, h, L, dk)
        q, k, v = [t.view(B, L, self.h, self.dk).transpose(1, 2) for t in (q, k, v)]
        scores = q @ k.transpose(-2, -1) / self.dk ** 0.5          # (B, h, L, L)
        if causal:
            mask = torch.triu(torch.ones(L, L, dtype=torch.bool, device=x.device), 1)
            scores = scores.masked_fill(mask, float('-inf'))
        attn = F.softmax(scores, dim=-1)
        y = (attn @ v).transpose(1, 2).contiguous().view(B, L, D)
        return self.out(y)
```

8. Two-layer neural network

```python
class TwoLayerNet:
    def __init__(self, d_in, d_hidden, n_classes, lr=0.1, seed=0):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, np.sqrt(2 / d_in), (d_in, d_hidden))   # He init
        self.b1 = np.zeros(d_hidden)
        self.W2 = rng.normal(0, np.sqrt(2 / d_hidden), (d_hidden, n_classes))
        self.b2 = np.zeros(n_classes)
        self.lr = lr

    def step(self, X, y):                                  # one gradient step, returns loss
        n = len(X)
        # forward
        h_pre = X @ self.W1 + self.b1
        h = np.maximum(0, h_pre)                           # ReLU
        logits = h @ self.W2 + self.b2
        probs = softmax(logits)
        loss = -np.log(probs[np.arange(n), y] + 1e-12).mean()
        # backward
        dlogits = probs.copy(); dlogits[np.arange(n), y] -= 1; dlogits /= n
        dW2 = h.T @ dlogits;      db2 = dlogits.sum(0)
        dh = dlogits @ self.W2.T
        dh_pre = dh * (h_pre > 0)                          # ReLU gradient
        dW1 = X.T @ dh_pre;       db1 = dh_pre.sum(0)
        # update
        for p, g in ((self.W1, dW1), (self.b1, db1), (self.W2, dW2), (self.b2, db2)):
            p -= self.lr * g
        return loss
```

9. Decision tree split

```
def gini(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / counts.sum()
    return 1 - np.sum(p ** 2)

def best_split(X, y):
    best = (None, None, np.inf)                          # (feature, threshold, weighted gini)
    n = len(y)
    for f in range(X.shape[1]):
        for t in np.unique(X[:, f]):
            left = X[:, f] <= t
            if left.all() or (~left).all():
                continue
            g = (left.sum() * gini(y[left]) + (~left).sum() * gini(y[~left])) / n
            if g < best[2]:
                best = (f, t, g)
    return best
```

10. PyTorch training loop

```
def train(model, train_loader, val_loader, epochs=10, lr=3e-4, device='cuda'):
    model.to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    loss_fn = nn.CrossEntropyLoss()
    best_val = float('inf')
    for epoch in range(epochs):
        model.train()
        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            opt.zero_grad()
            loss = loss_fn(model(X), y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        sched.step()

        model.eval()                                     # dropout/BN off
        val_loss, n = 0.0, 0
        with torch.no_grad():                            # no graph → saves memory
            for X, y in val_loader:
                X, y = X.to(device), y.to(device)
                val_loss += loss_fn(model(X), y).item() * len(y); n += len(y)
        val_loss /= n
        if val_loss < best_val:
            best_val = val_loss
            torch.save(model.state_dict(), 'best.pt')    # checkpoint best model
        print(f'epoch {epoch}: val_loss={val_loss:.4f}')
```

11. Cosine similarity top-k

```
def top_k_cosine(query, matrix, k=5):          # query (d,), matrix (n, d)
    q = query / np.linalg.norm(query)
    M = matrix / np.linalg.norm(matrix, axis=1, keepdims=True)
    sims = M @ q
    idx = np.argpartition(-sims, k)[:k]
    idx = idx[np.argsort(-sims[idx])]          # sort the top-k only
    return idx, sims[idx]
# Brute force O(n·d). At scale, use an ANN index (HNSW/IVF). See 02 §7.
```

12. BPE merge step

```
from collections import Counter

def bpe_train(words, num_merges):
    # words: list of strings; represent each as tuple of symbols
    vocab = Counter(tuple(w) + ('</w>',) for w in words)
    merges = []
    for _ in range(num_merges):
        pairs = Counter()
        for word, freq in vocab.items():
            for a, b in zip(word, word[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        best = max(pairs, key=pairs.get)
        merges.append(best)
        new_vocab = Counter()
        for word, freq in vocab.items():
            out, i = [], 0
            while i < len(word):
                if i < len(word) - 1 and (word[i], word[i + 1]) == best:
                    out.append(word[i] + word[i + 1]); i += 2
                else:
                    out.append(word[i]); i += 1
            new_vocab[tuple(out)] += freq
        vocab = new_vocab
    return merges
```

13. Tips

- Start by stating input/output shapes and assumptions. Write a tiny test example.
- Vectorize with NumPy; mention complexity (time & memory).
- Numerical stability: subtract max in softmax, clip in log/sigmoid, add epsilon.
- Edge cases: empty clusters, division by zero, ties, k > n.
- Other common asks: implement **dropout, batch norm forward, conv2d (naive loops), IoU + NMS, train/test split, stratified sampling, gradient check, beam search, top-p sampling, reservoir sampling, an LRU cache**.

Bonus: IoU + NMS (CV interviews)

```python
def iou(a, b):                                  # boxes [x1, y1, x2, y2]
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area = lambda r: (r[2] - r[0]) * (r[3] - r[1])
    return inter / (area(a) + area(b) - inter + 1e-9)

def nms(boxes, scores, thresh=0.5):
    order = np.argsort(scores)[::-1]
    keep = []
    while len(order):
        i = order[0]; keep.append(i)
        order = np.array([j for j in order[1:] if iou(boxes[i], boxes[j]) < thresh])
    return keep
```

Bonus: Top-p (nucleus) sampling

```python
def top_p_sample(logits, p=0.9, temperature=1.0, rng=np.random.default_rng()):
    probs = softmax(logits / temperature)
    order = np.argsort(probs)[::-1]
    cum = np.cumsum(probs[order])
    cutoff = np.searchsorted(cum, p) + 1         # smallest set with mass ≥ p
    keep = order[:cutoff]
    kept = probs[keep] / probs[keep].sum()
    return rng.choice(keep, p=kept)
```

13. MLOps & Production ML (Q&A)

Complements 02-ml-and-llm-systems.md. Asked heavily in ML Engineer / MLOps / AI Engineer interviews.

Core Q&A

Q: What is MLOps? Practices to reliably build, deploy, monitor and improve ML models in production: CI/CD for code **and** data **and** models, reproducibility, automation, monitoring.

Q: How is ML CI/CD different from software CI/CD? Software: test code → deploy. ML adds: data validation, training pipeline, model evaluation gates (must beat current model), model registry, continuous training (CT), and monitoring of data/model quality as well as uptime.

Q: MLOps maturity levels? Level 0: manual notebooks. Level 1: automated training pipeline + continuous training. Level 2: full CI/CD of pipelines with automated testing, deployment and monitoring.

Q: What should you version? Code (git), data (DVC/lakeFS/Delta), features, model artifacts (registry), hyperparameters/config, environment (Docker image, requirements lock), prompts (for LLMs), eval datasets.

Q: Which tools should you know by name? | Need | Tools | |---|---| | Orchestration | Airflow, Kubeflow Pipelines, Prefect, Dagster, SageMaker/Vertex Pipelines | | Experiment tracking | MLflow, Weights & Biases, Neptune | | Feature store | Feast, Tecton, Databricks/Vertex/SageMaker feature stores | | Model registry | MLflow Registry, SageMaker/Vertex registries | | Serving | Triton, TorchServe, BentoML, KServe, Ray Serve, vLLM/TGI (LLMs) | | Containers | Docker, Kubernetes (autoscaling, GPU scheduling) | | Monitoring | Prometheus + Grafana, Evidently, Arize, WhyLabs; LLM: Langfuse, LangSmith, Phoenix | | Data quality | Great Expectations, dbt tests, Pandera |

Q: How do you test ML systems? - Unit tests for feature code and preprocessing. - Data tests: schema, ranges, nulls, distribution. - Model tests: performance thresholds on holdout, slice tests (no segment worse than X), invariance tests (changing a name shouldn't change sentiment), directional tests, regression tests on known hard cases. - Integration tests: end-to-end pipeline on sample data. - Load tests: latency/throughput of serving.

Q: Deployment strategies? Shadow → canary → A/B → full; blue-green; keep previous version for instant rollback. See 02 §5.

Q: Online vs batch inference trade-offs? See 02 §4.

Q: How to make inference cheaper/faster? Batching, quantization, distillation, pruning, ONNX/TensorRT compilation, caching predictions, right-sizing hardware (CPU often enough for small models), autoscaling, spot instances for batch.

Q: How do you containerize and scale a model? Docker image with model + server, health/readiness endpoints, deploy on Kubernetes with HPA (autoscale on CPU/GPU/QPS/queue length), resource limits, GPU node pools, model loaded at startup (or pulled from registry).

Q: Continuous training triggers? Schedule, new labeled data volume, drift detected, performance drop.

Q: What is training-serving skew and how to prevent it? See 04 #23: shared feature code/feature store, log serving features and use them for training, compare distributions.

Q: Feedback loops? Model's predictions affect future training data (recommenders only learn from items shown). Mitigate with exploration, randomized holdouts, logging propensities.

Q: Responsible AI in production? Fairness checks by slice, explainability, model cards, data privacy (PII handling, retention), audit trails, human oversight for high-stakes decisions.

Q: What's different in LLMOps? Prompt versioning, eval suites with LLM judges, token cost monitoring, tracing multi-step chains/agents, guardrails, provider fallback, managing model version changes from vendors, RAG index pipelines.

What-if scenarios (MLOps)

- **Training pipeline fails at 3 AM** → alerting, retries for transient errors, idempotent steps, checkpointing; keep serving the previous model (never block serving on training).
- **Model registry has a model but no one knows what data it used** → enforce lineage metadata on registration (data version, code commit, metrics) in the pipeline; block manual uploads.
- **Kubernetes pods with the model restart constantly (OOMKilled)** → memory limits too low for model + batch, memory leak, too many workers each loading the model; right-size, share model across workers, smaller batch.
- **Cold starts take 2 minutes** → bake model in image or fast storage, keep min replicas, lazy-load less used models, smaller model formats.
- **Multiple teams deploy models differently, causing chaos** → internal ML platform: templates, standard serving, registry, monitoring defaults (paved road).
- **Cloud bill doubled** → find idle GPUs, oversized instances, unused endpoints, untagged experiments; autoscale to zero, spot/preemptible for training, budgets & alerts per team.

14. DSA Coding Round (for AI/ML roles)

Most ML/AI engineer loops include 1-2 standard coding rounds (LeetCode-style, usually medium). You don't need 500 problems. You need the **~15 patterns**. Learn the pattern, then do 5-8 problems each.

The method for every coding question (say it out loud)

1. **Clarify:** input size, types, edge cases (empty, duplicates, negatives), return format.
2. **Example:** walk through a small example by hand.
3. **Brute force first:** state it and its complexity.
4. **Optimize:** which pattern fits? State new complexity.
5. **Code** cleanly with good names.
6. **Test:** run your example + edge cases by hand.
7. **Complexity:** time and space.

Patterns → when to use → classic problems

#	Pattern	Signal in the question	Practice problems
1	**Hash map / set**	"find pair", "count", "seen before", O(1) lookup	Two Sum, Group Anagrams, Longest Consecutive Sequence
2	**Two pointers**	sorted array, pairs, palindrome, in-place	3Sum, Container With Most Water, Valid Palindrome
3	**Sliding window**	contiguous subarray/substring, "longest/shortest with condition"	Longest Substring Without Repeating, Minimum Window Substring
4	**Prefix sum**	range sums, "subarray sum equals k"	Subarray Sum Equals K, Range Sum Query
5	**Binary search**	sorted data, "minimum X such that…", search on answer	Search in Rotated Array, Koko Eating Bananas, Median of Two Sorted Arrays
6	**Stack / monotonic stack**	matching brackets, "next greater element"	Valid Parentheses, Daily Temperatures, Largest Rectangle in Histogram
7	**Heap / priority queue**	top-k, k-th largest, merge sorted, streaming median	Kth Largest, Top K Frequent, Merge K Sorted Lists, Find Median from Data Stream
8	**Linked list**	reverse, cycle, merge	Reverse List, Linked List Cycle, LRU Cache (hash map + doubly linked list)
9	**Trees (DFS/BFS)**	hierarchy, depth, paths	Max Depth, Level Order, Validate BST, Lowest Common Ancestor, Serialize Tree

#	Pattern	Signal in the question	Practice problems
10	**Graphs (BFS/DFS/union-find/topological sort)**	grid, connections, dependencies, shortest path	Number of Islands, Course Schedule, Clone Graph, Dijkstra (Network Delay Time)
11	**Backtracking**	"all combinations/permutations/subsets"	Subsets, Permutations, Combination Sum, Word Search
12	**Dynamic programming**	"number of ways", "min/max", overlapping subproblems	Climbing Stairs, Coin Change, Longest Increasing Subsequence, Edit Distance, LCS
13	**Greedy / intervals**	scheduling, merge intervals	Merge Intervals, Meeting Rooms II, Jump Game
14	**Trie**	prefix search, autocomplete	Implement Trie, Word Search II
15	**Bit manipulation / math**	XOR tricks, powers	Single Number, Counting Bits

ML-flavored coding questions (common at AI companies): Edit distance (NLP), LRU cache (model cache), top-k frequent words, reservoir sampling, matrix ops, implement tokenizer, rate limiter, moving average from a stream, merge k sorted lists (merge search results), autocomplete with trie, sample from a weighted distribution (prefix sums + binary search).

Complexity cheat sheet

Operation	Complexity
Hash map get/put	O(1) avg
Sort	O(n log n)
Binary search	O(log n)
Heap push/pop	O(log n); build heap O(n)
BFS/DFS	O(V + E)
Top-k with heap	O(n log k)
Recursion space	O(depth)

Rough limits: $n \le 10^5$ → need O(n log n) or better. $n \le 10^3$ → $O(n^2)$ ok. $n \le 20$ → exponential/backtracking ok.

Python templates

```python
# Sliding window (variable size)
def longest_unique_substring(s):
    seen, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1
        seen[ch] = right
        best = max(best, right - left + 1)
```

```
    return best

# Binary search on answer
def min_feasible(lo, hi, ok):            # smallest x in [lo, hi] with ok(x) True
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo

# BFS on grid
from collections import deque
def bfs(grid, start):
    R, C = len(grid), len(grid[0])
    q, seen = deque([start]), {start}
    while q:
        r, c = q.popleft()
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in seen and grid[nr][nc] != '#':
                seen.add((nr, nc)); q.append((nr, nc))
    return seen

# Top-k with heap
import heapq
from collections import Counter
def top_k_frequent(nums, k):
    return [x for x, _ in heapq.nlargest(k, Counter(nums).items(), key=lambda t: t[1])]

# LRU cache
from collections import OrderedDict
class LRUCache:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)

# DP: edit distance
def edit_distance(a, b):
    dp = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        prev, dp[0] = dp[0], i
        for j in range(1, len(b) + 1):
            cur = dp[j]
            dp[j] = prev if a[i-1] == b[j-1] else 1 + min(prev, dp[j], dp[j-1])
            prev = cur
    return dp[-1]
```

8-week plan

Weeks	Patterns	Daily
1-2	1-4 (hash, pointers, window, prefix)	2 problems
3-4	5-9 (binary search, stack, heap, lists, trees)	2 problems

Weeks	Patterns	Daily
5-6	10-12 (graphs, backtracking, DP)	2 problems
7-8	13-15 + mixed timed sets	1 timed mock (45 min, 2 problems)

Use a curated list (e.g., "NeetCode 150" / "Blind 75"). It covers these patterns.

15. Behavioral & Project Deep-Dive Round

For ML/AI roles, **"Tell me about a project you built"** is often the round that decides the offer. Interviewers dig 5-6 levels deep with "why?" and "what if?". If you can't explain *your own* decisions, it looks like you didn't make them.

Part A: The project deep dive

Prepare 2-3 projects using this template (write it down!)

Section	Your notes
1. Problem & business goal	What problem, for whom, why it mattered, how success was measured (business metric)
2. Your role	What *you* did vs. the team. Use "I".
3. Data	Source, size, labels (how obtained, quality), splits, imbalance, leakage risks you handled
4. Baseline	What simple approach did you compare against? Its score?
5. Approach	Model/architecture and **why this one over alternatives**
6. Evaluation	Offline metrics (why these?), validation strategy, error analysis findings
7. Deployment	How served (batch/real-time), latency, infra, monitoring
8. Results / impact	Numbers: "+12% recall at same precision", "cut cost 40%", "saved 20 hrs/week"
9. Challenges	The hardest problem and how you debugged it (use C.L.H.I.F.P. from 03)
10. Trade-offs	What you sacrificed and why
11. What you'd do differently	Shows maturity
12. Next steps	How you'd improve with more time/data

Questions you WILL be asked: prepare an answer for each

Data - How much data? How was it labeled? How did you check label quality? - Was it imbalanced? How did you handle it? - How did you split the data? Could there be leakage? - What features mattered most? Any surprising ones?

Model - Why this model? What else did you try? Why not a simpler/bigger one? - What hyperparameters mattered? How did you tune them? - Why this loss/metric?

Evaluation - How do you know it actually works? Offline vs online results? - Where does it fail? (error analysis: always have 2-3 failure categories ready) - Did you compare against a baseline?

Production - How is it served? Latency? Cost? - How do you monitor it? What happens when it degrades? - How often retrained?

What-ifs (this is where most people fail, so practice these) - What if the data doubled / was 100× bigger? - What if labels were unavailable? - What if latency had to be 10× lower? - What if the data distribution changed? - What if you had to explain predictions to a regulator? - What if a user group gets worse results? - (For RAG/LLM projects:) what if answers are inconsistent / hallucinated / slow / expensive? → 04

If you don't know, say *"We didn't measure that, but this is how I would check it: ..."*. That still scores well. Never invent numbers.

Example (RAG project): 2-minute pitch

> "I built an internal Q&A assistant over 50k HR and IT documents for ~5,000 employees. The goal was reducing support tickets. I owned the retrieval pipeline and evaluation. Baseline was keyword search, which answered ~40% of a labeled set of 300 real questions. I used structure-aware chunking with section headers, hybrid BM25 + embedding search, and a cross-encoder reranker. Recall@5 went from 58% to 86%. For generation we used temperature 0 with mandatory citations and an 'I don't know' path when retrieval scores were low; faithfulness on our eval set was 94%. The hardest problem was inconsistent answers. We traced it to duplicate policy versions in the corpus and fixed it with effective-date metadata and dedupe. After launch, IT tickets for covered topics dropped 25%. If I did it again, I'd build the eval set on day one instead of week four."

A pitch like this answers most of the follow-up questions before they're asked.

Part B: Behavioral questions (STAR method)

STAR: Situation (1 line) → **T**ask (your responsibility) → **A**ction (what *you* did, about 60% of the answer) → **R**esult (numbers + what you learned).

Prepare 6-8 stories that cover these

Theme	Typical question
Conflict	Tell me about a disagreement with a teammate/manager.
Failure	Tell me about a project that failed / a mistake you made.
Ambiguity	A time requirements were unclear.
Ownership	A time you went beyond your role.
Influence	Convincing others without authority (e.g., "use a simpler model").
Deadline / pressure	Delivering under a tight timeline; what you cut.
Learning	Learning a new technology quickly.
Impact	Your most impactful work.
Stakeholders	Explaining ML to non-technical people.
Ethics / quality	Pushing back on shipping a model that wasn't ready.

One story can cover several themes. Map each story to 2-3 themes.

Good vs weak answer: "Tell me about a failure"

- **Weak:** "I'm a perfectionist...", blaming others, or no lesson learned.
- **Good:** "I shipped a churn model without slice-level evaluation. Two weeks later sales noticed it flagged nearly all new customers. A feature was missing for accounts < 30 days old. I rolled back, added a default-handling fix and slice tests to our pipeline. Since then I always evaluate by segment before launch. The relaunched model hit its target."

Questions to ask the interviewer (always have 2-3)

- What does success look like in this role in the first 6 months?
- How do models/LLM features get from prototype to production here? How do you evaluate them?
- What's the biggest technical challenge the team is facing right now?

Part C: Communication habits that change outcomes

- **Think out loud.** If you're silent, the interviewer can't give you credit.
- **Structure first:** "I'll cover three things: data, model, and deployment."
- **Numbers over adjectives:** "p95 latency 400 ms" beats "pretty fast".
- **Admit unknowns, then reason:** "I haven't used X, but I'd expect it to behave like Y because..."
- **Check in:** "Does this level of detail make sense, or should I go deeper on retrieval?"
- **Simple English is fine.** Clear beats fancy. Short sentences.

16. Reinforcement Learning & Bandits

Asked in research, robotics, ads/recommendation, game AI, and LLM alignment (RLHF) interviews. Format: **Q → short answer you can say in 30-60 seconds.** "What if" scenarios at the end.

Contents

1. Core ideas
2. Value functions & the Bellman equation
3. Value-based methods: Q-learning & DQN
4. Policy-based methods: policy gradients, actor-critic, PPO
5. Exploration
6. Model-based, offline & imitation learning
7. Multi-armed & contextual bandits
8. RL for LLMs (RLHF and beyond)
9. RL in production & sim-to-real
10. What-if scenarios (RL)

---

1. Core ideas

Q: What is reinforcement learning? An **agent** takes **actions** in an **environment**, observes a new **state** and a **reward**, and learns a **policy** (what to do in each state) that maximizes total future reward. There is no labeled "correct action". It only gets reward signals, and they are often delayed.

Q: How is RL different from supervised learning? No correct labels, only rewards; rewards can be delayed (credit assignment problem); the agent's own actions change the data it sees (data is not i.i.d.); it must balance exploration and exploitation.

Q: What is an MDP (Markov Decision Process)? The math model of RL: states S, actions A, transition probabilities P(s'|s,a), reward function R(s,a), and discount factor γ. **Markov property:** the next state depends only on the current state and action, not on the full history.

Q: What if the state is not fully observable? It's a **POMDP**. Use history (frame stacking, RNN/Transformer memory) or a belief state to approximate the true state.

Q: What is the discount factor γ? A number in [0, 1) that makes future rewards worth less. Return $G = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$ Small γ → short-sighted agent. γ close to 1 → long-term planning but higher variance and slower learning.

Q: Episodic vs continuing tasks? Episodic tasks end (a game round). Continuing tasks never end (server cooling control), so they need discounting or average-reward formulations.

Q: Deterministic vs stochastic policy? Deterministic: $a = \pi(s)$. Stochastic: $\pi(a|s)$ gives probabilities. Stochastic policies help exploration and are needed in some games (rock-paper-scissors).

Q: On-policy vs off-policy? On-policy learns from data collected by the current policy (SARSA, PPO). It is stable, but can't reuse old data. Off-policy learns from data collected by any policy (Q-learning, DQN,

SAC). It is sample-efficient thanks to replay buffers, but less stable.

Q: Model-free vs model-based? Model-free learns values/policy directly from experience. Model-based learns (or is given) a model of the environment and plans with it. This is more sample-efficient, but model errors add up.

Q: What is the credit assignment problem? When a reward arrives late (win at the end of a chess game), which earlier actions caused it? Value functions, discounting, and advantage estimation help solve it.

2. Value functions & the Bellman equation

Q: State-value vs action-value? V(s) = expected return starting from state s following policy π. Q(s,a) = expected return after taking action a in s, then following π. Advantage A(s,a) = Q(s,a) − V(s) = "how much better is this action than average".

Q: Bellman equation in words? The value of a state = immediate reward + discounted value of the next state. `V(s) = E[ r + γ·V(s') ]`. Bellman **optimality**: `Q*(s,a) = E[ r + γ·max_a' Q*(s',a') ]`.

Q: Dynamic programming methods? When the model is known: **policy iteration** (evaluate policy, then improve greedily, repeat) and **value iteration** (apply Bellman optimality update until convergence).

Q: Monte Carlo vs Temporal Difference (TD) learning? MC waits until the episode ends and uses the full return: unbiased, but high variance. TD updates after each step using the estimate of the next state (bootstrapping): lower variance, some bias, and it works in continuing tasks. **TD(λ)** / n-step returns sit between.

Q: What is bootstrapping in RL? Updating an estimate using another estimate (V(s') in the TD target) instead of the real full return.

3. Value-based methods: Q-learning & DQN

Q: Q-learning update? `Q(s,a) ← Q(s,a) + α · [ r + γ·max_a' Q(s',a') − Q(s,a) ]` Off-policy: it learns the greedy policy while behaving with an exploratory one (ε-greedy).

Q: SARSA vs Q-learning? SARSA (on-policy) uses the action actually taken next: `r + γQ(s',a')`. Q-learning uses the max. SARSA is "safer" near cliffs because it accounts for its own exploration mistakes.

Q: Why can't we use a table for Atari? Too many states (pixels). Use a neural network to approximate Q → **DQN**.

Q: DQN's two key tricks? 1. **Experience replay:** store transitions in a buffer and train on random mini-batches. This breaks correlation between consecutive samples, reuses data. 2. **Target network:** a slowly updated copy of the Q-network computes the TD target, so the target doesn't move every step (more stable).

Q: The "deadly triad"? Function approximation + bootstrapping + off-policy learning together can make training diverge. DQN tricks reduce (not remove) this risk.

Q: DQN improvements? Double DQN (fixes overestimation of Q by separating action selection and evaluation), Dueling DQN (separate V and A streams), Prioritized replay (sample surprising transitions more), multi-step returns, distributional RL (learn the return distribution), Noisy Nets (learned exploration). Together = **Rainbow**.

Q: Why does Q-learning overestimate values? The max over noisy estimates is biased upward. Double Q-learning fixes it.

Q: Can DQN handle continuous actions? Not directly (max over infinite actions). Use DDPG, TD3, or SAC.

4. Policy-based methods: policy gradients, actor-critic, PPO

Q: Policy gradient idea? Directly adjust policy parameters to make high-return actions more likely: `∇J(θ) = E[ ∇log π(a|s) · G ]` (REINFORCE). Works for continuous actions and stochastic policies.

Q: Problem with REINFORCE? Very high variance → slow, unstable. Fix: subtract a **baseline** (usually V(s)) → use the **advantage** instead of raw return. This doesn't add bias.

Q: Actor-critic? Actor = policy network (chooses actions). **Critic** = value network (estimates V or Q to compute advantage). Critic reduces variance for the actor. Examples: A2C/A3C, SAC, DDPG, TD3.

Q: What is GAE (Generalized Advantage Estimation)? A weighted mix of n-step advantages controlled by λ. It trades bias against variance. Standard in PPO.

Q: Why are large policy updates dangerous? One bad update can collapse the policy, and then the agent collects bad data and can't recover. **TRPO** limits updates with a KL constraint; **PPO** does it more simply.

Q: How does PPO work? It clips the probability ratio `r = π_new(a|s) / π_old(a|s)` to `[1−ε, 1+ε]` (ε≈0.2) in the objective, so the policy can't move too far per update. It is simple and stable, the default choice, and the algorithm first used for RLHF.

Q: SAC (Soft Actor-Critic)? Off-policy actor-critic for continuous control that maximizes reward **plus entropy** → explores well, very sample-efficient and stable. Popular in robotics.

Q: DDPG / TD3? Deterministic actor-critic for continuous actions. TD3 fixes DDPG overestimation with twin critics, delayed actor updates, and target smoothing.

Q: Value-based vs policy-based: when to choose which? | Situation | Prefer | |---|---| | Discrete actions, need sample efficiency | DQN family | | Continuous actions (robotics) | SAC, TD3, PPO | | Need stability, easy parallel simulation | PPO | | Stochastic optimal policy needed | Policy gradient methods |

5. Exploration

Q: Exploration vs exploitation? Exploit = choose the best-known action. Explore = try others to learn if they're better. Too little exploration → stuck on a sub-optimal policy. Too much → wasted reward.

Q: Exploration strategies? - **ε-greedy** (random action with prob ε, decay ε over time) - **Boltzmann / softmax** over Q-values - **Entropy bonus** (policy gradients, SAC) - **Optimism / UCB** (prefer uncertain

actions) - **Thompson sampling** (sample from posterior) - **Intrinsic motivation / curiosity** (reward for novel states: count-based, RND, ICM), used for **sparse reward** problems.

Q: How do you deal with sparse rewards? Reward shaping (carefully), curiosity bonuses, curriculum learning (easy → hard tasks), hindsight experience replay (treat reached states as goals), demonstrations (imitation learning to bootstrap).

Q: Danger of reward shaping? The agent may optimize the shaped reward instead of the true goal. **Potential-based shaping** is guaranteed not to change the optimal policy.

6. Model-based, offline & imitation learning

Q: Model-based RL examples? Dyna (learn model, generate simulated experience), MCTS planning (AlphaGo/AlphaZero: MCTS + policy/value networks), MuZero (learns a latent model), Dreamer (learn world model, train policy in imagination).

Q: Offline (batch) RL? Learn a policy from a fixed logged dataset with no new interaction. This matters when exploration is dangerous or expensive (healthcare, finance, recommendations). **Main problem: distribution shift**. The policy picks actions that are not in the data, and Q-values for those are wrongly optimistic. Fixes: conservative methods (CQL), policy constraints (BCQ, BEAR, TD3+BC), IQL.

Q: Off-policy evaluation (OPE)? Estimate how a new policy would perform using logged data from an old policy, before deploying. Methods: importance sampling (IPS), self-normalized IPS, direct method (model the reward), **doubly robust** (combines both). Requires logging action probabilities (propensities).

Q: Imitation learning? Learn from expert demonstrations. **Behavioral cloning** = supervised learning on (state, expert action). It is simple, but errors build up when the agent drifts to unseen states. **DAgger** fixes this by asking the expert to label states the agent visits. **Inverse RL** learns the reward function from demonstrations.

7. Multi-armed & contextual bandits

Q: What is a multi-armed bandit? RL with a single state: choose among K arms (ads, headlines, prices) to maximize total reward. There are no state transitions, only the exploration/exploitation trade-off.

Q: What is regret? Total reward lost compared to always pulling the best arm. Good algorithms have regret growing as O(log T) (sublinear).

Q: Main bandit algorithms? | Algorithm | Idea | |---|---| | ε-greedy | random arm with prob ε; simple, but wastes exploration on clearly bad arms | | UCB | pick arm with highest `mean + c·sqrt(log t / n_arm)` (optimism under uncertainty) | | Thompson sampling | keep a posterior per arm (Beta for clicks), sample, pick the max; works very well in practice | | Contextual (LinUCB, neural bandits) | reward depends on context (user features) |

Q: Bandit vs A/B test? A/B test: fixed split, clean statistical answer, "regret" during the test. Bandit: shifts traffic to winners automatically → less regret, but harder statistical inference and sensitive to delayed rewards and non-stationarity. Use A/B for decisions you must measure precisely; bandits for continuous optimization (headlines, creatives, recommendations exploration).

Q: Contextual bandit vs full RL? Contextual bandit assumes the action doesn't affect future states (each recommendation is independent). Full RL is needed when actions change the future (long-term user engagement, inventory).

Q: Thompson sampling for click-through (code idea)?

```
import numpy as np
alpha = np.ones(K); beta = np.ones(K)       # Beta(1,1) prior per arm
def choose():
    return np.argmax(np.random.beta(alpha, beta))
def update(arm, clicked):
    alpha[arm] += clicked; beta[arm] += 1 - clicked
```

8. RL for LLMs (RLHF and beyond)

Q: How is RL used in LLM training? In **RLHF**: train a reward model on human preference pairs, then optimize the LLM (the policy) with PPO to maximize reward, with a **KL penalty** to the SFT model. See 09-nlp-and-llms.md.

Q: In RLHF, what are the state, action, and reward? State = prompt + tokens so far. Action = next token. Reward = reward model score at the end of the response (minus KL penalty per token).

Q: Why the KL penalty? Without it the model drifts far from natural language to exploit reward model weaknesses (**reward hacking**: overly long, flattering, repetitive outputs).

Q: Alternatives to PPO for LLMs? DPO (no RL loop, directly on preference pairs), GRPO (group-relative advantages, no critic; used for reasoning models), RLAIF (AI feedback instead of human), **RL with verifiable rewards** (math answers checked, code run against tests).

Q: What is reward hacking in general? The agent finds a way to get high reward that the designer didn't intend (a boat racing game agent circling to collect points instead of finishing). The reward is a proxy; the agent optimizes the proxy, not the intention (**Goodhart's law**).

9. RL in production & sim-to-real

Q: Where is RL used in industry? Recommendations and ads (bandits, long-term value), robotics and control, data center cooling, chip placement, trading/execution, games, LLM alignment, dynamic pricing, supply chain/inventory, traffic signal control.

Q: Why is RL hard in production? Needs lots of interaction; exploration can hurt real users or damage hardware; reward design is hard; results are high-variance and hard to reproduce; hard to evaluate offline.

Q: Sim-to-real gap? A policy trained in simulation fails on real hardware because physics, sensors, latency, and friction differ. Fixes: **domain randomization** (randomize physics, textures, noise in sim), system identification (make sim match reality), fine-tune with small real data, robust policies, adding sensor noise and actuation delay in sim.

Q: Safe RL? Constraints on actions (action masking, safety layers), constrained MDPs (cost limits), human oversight, conservative offline training before online, shadow mode.

Q: How do you make RL experiments trustworthy? Run multiple seeds (5-10), report mean ± confidence interval, use the same environment versions, log hyperparameters, compare with strong baselines (random, heuristic, tuned PPO).

---

10. What-if scenarios

Use the C.L.H.I.F.P. method from 03-debugging-framework.md. For RL the "layer map" is: **environment** → **observation/state** → **reward** → **policy/value network** → **algorithm/hyperparameters** → **evaluation**.

The reward is not increasing at all

Causes (ranked): reward is too sparse (agent never sees a positive reward); bug in the environment or reward function (wrong sign, reward never returned, `done` flag wrong); observations not normalized; learning rate wrong; exploration too low (ε decayed too fast, entropy too low); network too small or wrong architecture. **Isolate:** test on a trivial version of the task; run a random policy and a hand-coded policy. Do they get different rewards? If not, the reward is broken. Print reward per step; check episode lengths; check observations aren't constant. **Fix:** reward shaping or curriculum, curiosity bonus, demonstrations; normalize observations and rewards; tune LR, entropy coefficient; verify on a known-solvable env first with a reference implementation (e.g., Stable-Baselines3). **Prevent:** unit tests for env (reset/step contract), sanity runs on simple envs in CI.

The agent gets high reward but behaves badly (reward hacking)

Causes: reward is a proxy that misses the true goal; loophole in environment (physics bug, scoring bug); shaped reward dominates the real objective. **Isolate: watch videos/rollouts**. Never trust only the reward curve. Compare reward with true task metrics (task success rate). **Fix:** fix loopholes; redesign the reward (penalize the exploit, use potential-based shaping); add constraints; use human preferences or demonstrations to define the goal; KL penalty to a reference behavior. **Prevent:** track a separate "true success" metric; red-team the environment; human review of sample behavior.

Training is unstable: reward goes up, then collapses

Causes: too-large policy updates (LR too high, PPO clip too wide); Q-value divergence (deadly triad); replay buffer too small (forgets) ; non-stationary targets (target network updates too often); catastrophic forgetting of earlier skills. **Isolate:** plot KL between policy updates, value loss, Q-value magnitudes, entropy, gradient norms. Exploding Q-values → divergence; entropy crash → premature convergence. **Fix:** lower LR, gradient clipping, smaller clip ε, more frequent checkpoints (keep best), Double DQN, slower target updates (Polyak), larger replay buffer, reward scaling/normalization. **Prevent:** early-stopping on KL, automatic checkpoint selection on evaluation reward, multiple seeds.

Results vary hugely between random seeds

Normal in RL to some degree. Report mean ± CI over many seeds; tune for robustness, not the best seed; check for bugs that make some seeds fail entirely (e.g., agent stuck at start position).

Works in simulation, fails on the real robot

Causes: physics mismatch (friction, mass), sensor noise and latency, actuator delays, visual differences, unmodeled situations. **Isolate:** replay real sensor logs through the policy; compare sim vs real trajectories for the same commands. **Fix:** domain randomization, add noise/delay to sim, system identification, fine-tune on real data with safety limits, residual policies on top of classical controllers. **Prevent:** test across a wide range of randomized sim conditions before hardware; staged rollout (lab → controlled field → full).

A bandit is stuck showing the same option (exploiting too early)

Causes: ε decayed to ~0; Thompson/UCB posterior overconfident (too much prior weight, rewards counted wrongly); **delayed rewards** not yet logged so other arms look bad; non-stationarity (the best arm changed but old data dominates); position bias (the shown arm gets more clicks just from being shown). **Isolate:** look at pull counts and posterior widths per arm; check reward logging delay; check whether arm performance changes over time. **Fix:** keep a minimum exploration rate; use sliding windows or discounting for non-stationarity; handle delayed feedback (attribute rewards correctly); reset or widen priors for new arms. **Prevent:** monitor share of traffic per arm and alert when one arm gets 100% too early.

Offline RL policy looks great in evaluation but performs badly when deployed

Causes: OPE estimates wrong (missing or wrong propensities, high variance IPS); policy takes actions not covered by the logged data (distribution shift); environment changed since data collection. **Fix:** conservative offline RL (CQL/IQL), constrain policy close to behavior policy, doubly robust OPE with confidence intervals, small online A/B test before full rollout.

Training is too slow (needs billions of steps)

Use vectorized/parallel environments, faster simulators (GPU physics), off-policy algorithms with replay (more sample-efficient), model-based methods, pretraining with imitation learning, smaller observation (state features instead of pixels), frame skipping.

RLHF model becomes very verbose and sycophantic

Reward model prefers long, agreeable answers (bias in human labels). Fix: length-normalize/penalize, add counter-examples to preference data, increase KL penalty, train the reward model on harder comparisons, evaluate with length-controlled win rates.

The agent works in training environment but fails on slightly different levels

Overfitting to the training environment. Train on many procedurally generated levels, domain randomization, data augmentation on observations, evaluate on held-out levels.

17. Time Series & Forecasting

Asked in retail, supply chain, finance, energy, ride-sharing, cloud capacity planning, and IoT interviews.

Contents

1. Basics
2. Stationarity & autocorrelation
3. Classical models
4. Machine learning for time series
5. Deep learning & foundation models
6. Validation: backtesting
7. Metrics
8. Probabilistic forecasting
9. Hierarchical & intermittent demand
10. Time series anomaly detection
11. System design: demand forecasting platform
12. What-if scenarios (forecasting)

---

1. Basics

Q: What makes time series different from normal tabular data? Observations are ordered and **dependent** on the past. You can't shuffle data randomly, and the future may behave differently from the past.

Q: Components of a time series? - **Trend:** long-term increase/decrease. - **Seasonality:** repeating pattern with fixed period (daily, weekly, yearly). - **Cycles:** longer, irregular ups and downs (economic cycles). - **Noise / residual:** random variation. Decomposition: additive `y = T + S + R` or multiplicative `y = T × S × R` (when seasonal swings grow with the level). STL is a robust decomposition method.

Q: Univariate vs multivariate forecasting? Univariate uses only the series' own history. Multivariate uses other series or covariates (price, promotions, weather, holidays).

Q: Types of covariates? Known future (calendar, holidays, planned price/promotions), **observed only in the past** (actual weather, competitor sales), **static** (store size, product category). Only known-future covariates can be used directly for future time steps.

Q: Forecast horizon and granularity? Horizon = how far ahead (1 day, 12 weeks). Granularity = time step (hourly, daily). Pick based on the decision: staffing needs hourly next-week; factory planning needs weekly next-quarter.

Q: Point vs probabilistic forecast? Point = one number. Probabilistic = distribution or quantiles (P10, P50, P90). Business decisions (inventory, capacity) usually need uncertainty.

Q: Single (local) model vs global model? Local: one model per series (ARIMA per product). Global: one model trained across many series (LightGBM or deep model over all products). It shares patterns, handles

short series better, easier to maintain at scale.

Q: Recursive vs direct multi-step forecasting? Recursive: predict t+1, feed it back to predict t+2 (errors compound). Direct: separate model (or output) per horizon. MIMO/seq2seq: predict all horizons at once.

2. Stationarity & autocorrelation

Q: What is stationarity? Mean, variance, and autocorrelation don't change over time. Many classical models (ARMA) assume it.

Q: How to test and fix non-stationarity? Test: plot, rolling mean/std, **ADF** test (null: non-stationary) or **KPSS** (null: stationary). Fix: **differencing** ($y_t - y_{t-1}$), seasonal differencing ($y_t - y_{t-7}$), log/Box-Cox transform to stabilize variance, detrending.

Q: ACF vs PACF? ACF: correlation of the series with its lags (includes indirect effects). PACF: correlation with a lag after removing effects of shorter lags. Use for ARIMA order: PACF cuts off after p → AR(p); ACF cuts off after q → MA(q).

Q: What is a random walk and why does it matter? $y_t = y_{t-1} + \text{noise}$. Best forecast is the last value (**naive forecast**). Many financial series behave like this, so always compare models with a naive baseline.

Q: Baselines you must always try? Naive (last value), seasonal naive (value same time last week/year), moving average, and drift. If your model doesn't beat seasonal naive, it isn't useful.

3. Classical models

Q: ARIMA(p, d, q)? AR(p): regression on past values. I(d): differencing d times. MA(q): regression on past forecast errors. **SARIMA** adds seasonal terms (P, D, Q, m). **ARIMAX/SARIMAX** adds external regressors. Select orders with ACF/PACF or auto-ARIMA (AIC).

Q: Exponential smoothing (ETS)? Forecast = weighted average of past values with weights decaying exponentially. Simple (level), Holt (level + trend), **Holt-Winters** (level + trend + seasonality). Fast, robust, strong baseline.

Q: Prophet? Additive model: trend (with changepoints) + seasonality (Fourier terms) + holidays. Easy for business users, handles missing data and holidays; often beaten by tuned ML models but good quick baseline.

Q: Limits of classical models? One model per series (doesn't scale to millions of SKUs easily), limited use of covariates, linear assumptions, struggle with multiple seasonalities and short histories.

Q: VAR? Vector autoregression: multiple series that influence each other (e.g., interest rates and inflation).

4. Machine learning for time series

Q: How do you use gradient boosting (LightGBM/XGBoost) for forecasting? Turn the series into a supervised table: - **Lag features:** y_{t-1}, y_{t-7}, y_{t-28} - **Rolling stats:** mean/std/min/max of last 7/28 days (shifted so they only use past data!) - **Calendar:** day of week, month, holiday flags, days to next

holiday, payday - **Known future covariates:** price, promotion, planned events - **Static:** store, category (as categorical or target-encoded) One global model over all series. This won many forecasting competitions (e.g., M5).

Q: Biggest bug when building lag features? Leakage: rolling windows that include the current or future value, or features computed on the full dataset. Always shift before rolling, and make sure the feature exists at prediction time for each horizon (for horizon 7, lag-1 is not available!).

Q: How do you handle multiple horizons with tree models? Direct approach (one model per horizon or horizon as a feature with lags ≥ horizon), or recursive (predict and feed back).

Q: Target transformations? Log1p for skewed sales, scaling per series (divide by series mean) so a global model treats big and small series fairly; predict the difference/ratio from a baseline.

5. Deep learning & foundation models

Model	Key idea
RNN / LSTM / GRU, DeepAR	sequence models; DeepAR = probabilistic global RNN (Amazon)
Temporal CNN (TCN, WaveNet-style)	dilated causal convolutions
N-BEATS / N-HiTS	pure MLP blocks with backcast/forecast, interpretable trend/seasonal stacks
Temporal Fusion Transformer (TFT)	handles static, known-future and past covariates; attention + variable selection → interpretable
PatchTST, Informer, Autoformer	Transformers for long horizons; patching inputs
Foundation models (TimesFM, Chronos, Moirai, Lag-Llama)	pretrained on huge collections of series → **zero-shot** forecasting

Q: When does deep learning beat gradient boosting? Many related series, long histories, complex covariate interactions, need for probabilistic multi-horizon output. For small/medium tabular-style problems, LightGBM is often equal or better and much simpler.

Q: When use foundation models? Quick baseline, little history, many diverse series, no time to build features. Fine-tune or ensemble with domain models; always backtest, because a zero-shot model doesn't know your promotions.

6. Validation: backtesting

Q: Why not random k-fold? Future data would leak into training; you'd be "predicting the past from the future" → over-optimistic results.

Q: How do you validate a forecasting model? Rolling-origin backtesting (time-series cross-validation): train on data up to time T, forecast the next h steps, move T forward, repeat. Expanding window (all history) or sliding window (only recent history).

```
Fold 1: [train.........][test]
Fold 2: [train.............][test]
```

```
Fold 3: [train.................][test]
```

Add a **gap** between train and test if features need time to become available (data delays).

Q: What should the backtest simulate? Exactly the production setup: the same horizon, the same data availability (data that arrives late in production must be missing in the backtest), the same retraining frequency.

7. Metrics

Metric	Formula idea	Notes
MAE	mean $\|y - \hat{y}\|$	in units, robust, easy to explain
RMSE	sqrt(mean $(y - \hat{y})^2$)	punishes big errors
MAPE	mean $\|y - \hat{y}\| / \|y\|$	%; **explodes when y is near 0**, biased toward under-forecasting
sMAPE	$\|y-\hat{y}\| / ((\|y\|+\|\hat{y}\|)/2)$	symmetric-ish, still unstable near 0
WAPE	$\Sigma\|y - \hat{y}\| / \Sigma\|y\|$	% of total volume; common in retail, stable
MASE	MAE / MAE of seasonal naive	<1 means better than naive; scale-free, safe near 0
Bias	$\Sigma(\hat{y} - y) / \Sigma y$	systematic over/under-forecasting
Quantile (pinball) loss	asymmetric loss per quantile	for probabilistic forecasts
CRPS		full distribution quality
Coverage	% actuals inside prediction interval	P10-P90 should cover ~80%

Q: How to pick a metric? From the business cost. Over-stock vs stock-out costs differ → forecast a quantile (e.g., P70) and evaluate with pinball loss. Report error weighted by volume or revenue so important items count more.

8. Probabilistic forecasting

Q: How to produce uncertainty? Quantile regression (LightGBM with quantile objective, one model per quantile), distributional models (DeepAR predicts parameters of a distribution), conformal prediction (calibrated intervals from backtest residuals), bootstrapping residuals, ensembles.

Q: How do you use a quantile forecast for inventory? Newsvendor logic: order up to the quantile = cost_understock / (cost_understock + cost_overstock). If stock-outs cost 3× more than leftovers → P75.

Q: Count data? Use Poisson/negative-binomial/Tweedie losses (LightGBM `tweedie` objective is popular for retail sales with many zeros).

9. Hierarchical & intermittent demand

Q: What is hierarchical forecasting? Series form a hierarchy (product → category → total; store → region → country). Forecasts at different levels must add up (**coherence**). Approaches: bottom-up (sum low-level forecasts; noisy), top-down (split the total by proportions; loses detail), middle-out, and **reconciliation** (MinT, which combines all levels optimally).

Q: Intermittent demand (many zeros)? Spare parts, slow-moving SKUs. Methods: Croston's method (forecast size and interval separately), SBA/TSB variants, Tweedie/zero-inflated models, aggregate to weekly/monthly. Evaluate with MASE/pinball, not MAPE.

Q: Cold start: a new product with no history? Use similar products (by attributes/embeddings) as analogs, category-level curves, launch plans and price as features, global models with static features; update quickly as real sales arrive.

10. Time series anomaly detection

Q: Approaches? - Statistical: z-score on residuals after removing seasonality, rolling median + MAD, control charts. - Forecast-based: anomaly if actual is outside the prediction interval. - ML: Isolation Forest on window features, autoencoders/LSTM reconstruction error, matrix profile. **Q: Common difficulties?** Seasonality causes false alarms (Monday spikes are normal), gradual drifts vs spikes, labels are rare → evaluate with injected anomalies and human feedback, **alert fatigue** → tune thresholds by severity and duration.

11. System design: demand forecasting platform

> "Forecast daily demand for 50,000 products × 1,000 stores for the next 8 weeks, to drive replenishment."

Requirements: 50M series × 56 days; daily refresh before 6 AM; probabilistic (P50, P90); explain big changes to planners; handle promotions, holidays, new products.

Design:

```
Sources: POS sales, inventory (stock-outs!), prices, promo calendar, holidays, weather, product
catalog
   → Data lake → daily batch ETL (Spark) → data quality checks (missing stores, late files)
   → Feature pipeline (lags, rolling stats, calendar, promo, price elasticity features)
   → Training (weekly): global LightGBM per category cluster + quantile models
       → backtest gate vs current model and seasonal naive (WAPE, bias, pinball)
   → Inference (daily, batch, distributed): 50M series × horizons
   → Reconciliation (store/region/category totals)
   → Post-processing: business overrides (planner adjustments), caps
   → Forecast store (warehouse table) → replenishment system / planner UI
Monitoring: accuracy by category/store as actuals arrive, bias, drift, missing data alerts
```

Key deep dives: - **Stock-outs censor demand:** sales = 0 because shelf was empty, not because nobody wanted it → mark and impute or exclude those days. - **Promotions & cannibalization:** promo on one product reduces sales of substitutes. - **Scale:** batch inference parallelized by store/category; trees are cheap. - **Human-in-the-loop:** planners override for known events; track whether overrides improve accuracy (they often don't).

---

12. What-if scenarios

The forecast broke after a big shock (COVID-like event, new competitor, a massive holiday shift)

Causes: structural break. The past no longer represents the future; models use long history that is now misleading; lag features copy abnormal values forward. **Isolate:** plot residuals over time; check if error rose for all series at once (global shock) or some segments. **Fix (now):** shorter training window or heavier weight on recent data, manual planner overrides, switch to faster-adapting models (exponential smoothing), widen prediction intervals. **Fix (long-term):** add "shock" indicator features, exclude or flag the abnormal period in training later, scenario forecasting, retrain more often. **Prevent:** monitor rolling error and bias with alerts; have a playbook for regime changes.

Backtest accuracy is amazing, production accuracy is bad

Causes (ranked): leakage from the future (rolling features include current day, features computed with full data, using actual promo/price that wasn't known at forecast time, using revised data instead of first-reported data); backtest horizon differs from production; data arrives late in production (yesterday's sales not yet loaded at 6 AM). **Isolate:** recreate a past forecast exactly "as of" that date with point-in-time data and compare to the backtest output. **Fix:** point-in-time feature store, shift features, simulate data latency in backtests, gap between train and test.

New product with no history

See §9 cold start: analog products, attribute-based global model, category curves, launch plan inputs, fast updating with first sales, wider intervals.

MAPE is huge and unstable

Cause: many actual values near zero (low-volume items, intermittent demand), so you divide by tiny numbers. **Fix:** switch to WAPE, MASE, or weighted metrics; evaluate low-volume items separately; aggregate level for reporting; use pinball loss for decisions.

Forecast is accurate overall, but bad for specific stores/products

Causes: global metric dominated by big series; segment has different behavior (tourist store, new store, local events); missing local features. **Isolate:** slice errors by store, region, category, volume tier; look at the worst 1%. **Fix:** segment-specific models or features (store embeddings, local events), cluster series and train per cluster, weight training by business value.

Forecast is systematically too high (bias)

Causes: stock-out days not handled (actually under-forecast then), trend extrapolation, promotions in training not in future, loss function (MSE on skewed data), planners' overrides. **Fix:** bias metric in monitoring, bias correction, check transformations (log → back-transform needs bias correction), censored demand handling.

Daily forecast job misses the 6 AM deadline

Upstream data late (add data-arrival SLAs, fall back to yesterday's forecast with flag), job too slow (parallelize by partition, pre-compute features incrementally, smaller models for long tail), monitor job duration trend.

Planners don't trust the model and override everything

Show explanations (drivers: promo, holiday, trend), forecast-value-added analysis (compare accuracy with and without overrides), involve planners in error reviews, allow structured inputs (events) instead of free overrides.

Anomaly detector raises too many false alarms

Seasonality not removed, thresholds too tight, alerting on single points. Use seasonal decomposition, require persistence (N points), severity tiers, feedback labels to tune.

Energy/traffic forecast fails on special days (holidays, sports events)

Add event calendars and "days to/from event" features, similar-day methods (use last year's same holiday), separate model for special days.

18. Recommender Systems (Deep Dive)

Asked at e-commerce, streaming, social media, food delivery, ads, news, jobs and dating companies. It is one of the most common ML system design topics. For a full design answer see 05-case-studies.md.

Contents

1. Basics
2. Collaborative filtering & matrix factorization
3. Content-based & hybrid
4. The multi-stage architecture
5. Candidate generation: two-tower & ANN
6. Ranking models & features
7. Training data, negatives & losses
8. Sequential & real-time recommendation
9. Evaluation
10. Bias, feedback loops & exploration
11. Re-ranking & multi-objective
12. What-if scenarios (recommenders)

---

1. Basics

Q: What does a recommender system do? Predicts which items (products, videos, songs, jobs) a user will find valuable, and ranks them. Business goals: engagement, conversion, retention, revenue, discovery.

Q: Explicit vs implicit feedback? Explicit: ratings, likes (rare, clear meaning). Implicit: clicks, watch time, purchases, skips (plentiful, noisy, no true negatives: "not clicked" may mean "not seen").

Q: Main approaches? Popularity-based, collaborative filtering (users who liked X also liked Y), content-based (item attributes), hybrid, deep learning (two-tower, sequence models), knowledge/graph-based, and bandits/RL for exploration.

Q: What is the user-item interaction matrix? Rows = users, columns = items, values = interactions. It's extremely **sparse** (users interact with a tiny fraction of items).

2. Collaborative filtering & matrix factorization

Q: User-based vs item-based collaborative filtering? User-based: find similar users, recommend what they liked. Item-based: find items similar to ones the user liked (based on co-interaction). Item-based is more stable (item similarities change slowly) and easier to precompute. Amazon used it early on.

Q: Similarity measures? Cosine, Pearson correlation, Jaccard (for binary interactions).

Q: Matrix factorization? Approximate the interaction matrix $R \approx U \cdot V^T$, where each user and item gets a low-dimensional vector (embedding). Predicted score = dot product. Train with SGD or **ALS** (alternating least squares; scales well, good for implicit data).

Q: Implicit-feedback MF? Treat all interactions as positive with confidence weights (more views → more confidence), unobserved as weak negatives (Hu-Koren-Volinsky). Or use ranking losses (BPR).

Q: Weaknesses of collaborative filtering? Cold start (new users/items have no interactions), popularity bias, sparsity, can't use content/context features directly.

Q: Factorization Machines (FM)? Model pairwise interactions between all features (user, item, context) with factorized weights. It generalizes MF to include side features. Extensions: FFM, DeepFM.

3. Content-based & hybrid

Q: Content-based recommendation? Represent items by features (text, category, image embeddings, tags) and recommend items similar to what the user consumed. Works for new items; risk: over-specialization (only "more of the same").

Q: How do you create item embeddings from content? Text encoders (sentence embeddings of title/description), image encoders (CLIP), audio embeddings, or LLM-generated attribute tags.

Q: Hybrid systems? Combine CF and content signals, e.g., content features as inputs to a two-tower model, or blend scores. Best of both: personalization + cold-start handling.

Q: Can LLMs be used in recommendation? Yes: generating item descriptions/tags, embeddings, conversational recommendation, explaining recommendations, re-ranking small candidate sets, and generative retrieval. Main limits: latency and cost at large scale, so usually used offline or at the last stage.

4. The multi-stage architecture

Q: Why multiple stages? You can't run a heavy model on millions of items per request within 100-200 ms. So you narrow down step by step:

```
All items (10M)
  → Candidate generation / retrieval (≈ 1,000)   cheap: ANN on embeddings, co-occurrence,
popular, followed
  → (Light pre-ranking (≈ 300))                  small model
  → Ranking (≈ 100)                              heavy model with rich features
  → Re-ranking / business logic (≈ 20 shown)     diversity, freshness, dedupe, ads, policy
filters
```

Q: Why multiple candidate sources? Each captures different intent: personalized embeddings, "because you watched X", trending, new items, followed creators, location-based. Merging them increases recall and diversity.

Q: Online vs offline computation? Precompute item embeddings and some user candidates offline (batch). Compute user embedding, retrieval and ranking online for freshness. Inactive users can get fully precomputed lists.

5. Candidate generation: two-tower & ANN

Q: What is a two-tower model? A **user tower** (user features + history → embedding) and an **item tower** (item features → embedding). Score = dot product. Trained so that interacted pairs score high. **Why it's popular:** item embeddings are precomputed and indexed in an ANN index; at request time compute one user embedding and search → millisecond retrieval over millions of items.

Q: Limitations of two-tower? No feature interaction between user and item before the final dot product → less accurate than a ranking model. That's why it's used for retrieval, not final ranking.

Q: ANN indexes for retrieval? HNSW, IVF, ScaNN, FAISS. Trade-off recall vs latency vs memory. Refresh index as new items arrive (incremental updates or frequent rebuilds). See 02 §7.

Q: Other retrieval methods? Item-to-item co-occurrence ("bought together"), graph-based (random walks, PinSage), rule-based (trending in your city), and search queries.

6. Ranking models & features

Q: Ranking model types? Logistic regression → GBDT → deep models: **Wide & Deep**, **DeepFM**, **DCN** (deep & cross network), **DIN** (attention over user history relative to the candidate item), multi-task models (**MMoE**, PLE) predicting several outcomes at once.

Q: What does the ranking model predict? Probabilities of actions: P(click), P(watch > 30 s), P(purchase), P(like), P(share), P(hide/skip). Final score combines them (see §11).

Q: Feature categories? | Type | Examples | |---|---| | User | age bucket, country, device, long-term interests, embeddings | | Item | category, price, age of item, quality score, creator, embeddings | | User × item | past interactions with this creator/category, similarity between user and item embeddings | | Context | time of day, day of week, location, session length, page | | Real-time | last 10 items viewed, clicks in this session | | Counters | item CTR last 1 h / 24 h, popularity trend |

Q: How to handle high-cardinality IDs? Embedding tables (hashing trick to cap size), frequency thresholds, shared embeddings, mixed-dimension embeddings.

Q: Why calibrate ranking scores? When combining multiple predictions or using them in auctions (ads), probabilities must be accurate, not only correctly ordered. Negative downsampling during training distorts probabilities → recalibrate.

7. Training data, negatives & losses

Q: Where do labels come from? Logged impressions with outcomes: shown and clicked = positive; shown and not clicked = negative (for ranking). Only items **shown** get labels → selection bias.

Q: Negative sampling for retrieval? - **Random negatives:** cheap, easy negatives (model learns popular vs unpopular). - **In-batch negatives:** other users' positives in the batch. Efficient, but biased toward popular items → apply **logQ correction**. - **Hard negatives:** items similar to positives but not engaged. They improve precision; if too hard, you get false negatives. - Mix of easy + hard is common.

Q: Losses? | Loss | Use | |---|---| | Binary cross-entropy (pointwise) | CTR prediction in ranking | | **BPR** (pairwise) | score(positive) > score(negative) for implicit feedback | | **Sampled softmax** | retrieval: positive vs many sampled negatives | | Listwise (LambdaRank, softmax over list) | optimize ranking metrics like nDCG | | Contrastive / InfoNCE | two-tower training |

Q: Point-in-time correctness? Features must be computed as they were at impression time (e.g., item CTR at that moment), otherwise leakage. Log features at serving time and train on them.

8. Sequential & real-time recommendation

Q: Sequence-aware models? Treat user history as a sequence and predict the next item: GRU4Rec (RNN), **SASRec** (self-attention, causal), BERT4Rec (masked items), Transformers with side features. Capture order and short-term intent ("watching a series in order").

Q: Session-based recommendation? For anonymous users or short sessions: use the items clicked in the current session only (session-kNN, GRU4Rec).

Q: Real-time personalization architecture?

```
User events → Kafka → stream processor (Flink) → update online features (recent items, counters)
in Redis
Request → fetch fresh features → retrieval + ranking → response
```

Also: update user embedding online (from recent items) without retraining, near-real-time training (hourly incremental updates).

Q: Generative recommenders? Large sequence models trained on user action sequences (and semantic item IDs) that generate the next items directly. Large companies are actively working on this.

9. Evaluation

Q: Offline metrics? - Retrieval: **Recall@K**, hit rate@K. - Ranking: **nDCG@K**, MAP, MRR, AUC / GAUC (AUC per user, averaged), log loss. - Beyond accuracy: coverage (% catalog recommended), diversity, novelty, serendipity, popularity bias.

Q: Offline evaluation protocol? Time-based split (train on past, test on future), never random split. Leave-last-out per user for sequential models. Evaluate on all candidates or a fair sample (sampled metrics can mislead).

Q: Online evaluation? A/B tests with primary metric (watch time, purchases, retention) + guardrails (latency, diversity, complaints, creator-side metrics). **Interleaving** (mix results from two rankers in one list, see which gets more clicks). It needs 10-100× less traffic to detect a winner, good for fast ranker iteration.

Q: Why do offline and online results disagree? Logged data only contains items the old model showed (selection bias); offline metrics ignore position effects, diversity, freshness, latency; short-term clicks ≠ long-term satisfaction. See 04 #28.

Q: Counterfactual/off-policy evaluation? Use logged propensities with IPS or doubly robust estimators to estimate a new policy's performance offline. See 16 §6.

10. Bias, feedback loops & exploration

Q: Types of bias? | Bias | Meaning | Mitigation | |---|---|---| | **Position bias** | items at top get clicked more just because of position | position as feature during training (set fixed at serving), inverse propensity weighting, randomization experiments | | **Popularity bias** | popular items get recommended more, get more data, get even more popular | logQ correction, re-ranking for long tail, exposure fairness | | **Selection/exposure bias** | only shown items have labels | exploration traffic, IPS weighting | | **Conformity / presentation bias** | UI influences behavior | careful logging, experiments |

Q: What is a feedback loop? The model decides what users see → users interact only with that → training data reflects the model's own choices → model reinforces itself, reducing diversity (filter bubble).

Q: Exploration methods in recsys? Reserve a small % of slots for exploration (ε-greedy), bandits (Thompson sampling for new items), boost new items with uncertainty bonuses, randomized holdout traffic for unbiased data.

11. Re-ranking & multi-objective

Q: How to combine multiple objectives? Weighted score: `score = w1·P(click) + w2·E[watch time] + w3·P(like) − w4·P(hide)`; tune weights with online experiments; or constrained optimization (maximize engagement subject to diversity/quality constraints); Pareto-optimal tuning.

Q: Diversity in re-ranking? MMR (maximal marginal relevance: trade relevance against similarity to already-picked items), DPP (determinantal point processes), category caps ("max 2 items from same creator in top 10").

Q: Other re-ranking rules? Remove already-purchased/watched, dedupe near-duplicates, freshness boosts, policy/safety filters, business rules (sponsored slots, inventory availability), fairness for creators/sellers.

---

12. What-if scenarios

Already covered in 04-what-if-scenarios.md: popular items for everyone (#47), cold start (#48), CTR dropping (#49), clickbait optimization (#50). More below.

Users complain "it only shows me the same type of content" (filter bubble)

Causes: feedback loop; ranking only optimizes short-term engagement; no diversity constraints; retrieval sources too narrow (all from user's top category). **Isolate:** measure diversity per user (unique categories in top 20), compare over time; check share of candidates from each source. **Fix:** diversity in re-ranking (MMR, category caps), exploration slots, add "explore" candidate source, long-term satisfaction objectives (retention, survey responses), user controls ("show less like this"). **Prevent:** diversity and novelty as guardrail metrics in A/B tests.

Recommendations are stale after a big event (breaking news, a viral video, a new season launched)

Causes: batch pipeline updates daily; item embeddings missing for new items (not in ANN index yet); popularity counters computed hourly; model trained on last week. **Fix:** streaming features (real-time counters), incremental ANN index updates, a "trending now" candidate source, content-based embeddings for new items available immediately, more frequent incremental training. **Prevent:** monitor "time from item creation to first recommendation" and share of fresh items shown.

Offline Recall@K improved a lot, but online metrics didn't move

Causes: ranking stage throws away the new candidates anyway (retrieval wasn't the bottleneck); offline eval biased toward items the old system showed; improvement concentrated in items users don't care about; latency increased. **Isolate:** check how many new candidates survive ranking and get shown; evaluate end-to-end (retrieval + ranking) offline; slice by user activity. **Fix:** joint evaluation of the full funnel, retrain ranker with new candidates (ranker must see them in training), improve logging of exposure.

Embedding table is too large for memory (hundreds of millions of users/items)

Fix: hashing trick (with collisions), frequency-based pruning (rare IDs share a bucket), mixed-dimension embeddings (popular IDs get larger vectors), quantization (fp16/int8), sharded embedding servers (parameter servers / TorchRec), replace user ID embeddings with history-based user towers (compute user vector from recent items → no per-user table).

The same item appears multiple times (or near-duplicates fill the feed)

Causes: multiple candidate sources return the same item; same content uploaded several times (re-posts, different IDs); variants of a product (colors/sizes). **Fix:** dedupe by item ID after merging sources; content hashing / embedding similarity to cluster near-duplicates; group product variants; diversity rules in re-ranking.

A new ranking model increased clicks but decreased purchases/retention

Optimizing the wrong proxy. Use multi-task objectives with downstream outcomes (purchase, return visits), weight by value, add guardrail metrics, run longer tests to see long-term effects (holdouts).

Recommendations look great for heavy users, terrible for light users

Causes: model dominated by heavy users' data; light users have few features. **Fix:** per-segment evaluation, GAUC, content/context features, popular-in-segment fallback, onboarding signals, separate model or features for light users.

Latency of the recommendation API is above the 200 ms budget

Trace per stage. Fix: cache user embeddings and candidate lists, reduce candidates to the ranker, pre-ranking stage, model distillation/quantization, feature fetch batching, parallel candidate sources with timeouts (drop a slow source instead of waiting).

Sellers/creators complain they get no exposure

Popularity bias and rich-get-richer. Add exposure fairness constraints, new-creator boosts with exploration, monitor exposure distribution (Gini coefficient), creator-side metrics in experiments.

Model suddenly recommends inappropriate items to children/sensitive users

Policy filters missing in re-ranking for that surface, new items not yet classified, features for age missing. Fix: hard filters by policy before ranking, default to safe when classification is pending, audit logging.

19. Speech & Audio AI

Asked for voice assistants, call-center AI, meeting transcription, voice agents, media, accessibility and smart-device roles.

Contents

1. Audio basics
2. Audio features
3. Speech recognition (ASR)
4. ASR evaluation
5. Text-to-speech (TTS)
6. Speaker tasks: diarization, verification
7. Wake word, VAD & audio classification
8. Data & augmentation
9. Voice agents: system design
10. Deployment
11. What-if scenarios (speech)

---

1. Audio basics

Q: What is a digital audio signal? A sequence of amplitude samples taken at a fixed rate (the **sampling rate**). Each sample is stored with some **bit depth** (16-bit is common).

Q: Common sampling rates? 8 kHz (telephone), 16 kHz (most speech models), 22.05/24 kHz (TTS), 44.1/48 kHz (music, video). **Nyquist:** sampling rate must be at least 2× the highest frequency you want to capture. Telephone audio (8 kHz) loses everything above 4 kHz → harder ASR.

Q: What breaks if sampling rates don't match? A model trained on 16 kHz fed 8 kHz audio (or vice versa) without resampling produces garbage. Always resample to the model's expected rate.

Q: Mono vs stereo? Speech models usually take mono. In call centers, stereo with **agent and customer on separate channels** makes diarization trivial, so keep it if you have it.

Q: Audio formats/codecs? WAV/PCM (uncompressed), FLAC (lossless), MP3/AAC/Opus (lossy). Telephony codecs (G.711 μ-law) and heavy compression reduce ASR accuracy.

2. Audio features

Q: What is a spectrogram? A time × frequency image showing energy at each frequency over time, computed with the **Short-Time Fourier Transform (STFT)** using small overlapping windows (e.g., 25 ms window, 10 ms hop).

Q: Mel spectrogram? Spectrogram with frequencies mapped to the **mel scale**, which matches human hearing (more resolution at low frequencies), usually with log amplitude. Standard input for modern speech models (e.g., 80 mel bins).

Q: MFCC? Mel-frequency cepstral coefficients. Apply a DCT on the log mel spectrogram to get a compact (13-40) decorrelated representation. Classic for older ASR/speaker models; deep models now mostly use log-mel or raw waveform.

Q: Raw waveform models? wav2vec 2.0, HuBERT learn features directly from the waveform with self-supervised pretraining.

3. Speech recognition (ASR)

Q: Classic ASR pipeline? Acoustic model (audio → phonemes, HMM-GMM then HMM-DNN) + pronunciation lexicon + language model (n-gram) + decoder (WFST). Complex but controllable.

Q: End-to-end ASR approaches? | Approach | Idea | Notes | |---|---|---| | **CTC** | predicts a label per frame including a "blank"; collapse repeats and blanks | no alignment needed; assumes frame outputs independent → benefits from external LM; streaming-friendly | | **Attention encoder-decoder** (LAS, Whisper) | decoder attends to encoded audio and generates text | high accuracy; not naturally streaming; can **hallucinate** | | **RNN-Transducer (RNN-T)** | encoder + prediction network + joiner | streaming, accurate; used in on-device assistants | | Hybrid CTC/attention | multi-task | stable training |

Q: Conformer? Encoder combining convolution (local patterns) and self-attention (global context). The standard ASR encoder.

Q: Self-supervised speech models? wav2vec 2.0 / HuBERT / WavLM: pretrain on unlabeled audio by predicting masked parts, fine-tune with little labeled data → great for low-resource languages.

Q: Whisper-style models? Encoder-decoder Transformer trained on huge weakly-labeled multilingual data; does transcription, translation, language ID, timestamps. Strengths: robust to noise and accents. Weaknesses: 30-second windows, not built for real-time streaming, can hallucinate text on silence/noise, slower than CTC models.

Q: Streaming vs batch ASR? Streaming outputs words while the user talks (needed for voice agents, live captions). It uses chunked/causal encoders, CTC or RNN-T, partial and final hypotheses. Batch processes full files and gets better accuracy (can see future context).

Q: Role of a language model in ASR? Rescoring or shallow fusion with an LM improves rare words and grammar. **Contextual biasing / hotwords** boost domain terms (product names, contact names).

Q: Punctuation and casing? Often a separate post-processing model (or built into models like Whisper). Also **inverse text normalization**: "twenty five dollars" → "$25".

4. ASR evaluation

Q: WER? Word Error Rate = (Substitutions + Deletions + Insertions) / number of words in reference. Can be > 100% (many insertions). **CER** (character error rate) for languages without spaces (Chinese, Japanese) or for names.

Q: What normalization before WER? Lowercase, remove punctuation, normalize numbers ("25" vs "twenty-five"), contractions. Otherwise WER penalizes formatting, not recognition.

Q: Beyond WER? Entity/keyword error rate (did we get the account number right?), latency (first partial, final result), real-time factor (RTF = processing time / audio duration), WER by segment (accent, noise level, device), downstream task success (intent accuracy).

5. Text-to-speech (TTS)

Q: TTS pipeline? Text normalization ("Dr." → "doctor", numbers, dates) → grapheme-to-phoneme (pronunciation) → acoustic model (text/phonemes → mel spectrogram) → **vocoder** (mel → waveform).

Q: Key TTS models? Tacotron 2 (attention seq2seq → mel), FastSpeech 2 (non-autoregressive, fast, duration predictor), **VITS** (end-to-end with VAE + flow + GAN, no separate vocoder), neural codec language models (generate audio tokens, zero-shot voice cloning from a few seconds), diffusion/flow-matching TTS.

Q: Vocoders? WaveNet (high quality, slow), WaveRNN, **HiFi-GAN** (fast, high quality, common), neural audio codecs (EnCodec/SoundStream) for token-based models.

Q: TTS evaluation? MOS (mean opinion score from human raters), intelligibility (run ASR on output → WER), speaker similarity (embedding cosine), latency (time to first audio), naturalness of prosody.

Q: Risks? Voice cloning misuse (deepfakes, fraud) → consent, watermarking, speaker verification protections.

6. Speaker tasks: diarization, verification

Q: Speaker diarization? "Who spoke when." Pipeline: VAD → segment → speaker embeddings (x-vectors, ECAPA-TDNN) → clustering (spectral, agglomerative) → handle overlaps. End-to-end neural diarization (EEND) handles overlapping speech. Metric: **DER** (diarization error rate: missed speech + false alarm + speaker confusion).

Q: Speaker verification vs identification? Verification: "is this the claimed person?" (1:1, voice login). Identification: "which of N people is this?" (1:N). Metric: **EER** (equal error rate). Threat: replay attacks and synthetic voices → anti-spoofing / liveness detection.

7. Wake word, VAD & audio classification

Q: Wake word detection ("Hey device")? Tiny always-on model on device (low power), small CNN/RNN on mel features; then cloud verification to reduce false accepts. Metrics: false accept rate per hour vs false reject rate.

Q: VAD (voice activity detection)? Detects speech vs non-speech. Critical for: cutting silence before ASR (cost, hallucination prevention), end-of-turn detection in voice agents, diarization.

Q: Audio classification / event detection? Classify sounds (glass breaking, cough, music genre, machine faults). Use log-mel + CNN (PANNs) or audio transformers (AST), or pretrained audio embeddings (CLAP, BEATs). Multi-label is common.

8. Data & augmentation

Q: Data augmentation for speech? SpecAugment (mask random time and frequency bands on the spectrogram; the most important trick), speed perturbation (0.9×/1.1×), adding background noise (MUSAN), room impulse responses (reverb), codec/telephony simulation, volume changes, pitch shift.

Q: Where does labeled data come from? Public datasets (LibriSpeech, Common Voice), in-house transcriptions, pseudo-labeling with a strong model + filtering by confidence, synthetic speech from TTS for rare words.

Q: Privacy concerns? Voice is biometric personal data. Consent, PII redaction in transcripts (card numbers, names), retention limits, on-device processing when possible.

9. Voice agents: system design

> "Design a phone voice agent that books appointments."

Pipeline (cascaded):

```
Caller audio (telephony / WebRTC, 8-16 kHz)
  → streaming VAD + end-of-turn detection
  → streaming ASR (partial + final transcripts, with hotwords for names/services)
  → LLM (dialogue + tools: check_availability, book_slot), stream tokens
  → streaming TTS (start speaking after first sentence/phrase)
  → audio back to caller
Barge-in: if the caller starts talking while TTS plays → stop TTS immediately, listen
Logging, transcripts with PII redaction, human handoff, analytics
```

Latency budget (target ~0.8-1.2 s from user stops talking → agent starts speaking): | Stage | Typical | |---|---| | End-of-turn detection | 200-500 ms (biggest hidden delay) | | ASR final | 100-300 ms | | LLM time to first token | 200-500 ms | | TTS time to first audio | 100-300 ms | | Network/telephony | 100-200 ms |

Ways to cut latency: streaming everywhere, smarter end-of-turn models (semantic + acoustic, not just silence timeout), start LLM on partial transcripts (speculatively), small/fast LLM for routine turns, short first sentence, filler phrases ("Let me check that…") while tools run, colocate services, cache common responses.

Alternative: speech-to-speech models (audio in, audio out, one model). Lower latency, better prosody and emotion, but less control, harder to debug, tool use and guardrails are harder. Cascaded is easier to inspect and improve per stage.

Deep dives to mention: dialogue state and slot confirmation ("Tuesday 3 PM, correct?"), spelling of names/emails (letter-by-letter mode, confirmation), DTMF keypad fallback, handling silence and noisy lines, human escalation, compliance (call recording disclosure), evaluation with simulated callers, metrics (task completion, average handle time, transfer rate, latency p95).

10. Deployment

- On-device: small models (quantized Conformer/RNN-T), wake word, privacy, offline use.
- Server: GPU batching for batch transcription, streaming servers with WebSockets/gRPC, autoscaling by concurrent streams.
- Cost drivers: audio hours processed; reduce with VAD (skip silence), smaller models for easy audio, batch processing for non-real-time.

---

11. What-if scenarios

Use the C.L.H.I.F.P. method from 03-debugging-framework.md. Speech layer map: **microphone/telephony → codec/sampling → VAD → ASR (acoustic + LM) → post-processing (punctuation, ITN) → downstream NLU/LLM → TTS → playback**.

High WER for users with certain accents

Causes: training data under-represents those accents; LM lacks local vocabulary/names; code-switching (mixing languages). **Isolate:** WER sliced by accent/region/language; look at error types (substitutions on names vs common words). **Fix (now):** contextual biasing/hotwords for local names, choose a more robust multilingual model. **Fix (long-term):** collect and transcribe accent-specific data, fine-tune (or adapters), pseudo-label large unlabeled local audio, code-switching training data. **Prevent:** per-segment WER dashboard; fairness checks before release.

Accuracy is terrible on noisy call-center audio

Causes: 8 kHz narrowband telephony while model trained on 16 kHz clean audio; background noise; heavy compression; crosstalk. **Isolate:** compare WER on clean vs call audio; check sampling rate pipeline (accidental resampling bugs, wrong μ-law decoding). **Fix:** use telephony-trained or fine-tuned models, augment training with noise + codec simulation, noise suppression (careful, it can hurt ASR), use stereo channel separation, hotwords for domain terms.

The voice bot is too slow: users talk over it or hang up

Trace each stage (see §9 budget). Usual culprits: end-of-turn silence timeout too long, non-streaming ASR or TTS, big LLM, sequential tool calls, cross-region hops. Fix with streaming, better turn detection, smaller model routing, fillers during tool calls, colocation. Monitor p95 "user stopped → bot started" latency.

ASR outputs text that was never said (hallucinations on silence or music)

Cause: encoder-decoder models (Whisper-style) generate plausible text when there's no speech (e.g., "Thank you for watching!"), repeat loops on long audio. **Fix:** VAD to remove non-speech before ASR, confidence/no-speech probability thresholds, compression-ratio and repetition checks, chunking with overlap, CTC/RNN-T models for critical uses, temperature fallback settings. **Prevent:** test sets with silence, music, hold music and noise.

Diarization mixes up speakers

Causes: similar voices, short turns, overlapping speech, wrong number of speakers estimated, mono audio with crosstalk. **Fix:** use separate channels when available, set known speaker count, overlap-aware models (EEND), longer minimum segment length, enroll known speakers (agent voice prints), post-correction with LLM using conversation context.

TTS mispronounces names, brands, or numbers

Fix: custom pronunciation lexicon / phoneme input (SSML `<phoneme>`), text normalization rules (read "2024" as year vs number), test sets of domain words, for multilingual names use language tags. Prevent: pronunciation regression tests.

Transcripts contain customers' card numbers and personal data

Security/compliance issue. Pause storage, redact PII (regex + NER on transcripts, audio bleeping), pause recording during payment (DTMF), restrict access, retention policy, audit.

Wake word triggers on TV ads or random speech

High false accept rate. Collect false-trigger samples (hard negatives) and retrain, two-stage verification (on-device + cloud), adjust thresholds, speaker-aware wake word, detect audio from TV (echo/far-field cues).

ASR misses domain terms (drug names, product codes)

Contextual biasing, custom LM/vocabulary, fine-tuning with domain audio, post-correction with fuzzy matching against a known entity list, LLM-based correction with constrained vocabulary.

Live captions lag and keep changing words

Partial hypotheses are unstable. Show stable prefixes only, tune chunk size (latency vs stability trade-off), use streaming-native models (RNN-T), finalize faster on pauses.

Voice cloning used to impersonate customers for account access

Add anti-spoofing (synthetic speech detection), multi-factor authentication instead of voice-only, liveness challenges (random phrases), monitor unusual patterns.

20. Graph Machine Learning

Graphs show up in many interviews in fraud detection, recommendations, social networks, drug discovery, knowledge graphs and GraphRAG. Format: **Q → short answer you can say in 30-60 seconds**, then "what if" scenarios at the end.

Contents

1. Graph basics
2. Graph tasks
3. Classic graph features
4. Node embeddings: DeepWalk and node2vec
5. Graph neural networks (GNNs)
6. Training and scaling GNNs
7. Knowledge graphs
8. GraphRAG
9. Use cases and system design
10. What-if scenarios

---

1. Graph basics

Q: What is a graph? A set of **nodes** (entities: users, products, molecules' atoms) and **edges** (relationships: follows, bought, bonded). Nodes and edges can have features.

Q: Directed vs undirected? Directed edges have a direction (A follows B). Undirected edges are mutual (A is friends with B).

Q: Weighted vs unweighted? Weighted edges carry a strength (number of transactions, distance).

Q: Homogeneous vs heterogeneous graph? Homogeneous: one node type and one edge type. Heterogeneous: many types (user, item, merchant; edges "bought", "reviewed", "shares-device-with"). Most real business graphs are heterogeneous.

Q: Bipartite graph? Two node types, edges only between types (users ↔ items). The standard shape for recommender data.

Q: How do you store a graph in memory? | Format | Good for | |---|---| | Adjacency matrix (N × N) | small dense graphs, math | | Adjacency list | sparse graphs, traversal | | Edge list (src, dst) | storage, input to GNN libraries | | CSR (compressed sparse row) | fast neighbor lookup at scale |

Q: Degree? Number of edges of a node. In-degree / out-degree for directed graphs. Real graphs have a **power-law degree distribution**: a few "hub" nodes with huge degree.

Q: Graph databases? Neo4j, Amazon Neptune, TigerGraph, JanusGraph. They store nodes and edges natively and support traversal queries (Cypher, Gremlin). Use them when the core queries are multi-hop relationships.

Q: Homophily vs heterophily? Homophily: connected nodes tend to be similar (friends share interests). Heterophily: connected nodes tend to differ (fraudsters connect to normal users). Many simple GNNs assume homophily and do worse on heterophily.

2. Graph tasks

Level	Task	Example
Node	classify / regress a node	Is this account a fraudster?
Edge (link)	predict if an edge exists / its type	Will user buy item? Friend suggestion? Drug-target interaction?
Graph	classify a whole graph	Is this molecule toxic?
Community	find groups	Fraud rings, customer segments
Generation	create new graphs	New molecules

Q: Transductive vs inductive learning? Transductive: all nodes (including test nodes) are present during training; can't handle new nodes without retraining (e.g., classic node2vec, basic GCN). Inductive: learns a function that works on unseen nodes/graphs (GraphSAGE). **Production systems with new users every minute need inductive models.**

Q: How do you split data for link prediction? Hide a fraction of edges as positives for validation/test, sample non-edges as negatives, and make sure hidden edges are **removed from the graph used for message passing**. Otherwise the model sees the answer (leakage). For time-based data, split by time.

3. Classic graph features

Before GNNs, and still very strong as features for gradient-boosted trees.

Q: Degree centrality? Number of neighbors (popularity).

Q: Betweenness centrality? How often a node lies on shortest paths between others ("bridges" and brokers). Expensive to compute exactly on big graphs.

Q: Closeness centrality? How close a node is to everyone else (inverse of average distance).

Q: PageRank? A node is important if important nodes link to it. Computed by a random walk with a damping factor (~0.85 chance to follow a link, 0.15 to jump randomly). Used for web ranking, influencer detection, fraud scoring (personalized PageRank from known fraudsters).

Q: Personalized PageRank? Random walk that restarts at specific seed nodes → scores how close every node is to the seeds. Great for "who is related to these known bad accounts?" and for recommendations.

Q: Clustering coefficient? How many of a node's neighbors are also connected to each other. It measures tight groups.

Q: Community detection algorithms? Louvain / Leiden (maximize modularity, fast, most used), label propagation (very fast), connected components (simplest, and often enough for fraud rings that share a device or card).

Q: Common neighbors / Jaccard / Adamic-Adar? Simple link prediction scores: two nodes with many shared neighbors are likely to connect. Strong baselines for friend suggestions.

Q: Why mention classic features in an interview? Because they are cheap, explainable, and often beat a GNN on a small budget. A good answer: *"I'd start with graph features (degree, PageRank, component size, shared-device counts) in XGBoost as a baseline, then try a GNN if it adds lift."*

4. Node embeddings: DeepWalk and node2vec

Q: What is a node embedding? A vector per node so that similar/connected nodes are close in vector space. Used as features or for nearest-neighbor search.

Q: DeepWalk? Do random walks on the graph, treat each walk as a "sentence" of node IDs, train Word2Vec (skip-gram) on them. Nodes that appear in similar walks get similar vectors.

Q: node2vec? DeepWalk with **biased** random walks controlled by p (return) and q (in-out). Low q → explore far (DFS-like, captures communities/homophily). High q → stay local (BFS-like, captures structural roles).

Q: Limitations of DeepWalk/node2vec? Transductive (new node → no embedding without retraining), ignore node features, one embedding per node means memory grows with graph size.

Q: Matrix factorization view? These methods are approximately factorizing a matrix of node co-occurrence, the same idea as collaborative filtering.

5. Graph neural networks (GNNs)

Q: Core idea of a GNN: message passing? Each layer, every node: 1. collects messages from its neighbors (their current vectors), 2. **aggregates** them (sum, mean, max, attention), 3. **updates** its own vector by combining with its previous vector through a small neural network. After k layers, a node's vector contains information from its **k-hop neighborhood**.

```
h_v(new) = UPDATE( h_v , AGGREGATE( { h_u : u in neighbors(v) } ) )
```

Q: Why not just use a normal neural network on the adjacency matrix? Graphs have no fixed size or node order. GNNs are **permutation invariant/equivariant** (reordering nodes doesn't change results), and they share weights across nodes.

Q: GCN (Graph Convolutional Network)? Aggregates neighbors with a normalized average (by degree), then a linear layer + non-linearity: `H' = σ(D^-½ (A+I) D^-½ H W)`. Simple and strong on homophilous graphs. Transductive in its original form (needs the full graph).

Q: GraphSAGE? Samples a fixed number of neighbors per node and learns aggregator functions (mean, pooling, LSTM). **Inductive**: works for new nodes from their features and neighbors. Built for huge graphs (originally Pinterest-scale ideas → PinSage).

Q: GAT (Graph Attention Network)? Learns attention weights for each neighbor, so important neighbors matter more. Helps when neighbors vary in relevance.

Q: GIN (Graph Isomorphism Network)? Uses sum aggregation + MLP; as expressive as the Weisfeiler-Lehman graph isomorphism test. Good for graph-level tasks (molecules).

Q: Heterogeneous GNNs (R-GCN, HAN, HGT)? Separate weights per edge type / node type, so "bought" and "reviewed" edges are treated differently.

Q: Temporal GNNs? For graphs that change over time (transactions): TGN, TGAT. They include edge timestamps and memory per node. Useful for fraud where order of events matters.

Q: How do you do graph-level prediction? Run message passing, then a **readout/pooling** (sum/mean/max of node vectors, or hierarchical pooling) → classifier.

Q: Link prediction with GNN? Compute node embeddings, then score a pair with dot product or an MLP on [h_u, h_v]. Train with positive edges and sampled negative edges.

Q: Over-smoothing? With many layers, all node vectors become similar (every node has averaged over the whole graph) → performance drops. That's why most GNNs use only **2-3 layers**. Fixes: residual/skip connections, jumping knowledge (combine all layers), normalization (PairNorm), DropEdge.

Q: Over-squashing? Information from many distant nodes gets compressed into a fixed-size vector through bottleneck edges → long-range signals are lost. Fixes: graph rewiring, adding virtual nodes, graph transformers.

Q: Graph Transformers? Full attention across nodes with positional encodings from graph structure (Laplacian eigenvectors, random-walk encodings). Handle long-range dependencies but cost $O(N^2)$, so they are used on small/medium graphs (molecules).

Q: Expressiveness limit of message-passing GNNs? They can't distinguish some different graphs (bounded by the 1-WL test). Adding node IDs, positional encodings, or subgraph features helps.

6. Training and scaling GNNs

Q: Why are GNNs hard to scale? Neighborhood explosion: with 2 layers and 100 neighbors each, one node needs 10,000 nodes' features. Full graph won't fit in GPU memory for billions of edges.

Q: Solutions? | Technique | How it works | |---|---| | **Neighbor sampling** (GraphSAGE) | sample e.g. 15 then 10 neighbors per layer | | **Cluster-GCN / GraphSAINT** | train on subgraphs (clusters or sampled subgraphs) | | **Precomputation (SIGN, SGC)** | precompute aggregated features offline, then a simple MLP; very fast inference | | **Distributed training** | partition graph across machines (DistDGL, PyG distributed) | | **Historical embeddings / caching** | reuse stale neighbor embeddings |

Q: Libraries? PyTorch Geometric (PyG), DGL, GraphStorm; graph databases or Spark GraphFrames for features.

Q: Online inference for GNNs: what makes it hard? At request time you need the node's neighborhood and their features fast. Options: precompute embeddings in batch (daily/hourly) and store in a feature store/vector DB; for new nodes use an inductive model with sampled neighbors from a graph store; cap neighbor count for latency.

Q: Negative sampling for link prediction? Random negatives are too easy. Use **hard negatives** (nodes 2-3 hops away, popular items) to make the model learn fine distinctions. Evaluate with ranking metrics (MRR, Hits@K) not just AUC.

Q: Class imbalance in node classification (fraud = 0.1%)? Weighted loss, oversampling minority nodes' neighborhoods, focal loss, threshold tuning, evaluate with PR-AUC. See 07 §8.

7. Knowledge graphs

Q: What is a knowledge graph (KG)? Facts stored as triples **(head, relation, tail)**: (Paris, capital_of, France). Wikidata, product graphs, enterprise KGs.

Q: How do you build a KG from text? Named entity recognition → entity linking (map "Apple" to the right entity) → relation extraction (rules, fine-tuned models, or LLMs with structured output) → deduplication / entity resolution → store in graph DB → human review for high-value facts.

Q: KG embeddings? - **TransE:** relation is a translation: h + r ≈ t. Simple, fails on 1-to-many relations. - **DistMult / ComplEx:** multiplicative scores; ComplEx handles asymmetric relations. - **RotatE:** relation is a rotation in complex space. Used for **link prediction / KG completion** (predict missing facts).

Q: Entity resolution? Deciding that "IBM", "I.B.M." and "International Business Machines" are the same entity. Blocking (cheap candidate pairs) + similarity model + clustering. Critical for fraud graphs (same person, different names).

Q: Ontology / schema? Defines allowed entity types and relations. Keeps the KG consistent.

8. GraphRAG

Q: What is GraphRAG? RAG that uses a graph built from the documents: extract entities and relations with an LLM, build a KG, optionally detect communities and summarize each, then retrieve **entities + their neighborhoods + community summaries** in addition to (or instead of) text chunks. Plain RAG basics are in 02 §7.

Q: When does GraphRAG beat normal vector RAG? - Multi-hop questions ("Which suppliers of our top customer had compliance issues?"). See 04 #10. - Global/summary questions over a whole corpus ("What are the main themes in these 5,000 reports?"). - Domains with many entities and relations (legal, biomedical, supply chain).

Q: Downsides? Expensive indexing (many LLM calls to extract entities), extraction errors and duplicate entities, harder to keep fresh, more complex pipeline. For simple fact lookup, hybrid vector + keyword RAG is usually enough.

Q: Text-to-graph-query? LLM converts the question into Cypher/SPARQL and runs it on the KG. Precise for structured questions; risky if schema is complex → give the schema in the prompt, validate the query, limit results.

9. Use cases and system design

Use case	Graph	Task	Typical approach
Fraud rings	accounts, devices, cards, IPs, addresses	node classification, community detection	shared-attribute edges → connected components + graph features in XGBoost → GNN (often temporal, heterogeneous)
Recommendations	user-item bipartite (+ social)	link prediction	PinSage/LightGCN-style embeddings → ANN candidate generation → ranker. See 05 #4
Social networks	users, follows	friend suggestion, influence, spam	common neighbors, PPR, GNN embeddings
Drug discovery	molecules (atoms/bonds), protein interaction graphs	graph property prediction, link prediction	GIN / graph transformers / KG embeddings
Supply chain / risk	companies, suppliers	risk propagation	KG + PageRank-style propagation
Maps / ETA	road network	travel time	GNN over road segments
Search / Q&A	knowledge graph	entity answers	KG lookup + GraphRAG

Mini system design: real-time fraud ring detection

```
Events (signups, logins, payments) → Kafka → Graph builder (stream)
   → Graph store (accounts ↔ device / card / IP / address edges)
   → Streaming features: component size, # accounts per device, distance to known fraud (PPR)
   → Online scorer: XGBoost on graph + tabular features (fast)
   → Offline: daily GNN embeddings (heterogeneous, temporal) → feature store
   → Investigator UI shows the ring visually (explainability)
```

Trade-offs to mention: real-time graph updates vs. batch embeddings, hub nodes (a public Wi-Fi IP connects thousands of honest users, so cap or down-weight hubs), privacy of linked data.

---

10. What-if scenarios

GNN performs worse than a simple XGBoost on tabular features.

Causes (ranked): graph edges are noisy or not predictive; heterophily (neighbors differ from the node, so averaging hurts); leakage-free split made the task honest while the tabular model leaks; over-smoothing from too many layers; poor hyperparameters. **Isolate:** measure homophily of the labels; try 1-layer vs 2 vs 3; add graph features (degree, PageRank) to XGBoost and see if they help. **Fix:** use graph features in XGBoost (often the best production answer); heterophily-aware GNNs (separate self and neighbor embeddings, e.g., GraphSAGE concat, H2GCN); better edge definitions. **Prevent:** always keep a strong non-graph baseline in the eval.

Accuracy drops when you add more GNN layers.

Over-smoothing (or over-squashing). Keep 2-3 layers, add residual / jumping-knowledge connections, normalization, DropEdge.

Link prediction AUC is 0.99 offline but recommendations are bad online.

Causes: test edges leaked into the message-passing graph; random negatives are too easy; random split instead of time split; AUC doesn't reflect ranking quality. **Fix:** remove test edges from the training graph, time-based split, hard negatives, evaluate with Hits@K / MRR / recall@K on future interactions.

Training runs out of GPU memory on a 1-billion-edge graph.

Neighbor sampling (fanout 10-15 per layer), subgraph training (Cluster-GCN/GraphSAINT), precomputed aggregations (SIGN), store features on CPU/disk with GPU caching, distributed training with graph partitioning.

New users/items get no embedding (cold start).

Transductive model (node2vec, original GCN). Switch to an **inductive** model (GraphSAGE) that uses node features, or fall back to feature-only embeddings until the node has edges.

Fraud graph flags thousands of honest users as one "ring".

Cause: hub nodes (a shared public IP, a big office, a payment gateway, a default device ID) connect unrelated users. **Fix:** remove or down-weight high-degree hub nodes, use edge types with stronger meaning (same card > same IP), time windows on edges, human review for large components.

Online GNN scoring is too slow (needs 2-hop neighbors in real time).

Precompute embeddings in batch and serve from a feature store; cap neighbors; use 1 hop online + precomputed neighbor embeddings; simpler precomputed models (SGC/SIGN) for inference.

Knowledge graph has many duplicate entities.

Weak entity resolution. Add blocking + similarity model + human review queue, canonical IDs, merge rules; measure duplicate rate as a data-quality metric.

GraphRAG gives wrong answers about relationships.

Causes: LLM extraction errors (wrong relation, wrong entity linking), stale graph, missing edges. **Isolate:** trace the answer back to the triples used; check them against the source text. **Fix:** store source-text provenance for every triple, verify triples with a second pass, confidence scores, refresh pipeline on document change, combine with text-chunk retrieval as evidence.

The graph changes every second, but embeddings update daily.

Hybrid: real-time graph features (counts, component membership) computed in streaming + daily embeddings; temporal GNNs with per-node memory if the budget allows; monitor how stale embeddings

affect metrics.

21. Generative & Multimodal AI

Image, video and audio generation, vision-language models, and multimodal RAG. Text LLMs are in 09-nlp-and-llms.md; basic generative model types (VAE, GAN) are in 08 §6.

Contents

1. The generative model families
2. Diffusion models in depth
3. Controlling and customizing image generation
4. Video, audio and 3D generation
5. Vision-language models (VLMs)
6. Multimodal RAG and document understanding
7. Evaluation
8. Serving and cost
9. Safety, copyright and trust
10. System design: an image generation product
11. What-if scenarios

---

1. The generative model families

Family	Idea	Strengths	Weaknesses
Autoregressive	generate one token at a time	text, code, also images/audio as tokens	slow for long outputs
VAE	encode to latent distribution, decode samples	smooth latent space, fast	blurry outputs alone
GAN	generator vs discriminator	sharp, very fast sampling	unstable training, mode collapse
Diffusion	learn to remove noise step by step	best quality and diversity, stable training	slow sampling (many steps)
Flow matching / rectified flow	learn a straight path from noise to data	fewer steps, used in newer image/video models	newer, similar ideas to diffusion

Q: Why did diffusion replace GANs for most image generation? More stable training, better diversity (no mode collapse), scales well with data and text conditioning. GANs are still used where speed matters (super-resolution, real-time filters).

2. Diffusion models in depth

Q: Forward process? Take a real image and add a little Gaussian noise over T steps (e.g., 1000) until it is pure noise. This is fixed, not learned.

Q: Reverse process? A neural network learns to remove noise one step at a time. Starting from pure noise and running the reverse process produces a new image.

Q: What does the network actually predict? Usually the **noise ε** that was added (ε-prediction). Alternatives: predict the clean image (x_0) or the "velocity" v. Training loss is simply MSE between predicted and true noise:

```
1. pick a real image x0, a random step t, random noise ε
2. make noisy image x_t = sqrt(ᾱ_t)·x0 + sqrt(1-ᾱ_t)·ε
3. loss = || ε - model(x_t, t, text) ||²
```

Q: Noise schedule? How much noise is added at each step (linear, cosine). It affects quality, especially at high resolution.

Q: Which architecture? Originally a **U-Net** with attention layers. Newer models use **Diffusion Transformers (DiT)**, which treat latent patches as tokens and scale better.

Q: How is text used (conditioning)? A text encoder (CLIP text encoder, T5) turns the prompt into embeddings; the denoiser attends to them through **cross-attention**.

Q: Classifier-free guidance (CFG)? Train the model sometimes with the prompt and sometimes without (empty prompt). At sampling, push the prediction away from the unconditional one: `ε = ε_uncond + w · (ε_cond − ε_uncond)`. Higher guidance scale w → follows prompt more closely but less diverse and can look over-saturated. Typical w ≈ 5-8.

Q: Negative prompts? Replace the empty prompt in CFG with things to avoid ("blurry, extra fingers").

Q: Samplers (schedulers)? DDPM (many steps, slow), DDIM (deterministic, fewer steps), DPM-Solver / Euler (good quality in 20-30 steps). Same model, different speed/quality trade-off.

Q: Latent diffusion (Stable Diffusion idea)? Run diffusion in the small latent space of a VAE instead of pixel space (e.g., 64×64×4 instead of 512×512×3) → ~50× cheaper. VAE decoder turns the final latent into the image.

Q: How to make sampling faster? Fewer steps with better samplers, **distillation** (consistency models, progressive/adversarial distillation → 1-4 steps), smaller models, caching, fp16/quantization, TensorRT compilation.

Q: Why are hands and text inside images often wrong? Small, detailed structures with many valid configurations and less training data; VAE compression loses fine details; text rendering needs exact characters. Newer models with better text encoders and higher-resolution latents improve it.

3. Controlling and customizing image generation

Technique	What it does
Img2img	start from a noised version of an input image instead of pure noise → keeps layout
Inpainting / outpainting	regenerate a masked region / extend the canvas

Technique	What it does
ControlNet	extra network that conditions on structure: edges, depth map, pose skeleton, segmentation
IP-Adapter / image prompts	use a reference image as a style or subject prompt
LoRA for diffusion	small low-rank adapters trained on 10-100 images to learn a style or object; swap per user
DreamBooth	fine-tune to bind a rare token to a specific subject ("sks dog")
Textual inversion	learn a new word embedding for a concept, model frozen
Super-resolution / upscalers	generate small, then upscale

Q: Fine-tuning risks for diffusion? Overfitting (every image looks like the training images), language drift (forgets what "dog" means in general). Fix with prior-preservation loss, fewer steps, lower LR, LoRA instead of full fine-tune.

Q: Brand consistency for a company? Brand LoRA trained on approved assets + reference image adapters + fixed prompt templates + brand-safety classifier + human approval for public use.

4. Video, audio and 3D generation

Q: How do video models differ from image models? They generate many frames that must be **temporally consistent**. Approaches: 3D (space-time) attention or DiT over space-time patches, generating a low-frame-rate clip then interpolating, latent video VAEs. Main problems: flicker, identity drift, physics errors, huge compute.

Q: Text-to-speech (TTS) pipeline? Text → normalization (numbers, abbreviations) → acoustic model (predicts mel-spectrogram or audio tokens) → vocoder (turns it into waveform). Modern systems use neural codec tokens + autoregressive or diffusion models; voice cloning from a few seconds of audio.

Q: Speech-to-text (ASR)? Encoder-decoder Transformer on log-mel spectrograms (Whisper-style) or CTC models. Metrics: Word Error Rate (WER). Issues: accents, noise, code-switching, domain words.

Q: Music/audio generation? Audio tokenized by a neural codec, then a Transformer or diffusion model generates tokens conditioned on text or melody.

Q: 3D generation? Score distillation from 2D diffusion, NeRF / Gaussian splatting reconstruction from images, direct 3D diffusion on meshes/point clouds.

5. Vision-language models (VLMs)

Q: CLIP? Image encoder + text encoder trained with **contrastive loss** on hundreds of millions of image-caption pairs. Matching pairs are pulled together. Gives zero-shot classification ("a photo of a {label}") and image-text search. Weak at counting, spatial relations, reading text.

Q: How do LLaVA-style multimodal LLMs work?

```
Image → vision encoder (e.g., CLIP/SigLIP ViT) → patch embeddings
      → projector (MLP) maps them into the LLM's token space
      → LLM reads [image tokens] + [text tokens] → generates answer
```

Training: (1) align the projector on image-caption pairs, (2) instruction tune on visual Q&A / conversations.

Q: Other designs? Cross-attention from LLM to image features (Flamingo-style), Q-Former to compress image features (BLIP-2), natively multimodal models trained on interleaved image-text from the start.

Q: Why do VLMs struggle with documents, charts and small text? Image is resized to a low resolution (e.g., 336×336) → small text becomes unreadable. Fixes: high-resolution tiling ("any resolution": split into crops + a global view), OCR-free document models trained on documents, or combine with OCR output.

Q: Image token cost? A high-res image can be hundreds to thousands of tokens → latency and cost. Downscale when detail isn't needed; crop to the relevant region.

Q: Common VLM tasks in industry? Document extraction (invoices, IDs), visual Q&A for support (photo of a broken product), product tagging, content moderation, accessibility (alt text), chart/table understanding, UI understanding for computer-use agents.

Q: VLM hallucination? Describing objects that aren't there. Mitigate: ask for grounded answers (bounding boxes, quotes of visible text), verify with detectors/OCR, lower temperature, eval sets with "trick" images.

6. Multimodal RAG and document understanding

Q: Problem? Enterprise documents contain tables, charts, scanned pages, diagrams. Text-only parsing loses them.

Q: Approaches? | Approach | How | Trade-off | |---|---|---| | **Parse to text** | OCR + layout parser, tables → Markdown/HTML, charts → captions generated by a VLM | cheap retrieval, may lose detail | | **Caption everything** | VLM writes a description per image/chart, index the description | good recall, captioning cost at ingest | | **Multimodal embeddings** | embed page images directly (CLIP-like or document-page retrievers that embed screenshots) | keeps visual layout, bigger index | | **Hybrid** | retrieve by text, then send the original page image to a VLM for the answer | best quality, highest cost |

Normal RAG pipeline and its failures: 02 §7, 04 section A.

Q: Tables specifically? Keep tables whole as a chunk with headers repeated, store as Markdown/HTML, add a natural-language summary for retrieval, or load into a database and use text-to-SQL for numeric questions.

7. Evaluation

Metric	Measures	Caveat
FID (Fréchet Inception Distance)	how close the distribution of generated images is to real ones (lower = better)	needs many samples, uses ImageNet features, doesn't check the prompt

Metric	Measures	Caveat
IS (Inception Score)	quality and diversity	outdated, unreliable
CLIP score	similarity between prompt and image	can be gamed, weak on counting/relations
Prompt-alignment benchmarks	VQA-based checks: "is there a red cube left of a blue ball?"	closer to what users care about
Human preference	side-by-side ratings, Elo/win rate	gold standard, expensive
Aesthetic / preference models	learned predictors of human preference	biased to training raters
TTS: MOS (mean opinion score), WER of re-transcribed audio, speaker similarity		
VLM: accuracy on VQA/document benchmarks + task-specific eval set		

Q: How would you evaluate an image generation feature for a product? Build a prompt set from real user prompts (by category), generate with fixed seeds, measure prompt alignment (VQA-based), human side-by-side preference vs current model, safety violation rate, latency and cost per image, and online: download/regenerate rate.

8. Serving and cost

Q: Why is image generation expensive? Each image = N denoising steps × a large network; video multiplies by frames. A GPU might produce a few to tens of images per second depending on model, resolution and steps.

Cost/latency levers: - Fewer steps (better samplers, distilled 1-4 step models). - Lower default resolution + upscaler only when the user wants to download. - Batch requests on the GPU; queue with priority tiers (paid users first). - fp16/bf16, quantization, compiled engines. - Cache results for popular prompts / templates. - Smaller "draft" previews, full quality on demand. - Async jobs for video (minutes) with progress updates and notifications.

Q: Real-time vs async? Images: near real-time (a few seconds) with a queue. Video/long audio: always async jobs (submit → job ID → poll/webhook).

9. Safety, copyright and trust

Q: Main risks? Deepfakes of real people, sexual or violent content, minors, hate symbols, copyrighted characters/styles, trademarked logos, misinformation, voice cloning fraud.

Q: Defenses? - **Input**: prompt classifiers, block lists for real-person names in sensitive contexts, policy-trained LLM filter. - **Output**: image safety classifiers (nudity, violence, minors), face matching against protected persons where legally appropriate, logo detection. - **Training data**: filter unsafe and opt-out content, licensed datasets, record data provenance. - **Provenance**: invisible **watermarks** and signed **content credentials** (C2PA-style metadata) so generated media can be identified. Note: metadata can be

stripped; watermarks can be weakened by editing. Use both and never rely on just one. - **Voice**: consent verification for voice cloning, audio watermarking. - **Human review and appeals** for borderline cases; abuse monitoring per account.

Q: Copyright questions in interviews? Say: training data licensing and opt-outs, avoid generating near-copies (memorization checks: compare outputs to training set with perceptual hashes), block specific protected characters/brands, legal review. Don't claim legal certainty.

10. System design: an image generation product

> "Design a text-to-image feature for a design app with 5M users."

```
Client → API gateway (auth, quota per plan)
      → Prompt service: safety check (input), prompt rewrite/expansion (LLM), template/brand
params
      → Job queue (priority by plan) → GPU workers (autoscaled, batched, distilled model + LoRA
hot-swap)
      → Output safety classifiers → watermark + content credentials
      → Object storage + CDN → client (progress via websocket/SSE)
Metadata DB: jobs, prompts, seeds, model version (for reproducibility and abuse investigations)
Monitoring: queue time, GPU utilization, cost/image, safety block rate, regenerate rate
```

Deep dives: LoRA hot-swapping per brand (keep base model loaded, swap small adapters), GPU autoscaling on queue length, fairness of the queue, reproducibility (seed + model version + sampler), abuse detection.

---

11. What-if scenarios

Generated images ignore part of the prompt ("a red cube on a blue ball" → colors swapped).

Causes (ranked): text encoder weak at attribute binding (CLIP text encoder); guidance scale too low; prompt too long (truncated at the text encoder's token limit, often 77 tokens); concept rare in training data. **Isolate:** test shorter prompts; vary guidance; check token count; compare with a stronger text encoder model. **Fix now:** LLM prompt rewriting into clear, short descriptions; tune guidance; regional prompting / layout control (ControlNet, boxes). **Long-term:** model with a stronger text encoder (T5/LLM-based), fine-tune on compositional data. **Prevent:** compositional prompt eval set with VQA-based scoring.

Faces and hands look distorted.

Low resolution of the face region in latent space, too few steps, aggressive distillation, LoRA overfitting. **Fix:** face/hand detailer pass (detect → inpaint at higher resolution), more steps, better base model, negative prompts, pose ControlNet.

The multimodal model misreads numbers in charts or tables.

Causes: image downscaled (small text unreadable); model weak at chart reasoning; dense table. **Isolate:** crop the region and retry; compare with OCR text. **Fix:** high-res tiling/crops, OCR + VLM together (give extracted text as extra context), extract tables to structured data and compute answers with code/SQL, ask the model to quote values before reasoning. **Prevent:** document/chart eval set with numeric exact-match checks.

Image generation is too slow (12 s per image) and too expensive.

Trace: queue time vs GPU time. Fix: distilled few-step model, fewer steps, lower default resolution + optional upscale, batching, fp16 + compiled engine, autoscale on queue length, cache popular templates, previews first. Trade-off: quality. Measure it with human preference on a fixed prompt set.

Outputs are brand-unsafe or off-brand (wrong colors, competitor logos).

Brand LoRA + reference-image conditioning, color palette constraints, logo/brand classifiers on output, prompt templates locked for non-expert users, human approval before publishing. Track off-brand rate from reviewer rejections.

A user generated a deepfake of a real politician.

Mitigate: remove content, review account, preserve logs for investigation. **Causes:** prompt filter missed name variants/descriptions; image-to-image with an uploaded photo bypassed text checks. **Fix:** checks on uploaded images too (face matching for public figures in restricted contexts), better prompt classifiers, watermark + content credentials on all outputs, rate limits for new accounts. **Prevent:** red-team regularly with evasion prompts; abuse monitoring.

The model generates near-copies of a training image.

Memorization (often from duplicated training images). Deduplicate training data, check outputs against training set with perceptual hashing, lower guidance, block exact-match outputs.

Video output flickers between frames.

Temporal inconsistency. Use models with temporal attention, fewer independently generated frames (generate keyframes + interpolate), consistent seeds/latents, post-processing stabilization.

Fine-tuned brand LoRA makes every image look the same.

Overfitting: too many steps, too-high LR, too few/too-similar training images. Lower rank/steps, more varied images, lower LoRA strength at inference, prior-preservation images.

VLM answers confidently about objects that aren't in the photo.

Object hallucination. Ask for evidence (boxes or quotes), verify with an object detector/OCR, add "say not visible if unsure" instruction, evaluate on images designed to trick (absent-object questions).

22. AI Agents

Agents are LLM systems that **take actions**: call tools, browse, write and run code, update records. Interviews now ask how to design, evaluate and secure them. Basics are in 02 §9; the support-agent case study is 05 #2.

Contents

1. What an agent is
2. Tool calling
3. Reasoning and planning patterns
4. Memory
5. Multi-agent systems
6. Orchestration: workflows vs free-form agents
7. Tool protocols and integrations
8. Safety: human-in-the-loop, sandboxing, permissions
9. Evaluating agents
10. Observability and operations
11. System design: an agent platform
12. What-if scenarios

---

1. What an agent is

Q: Define an AI agent. An LLM in a loop: it reads the goal and current state, decides the next action (often a tool call), observes the result, and repeats until it finishes or hits a limit.

```
goal → [LLM decides] → tool call → result → [LLM decides] → ... → final answer
```

Q: Chatbot vs RAG vs agent? Chatbot: answers from its own knowledge. RAG: retrieves documents then answers (fixed pipeline). Agent: decides **itself** which steps and tools to use, can take actions with side effects.

Q: When should you NOT use an agent? When the steps are known and fixed. Use a deterministic workflow (cheaper, faster, testable). Agents add cost, latency and unpredictability. Good rule: "*Use the simplest thing that works: single LLM call → fixed workflow → agent.*"

Q: Main parts of an agent? Model, instructions (system prompt), tools, memory/state, control loop with limits, guardrails, and observability.

Q: Why do agents fail more than single LLM calls? Errors compound: if each step is 95% reliable, a 20-step task succeeds only ~36% of the time (0.95^{20}). So reduce steps, add verification, and allow recovery.

2. Tool calling

Q: How does tool (function) calling work? You give the model tool definitions (name, description, JSON schema of arguments). The model outputs a structured call; **your code** executes it and returns the result as a new message; the model continues.

Q: What makes a good tool? - Clear name and description that says *when* to use it and when not. - Small, typed arguments with enums and examples. - Returns concise, structured results (not 50 KB of raw HTML). - Helpful error messages the model can act on ("order_id must look like ORD-12345"). - Idempotent where possible; side effects clearly marked. - Few tools per agent (too many similar tools → wrong choices). Group or route.

Q: Parallel tool calls? Models can request several independent calls at once (e.g., fetch weather for 3 cities) → run them concurrently to cut latency.

Q: Who enforces permissions, the model or the tool? Always the **tool/backend**. The model can be tricked; authorization must live in code (user's identity and scopes are passed by the system, not chosen by the model).

Q: Structured output vs tool calls? Structured output = model returns data in a schema. Tool calls = model asks the system to do something. Same mechanism under the hood in many APIs.

3. Reasoning and planning patterns

Pattern	How it works	Good for
ReAct	interleave Thought → Action → Observation	general tool use
Plan-and-execute	first write a plan, then execute steps (maybe with a cheaper model), re-plan on failure	long multi-step tasks
Reflection / self-critique	model reviews its own output and fixes it	writing, code
Verifier / judge step	separate check that the result meets the goal (tests, schema, LLM judge)	anything with side effects
Tree search (ToT, MCTS-like)	explore several candidate paths, pick best	puzzles, hard reasoning; costly
Routing	classifier/LLM sends request to the right specialist flow	many task types
Reasoning models	model "thinks" longer before answering	complex planning; higher latency/cost

Q: How do coding agents verify their work? Run tests, linters, type checks; read the error; fix; repeat. **Executable feedback is the best verifier**, so prefer tasks/tools where results can be checked by code.

4. Memory

Q: Types of agent memory? | Type | What | Where stored | |---|---|---| | **Working / short-term** | current conversation, tool results, scratchpad | the context window | | **Episodic** | past sessions and what happened

| DB / vector store, retrieved when relevant | | **Semantic / long-term facts** | user preferences, facts learned | key-value or vector store, with update/delete | | **Procedural** | how to do tasks (instructions, learned playbooks) | prompts, skill files, fine-tuning |

Q: The context window fills up on long tasks. What do you do? Summarize/compact older steps, keep only the latest tool results in full, store large outputs externally and keep a reference (file path, ID), write progress notes / to-do list the agent can re-read, split work into sub-agents with fresh contexts that return short summaries.

Q: Risks of long-term memory? Storing wrong or outdated facts, storing sensitive data, memory poisoning (malicious content saved as "fact"), cross-user leakage. Need: user-visible and deletable memory, scoping per user/tenant, TTLs, validation before saving.

5. Multi-agent systems

Q: Common multi-agent patterns? - **Orchestrator-workers (supervisor):** a lead agent splits the task and delegates to sub-agents, then combines results. Good for research/search that can run in parallel. - **Pipeline / assembly line:** fixed sequence of specialist agents (extract → validate → write). - **Debate / critic:** one agent proposes, another critiques. - **Handoffs:** conversation is transferred to a specialist agent (billing agent, tech agent).

Q: Pros and cons? Pros: parallelism, separation of concerns, fresh context per sub-task, specialized prompts/tools. Cons: more cost (often several times the tokens), coordination errors, duplicated work, contradictions, harder debugging.

Q: When is multi-agent worth it? Broad tasks that split into **independent** parallel parts (researching 10 companies). Not worth it for tightly coupled tasks where every step depends on the last (most coding edits). A single agent with good tools usually does better.

Q: How do sub-agents share information? Clear task spec in, short structured result out; shared artifacts (files, DB records) instead of passing huge text; the orchestrator owns the final decision.

6. Orchestration: workflows vs free-form agents

Q: Workflow (state machine / graph) vs autonomous agent? | Workflow / graph | Free-form agent | |---|---| | Steps and transitions defined in code; LLM used inside steps | LLM decides next step every time | | Predictable, testable, cheaper | Flexible for open-ended tasks | | Hard to cover unexpected cases | Harder to control and evaluate |

Most production systems are **hybrids**: a state machine with a few agentic steps where flexibility is needed.

Q: What do orchestration frameworks give you? Graph/state-machine definitions, tool abstractions, retries, persistence/checkpoints (resume after crash), streaming, human-approval steps, tracing integrations. Many open-source and cloud options exist; in an interview, describe the capabilities rather than depending on one library.

Q: Durable execution? Long tasks (hours) must survive crashes and deploys: persist state after each step, make steps idempotent, resume from the last checkpoint. Same idea as workflow engines in backend

systems.

Q: Stopping conditions? Goal reached (verified), max steps, max cost/tokens, max time, repeated identical actions, explicit "cannot complete" → escalate to a human.

7. Tool protocols and integrations

Q: Why standard tool protocols? Without a standard, every app writes custom connectors for every tool. Protocols like the **Model Context Protocol (MCP)** define a common way for an AI app (client) to discover and call tools, read resources and use prompts exposed by servers (GitHub, databases, internal APIs). Build a connector once, use it from many agents.

Q: Security concerns with tool servers? A malicious or compromised server can return instructions inside results (prompt injection) or request excessive permissions. Use allow-listed servers, least-privilege credentials per server, review tool descriptions, and treat tool outputs as untrusted data.

Q: Agent-to-agent communication? Emerging protocols let agents from different systems delegate tasks to each other. Same principles: authentication, scoped permissions, audit logs, untrusted inputs.

Q: Computer-use / browser agents? The model sees screenshots (or the page's accessibility tree) and outputs clicks and keystrokes. Useful for apps without APIs, but slow, brittle, and risky. Prefer APIs where available; sandbox the browser; require confirmation for purchases or sends.

8. Safety: human-in-the-loop, sandboxing, permissions

Q: Core principle? *Assume the model will sometimes be wrong or manipulated. Design so that a wrong or tricked agent cannot cause serious harm.* Prompt injection basics: 04 #15.

Q: Classify actions by risk. | Risk | Examples | Control | |---|---|---| | Read-only | search, read file, get order | allow, log | | Reversible write | draft email, create ticket, edit file in a branch | allow with limits, easy undo | | Irreversible / external | send email, payment, delete data, deploy, post publicly | **human approval**, strict limits, idempotency keys |

Q: Sandboxing? Run code/browser in isolated containers or VMs: no production credentials, restricted network (allow-list), CPU/memory/time limits, read-only mounts where possible, throw away after use.

Q: Least privilege? Each agent/tool gets only the scopes it needs, short-lived tokens, acting **as the user** (not as an admin), per-tool rate limits and spend limits.

Q: The "lethal trifecta" for data exfiltration? An agent that (1) has access to private data, (2) reads untrusted content, and (3) can send data out (web requests, emails). If all three are present, injected instructions can leak data. Remove at least one leg, or require approval for outbound actions.

Q: Guardrails? Input classifiers, output checks (PII, policy), tool-argument validation (amount ≤ limit, recipient in allow-list), and monitoring of unusual action patterns.

9. Evaluating agents

Q: Why is agent evaluation hard? Many valid paths to the same result, non-deterministic, side effects, long tasks, the environment (APIs, websites) changes.

Q: What to measure? | Level | Metric | |---|---| | Outcome | task success rate (verified by checks on the final state: DB record correct, tests pass) | | Trajectory | right tools chosen, no unnecessary or unsafe calls, number of steps | | Efficiency | tokens, cost, wall-clock time per task | | Safety | rate of policy violations, unsafe actions attempted, injection success rate | | Reliability | **pass^k**: succeeds in all k repeated runs (users need consistency, not lucky runs) | | User | resolution rate, escalation rate, satisfaction, undo rate |

Q: How to build an agent eval set? Real tasks from logs (anonymized) + hand-written edge cases + adversarial cases; each with a **checkable end state**; run in a sandboxed environment with mocked or replayable APIs; run each task several times; track results per model/prompt version in CI.

Q: LLM-as-judge for trajectories? Useful for "was this reasonable?" but calibrate against human reviews and prefer programmatic checks for anything objective.

10. Observability and operations

- **Trace every step:** prompt, model version, tool name + arguments + result, latency, tokens, cost, errors, all linked by a trace ID per task.
- **Dashboards:** success rate, steps per task, cost per task, tool error rates, human-approval rate, top failure reasons.
- **Replay:** re-run a failed trace with a new prompt/model to test fixes.
- **Budgets and alerts:** per-user and per-task spend caps; alert on loops and cost spikes.
- **Versioning:** prompts, tool schemas and model versions are deployable artifacts with canary rollout.
- **Feedback loop:** failures → labeled → eval set → fix → regression test.

11. System design: an agent platform

> "Design a platform where teams can build and run AI agents that use internal tools."

```
Clients (chat, Slack, API, scheduled jobs)
   → Agent gateway (auth, tenant, quotas)
   → Orchestrator / runtime
        - loads agent definition (prompt, tools, model, limits) from Agent registry (versioned)
        - durable state store (checkpoint each step) + task queue (long-running tasks)
        - LLM gateway (routing, fallback, caching)  → see 05 #3
        - Tool layer: tool registry + protocol servers; per-tool authZ, rate limits, argument
validation
        - Sandbox pool (containers/VMs) for code and browser tools
        - Approval service (human-in-the-loop queue with notifications)
        - Memory service (per-user scoped, deletable)
   → Observability: traces, cost metering per team, audit log (immutable)
   → Eval service: run suites on every agent/prompt/model change; canary + rollback
```

Deep dives to offer: isolation between tenants, credential management (agents never see raw secrets; the tools hold them), durable execution for multi-hour tasks, cost control, approval UX (show exactly what will happen), and how you evaluate before rollout.

---

12. What-if scenarios

Already covered in 04: prompt injection (#15), agent stuck in a loop (#16), provider down / rate limited (#17).

The agent keeps choosing the wrong tool.

Causes (ranked): overlapping or vague tool descriptions; too many tools; tool names that don't match user language; missing examples; weak model for routing. **Isolate:** collect failing traces, list which tool was chosen vs expected; check if a human could pick correctly from the descriptions alone. **Fix now:** rewrite descriptions ("Use this when…, do NOT use for…"), merge or remove overlapping tools, add few-shot examples. **Long-term:** router step that narrows tools per intent; hierarchical tool groups; fine-tune on correct trajectories. **Prevent:** tool-selection accuracy in the eval suite.

The agent claims it did something it didn't ("I've updated your address"): hallucinated tool result.

Causes: tool call failed or was never made, but the model continued; error returned as text the model ignored; model trained to sound helpful. **Isolate:** compare final message with the actual tool-call log and system state. **Fix:** final answer generated from **verified state** (re-read the record), structured success/failure flags from tools, a verifier step before claiming success, UI shows actual action receipts. **Prevent:** eval checks the end state, not the message text; alert on "claimed action without matching tool call".

On long tasks, the agent forgets the goal or repeats finished work.

Context overflow / lost in the middle. **Fix:** persistent plan or to-do list the agent updates and re-reads, compaction of old steps, store outputs externally with references, sub-agents for sub-tasks, checkpoints. **Prevent:** long-task evals with progress checks.

Costs per task suddenly 5× higher.

Causes: loops/retries, large tool outputs pasted into context every step, multi-agent fan-out, switched to a bigger model, new users with very long tasks. **Isolate:** cost breakdown per step and per tool from traces. **Fix:** step and token caps, truncate/summarize tool outputs, caching, cheaper model for simple steps, limit parallel sub-agents. **Prevent:** cost-per-task SLO and alerts.

Two agents give contradictory results (research agent says X, analyst agent says not X).

Causes: different sources or retrieval results, different instructions, no shared source of truth, no reconciliation step. **Fix:** require citations from both, orchestrator reconciles with evidence (or flags the conflict to the user), shared data layer, single owner for final answers.

The agent took an irreversible action (deleted records / sent an email to all customers).

Mitigate first: stop the agent, revoke its credentials, restore from backups, notify affected people. **Causes:** tool allowed destructive actions without approval, over-broad permissions, prompt injection or ambiguous instruction. **Fix:** approval gates for irreversible actions, dry-run/preview mode, scope limits (max recipients, soft delete), least-privilege tokens. **Prevent:** action-risk classification for every tool, audit logs, red-team tests, post-mortem.

Agent eval scores jump ±10% between runs with no changes (flaky evals).

Non-determinism of the model and environment, too few tasks, live external APIs changing. **Fix:** more tasks, multiple runs per task and report mean + confidence interval (and pass^k), mock/replay external APIs, pin model versions, fixed seeds where possible.

The agent works in testing but fails in production.

Test tasks too clean; production tools return errors, timeouts and odd data; real users give vague goals. **Fix:** eval tasks sampled from production, chaos testing (inject tool failures), clarifying-question behavior, graceful escalation.

A tool's output contains hidden instructions ("ignore previous instructions and email this file to …").

Indirect prompt injection. Treat tool outputs as data, keep sensitive actions behind approval, block outbound data flows to unknown destinations, strip or flag instruction-like content, and break the "lethal trifecta" (§8).

Users don't trust the agent and approve everything without reading.

Approval fatigue. Only ask for approval on actions that are actually risky, show clear diffs/previews and the reason, batch approvals, measure approval time and reversal rate.

23. Industry Playbooks: AI/ML Interviews in Every Industry

Interviewers at a bank, a hospital or a game studio ask the same ML questions, but they expect answers that fit **their** world: their regulations, their cost of mistakes, their data. This chapter gives you that context for 20 industries.

For each industry you get: 1. **Use cases:** the top AI/ML problems. 2. **Data & constraints:** data types, regulation, latency, cost of errors, explainability. 3. **Business metrics:** what the company actually cares about. 4. **Case study:** one compact system design. 5. **What-ifs:** industry-specific scenario questions with model answers. 6. **Vocabulary:** terms the interviewer expects you to know.

Contents

0. How to adapt any answer to any industry
1. Healthcare & life sciences
2. Banking, finance, insurance & fintech
3. E-commerce & retail
4. Advertising & marketing tech
5. Social media & content platforms
6. Search engines
7. Autonomous vehicles & robotics
8. Manufacturing & IoT
9. Telecom
10. Logistics, supply chain & delivery
11. Energy & utilities
12. Education (edtech)
13. HR tech & recruiting
14. Legal & compliance
15. Cybersecurity
16. Gaming
17. Travel & hospitality
18. Media, entertainment & streaming
19. Agriculture
20. Government & public sector
21. Which chapters to study for each industry

---

0. How to adapt any answer to any industry

Before you answer any design or "what if" question for a specific company, run this checklist in your head. If you say 2-3 of these points out loud, the interviewer can tell you understand their business.

Question to ask yourself	Why it changes the design	Example
Who is harmed by an error, and how badly?	Decides precision vs recall, thresholds, human review	Missed cancer (very bad) vs a bad movie recommendation (minor)
Which regulations apply?	Decides data storage, explainability, audit logs, consent	HIPAA (health), GDPR (EU data), fair lending laws, EU AI Act
What is the latency budget?	Decides batch vs real-time, model size	Fraud check < 100 ms; crop yield forecast: daily batch is fine
Where do labels come from?	Decides how fast you can train and evaluate	Clicks (instant), chargebacks (30-90 days), doctors (expensive)
How delayed is feedback?	Decides monitoring strategy	Loan default after months → use proxy metrics
Does the decision need to be explained?	Decides model choice (simple vs deep), SHAP, reason codes	Loan denial must give reasons by law
How sensitive is the data?	Decides privacy tech: anonymization, on-prem, federated learning	Patient records, children's data, location
Is there an adversary?	Decides robustness, fast retraining, monitoring	Fraudsters, spammers, hackers, cheaters adapt to your model
What is the cost of being down?	Decides fallback strategy: fail-open vs fail-closed	Payments fail-open with rules; medical device fail-safe
Is there a fairness risk?	Decides slice evaluation, bias audits	Hiring, lending, policing, healthcare
Edge or cloud?	Decides model compression, offline operation	Cars, factory cameras, farms with weak internet

One sentence that works in every industry interview:

> "Before choosing a model, I'd like to understand the cost of each type of error here, the regulations we must follow, and how quickly we get feedback labels, because those three decide most of the design."

---

1. Healthcare & life sciences

Use cases: medical imaging diagnosis (X-ray, CT, MRI, pathology), clinical note summarization and coding (ICD codes), patient risk prediction (readmission, sepsis), drug discovery (molecule property prediction, protein structure), clinical trial matching, medical chatbots and triage, hospital operations (bed and staff forecasting), wearable signal analysis (ECG, glucose).

Data & constraints: | Aspect | Details | |---|---| | Data | images (DICOM), EHR tables, clinical text, genomics, time-series vitals | | Regulation | HIPAA (US), GDPR, FDA / CE approval for medical devices (Software as a Medical Device), audit trails | | Cost of errors | very high: missed diagnosis harms patients; false alarms cause alarm fatigue | | Explainability | doctors must understand and trust outputs; the AI supports, the doctor decides | | Labels | expensive (specialist time), noisy (doctors disagree), small datasets | | Privacy | de-identification, on-prem or private cloud, federated learning across hospitals |

Business metrics: sensitivity/specificity at clinically chosen operating point, time saved per clinician, reduction in readmissions, diagnostic turnaround time, regulatory approval, clinician adoption rate.

Case study: sepsis early warning in a hospital - Requirements: alert nurses 4-6 hours before sepsis, using vitals and lab results every 15 minutes; low false-alarm rate; explanation for each alert.

```
Bedside monitors + lab system + EHR (HL7/FHIR feeds)
        │
        ▼
Streaming ingestion (Kafka) → feature builder (rolling vitals, lab trends, missingness flags)
        │
        ▼
Risk model (gradient boosting / time-series model) → calibrated risk score + top contributing
factors (SHAP)
        │
        ▼
Alert policy (threshold + suppression: no repeated alerts within 4h) → nurse dashboard / pager
        │
        ▼
Clinician feedback (acted / dismissed) + outcomes → monitoring & retraining (with clinical
review board)
```

- Deep dives: **alarm fatigue** (precision matters a lot; tune threshold with clinicians), **missing data is informative** (a lab test being ordered is a signal), patient-level train/test split, temporal validation (train on 2022, test on 2023), validation at new hospitals before rollout (site shift), **silent mode** pilot before alerts go live.

What-ifs: - **Model trained at Hospital A performs poorly at Hospital B.** Causes: different devices, patient population, coding practices, lab units. Isolate: compare feature distributions and per-site metrics. Fix: local fine-tuning or recalibration, multi-site training, federated learning. Prevent: external validation for every new site before go-live. - **Doctors ignore the alerts.** Too many false alarms, no explanation, alerts at the wrong time or in the wrong tool. Fix: raise threshold, add reasons, integrate into existing workflow, co-design with clinicians; measure "alerts acted on". - **Imaging model detects the hospital's scanner, not the disease** (shortcut learning). Use saliency maps (Grad-CAM), test on other scanners, remove text markers, balance data across sites. - **A patient asks the medical chatbot about chest pain.** Safety first: detect emergencies and route to "call emergency services now"; never diagnose; ground answers in vetted medical sources; human escalation; log and review. - **Radiologists disagree on 20% of labels.** Use consensus labels (majority or adjudication), model the uncertainty, measure inter-rater agreement (Cohen's kappa) as the performance ceiling.

Vocabulary: EHR/EMR, FHIR, HL7, DICOM, HIPAA, PHI (protected health information), de-identification, SaMD, FDA 510(k), sensitivity/specificity, PPV/NPV, AUROC, calibration, alarm fatigue, clinical validation, prospective vs retrospective study, ICD codes.

---

2. Banking, finance, insurance & fintech

Use cases: fraud detection (cards, payments, account takeover), credit scoring and loan underwriting, anti-money laundering (AML), algorithmic trading and price forecasting, insurance claims triage and pricing, customer churn, KYC document verification, chatbots for banking, financial document extraction.

Data & constraints: | Aspect | Details | |---|---| | Data | transactions (high volume, tabular), credit bureau data, documents, market time series | | Regulation | fair lending laws (ECOA, FCRA in US), model risk management (SR 11-7), Basel, PCI-DSS (card data), GDPR, explainability for adverse actions | | Latency | fraud: < 100 ms at authorization; credit: seconds to minutes; trading: microseconds to milliseconds | | Cost of errors | missed fraud = money lost; false decline = angry customer, lost revenue | | Explainability | required: "reason codes" for loan denials; models are validated by a separate model risk team | | Adversary | yes: fraudsters adapt quickly |

Business metrics: fraud loss (basis points of volume), false positive rate / decline rate, approval rate at fixed default rate, expected loss, customer lifetime value, AML alerts reviewed per analyst and true-positive rate (SAR conversion).

Case study: credit scoring for instant loans - Requirements: decision in < 5 s, explain denials with top reasons, fair across protected groups, auditable.

```
Application → KYC/identity check → feature assembly (bureau data, income, bank transactions via
open banking)
           → Rules (hard policy: age, sanctions list) → Scoring model (GBDT, monotonic
constraints)
           → Calibrated probability of default → decision policy (approve / reject / manual
review, limit & price)
           → Reason codes (SHAP top features mapped to approved text) → audit log (inputs, model
version, decision)
Outcomes (repayment months later) → performance monitoring → annual model validation &
retraining
```

- Deep dives: **reject inference** (you never see outcomes for rejected applicants → biased training data), monotonic constraints (more income should never increase risk), fairness testing (disparate impact ratio), champion/challenger, model documentation for regulators.

What-ifs: - **Fraud model's false declines doubled after a holiday sale.** Distribution shift: more first-time buyers, bigger baskets, new merchants. Mitigate: temporary threshold adjust and step-up authentication (OTP) instead of hard declines. Long-term: seasonal features, faster retraining, segment-level thresholds. - **Regulator asks why a customer was denied.** Show reason codes, input data, model version, and the validation report; that is why you log everything and use explainable models or SHAP-based explanations. - **Model approves fewer women than men at equal risk.** Measure approval and default rates by group; check proxy features (zip code, shopping categories); remove or constrain proxies; fairness-aware thresholds with legal review; document. - **Fraud ring uses many new accounts that look normal one by one.** Add graph features (shared devices, IPs, addresses), graph ML or link analysis, velocity features across accounts. - **Trading model was profitable in backtest, loses money live.** Look-ahead bias, survivorship bias, overfitting to history, ignoring transaction costs and slippage, regime change. Fix: walk-forward validation, realistic cost modeling, paper trading before live.

Vocabulary: PD/LGD/EAD (probability of default, loss given default, exposure at default), KS statistic, Gini coefficient, reason codes, adverse action notice, model risk management, AML, KYC, SAR, chargeback, step-up authentication, open banking, backtesting, look-ahead bias, basis points.

---

3. E-commerce & retail

Use cases: product recommendations, search ranking, dynamic pricing, demand forecasting and inventory, customer segmentation, churn and lifetime value, product categorization and attribute extraction, review summarization, visual search, AI shopping assistants, return prediction, fake review detection.

Data & constraints: clickstream (huge, implicit feedback), product catalog (text + images), transactions, inventory. Latency: search and recs < 200 ms. Cost of errors: lost sales (medium). Seasonality is strong (holidays, sales events). Cold start for new products is constant.

Business metrics: conversion rate, GMV (gross merchandise value), revenue per visitor, average order value, add-to-cart rate, search exit rate / zero-result rate, forecast accuracy (MAPE, WAPE), stock-out rate, inventory holding cost, return rate.

Case study: demand forecasting for 50,000 products × 1,000 stores

```
Sales history + promotions + prices + holidays + weather + store info
        │
        ▼
Feature pipeline (lags, rolling means, promo flags, price elasticity, calendar)
        │
        ▼
Global model (LightGBM or deep forecasting model across all series) + hierarchy reconciliation
(store → region → total)
        │
        ▼
Probabilistic forecasts (P10 / P50 / P90) → replenishment optimizer (order quantity vs holding &
stock-out cost)
        │
        ▼
Store orders → actual sales → accuracy monitoring by category
```

- Deep dives: one global model vs per-series models, intermittent demand (many zeros), **censored demand** (sales = 0 because out of stock, not because no demand), new products (use similar-product features), forecast quantiles because the business decision depends on uncertainty.

What-ifs: - **Recommendation clicks went up but revenue went down.** Recs push cheap or discounted items; optimize a revenue-aware or multi-objective score; add guardrail metrics. - **Forecast badly missed during a promotion.** Promo not in features, or new promo type; cannibalization of other products. Add promo features, uplift models, planner overrides with feedback. - **Search returns zero results for 15% of queries.** Spelling errors, synonyms, other languages, out-of-catalog items. Fix: spell correction, synonym mining from query logs, semantic search, query relaxation; track zero-result rate. - **Dynamic pricing raised prices during an emergency and caused bad press.** Add business rules and price caps, human approval for large changes, fairness and reputation checks; not everything should be optimized. - **Fake reviews are rising.** Classifier on text + behavior (account age, review bursts, purchase verified), graph of reviewer-seller links, human moderation for top sellers.

Vocabulary: GMV, AOV, conversion funnel, CTR, add-to-cart, basket analysis, collaborative filtering, cold start, price elasticity, cannibalization, SKU, stock-out, safety stock, MAPE/WAPE, hierarchical forecasting, uplift modeling.

---

4. Advertising & marketing tech

Use cases: click-through rate (CTR) and conversion rate (CVR) prediction, real-time bidding, budget pacing, audience targeting and lookalike modeling, attribution, creative generation and testing, customer lifetime value, marketing mix modeling, uplift modeling for campaigns.

Data & constraints: billions of impressions per day, extremely sparse categorical features, delayed conversions (hours to days). Latency: bid decisions in ~10-100 ms. Privacy: cookie deprecation, consent (GDPR, CCPA), Apple ATT. Calibration matters because predicted probabilities directly set bid prices.

Business metrics: revenue, ROAS (return on ad spend), CPA (cost per acquisition), CPM/CPC, advertiser retention, auction efficiency, calibration error, incremental lift.

Case study: CTR prediction for an ad platform

```
Ad request (user, context, page) → candidate ads (targeting filters, budget available)
   → feature lookup (user embeddings, ad history, context) → CTR & CVR models (deep + cross /
wide & deep, embeddings for sparse IDs)
   → expected value = bid × pCTR × pCVR (+ quality score) → auction (second-price / first-price)
→ ad served
Impression / click / conversion logs → joined with delay handling → online/continuous training →
calibration check
```

- Deep dives: **calibration** (bids depend on exact probability), **delayed feedback** (a non-click now may convert later), feature hashing for huge ID spaces, continuous training because ads change daily, exploration for new ads.

What-ifs: - **Predicted CTR is accurate on average but advertisers overpay.** Calibration drift per segment; recalibrate (isotonic per segment), monitor predicted/actual ratio per campaign. - **Third-party cookies disappear.** Shift to first-party data, contextual targeting (page content embeddings), privacy-preserving APIs, aggregated measurement, modeled conversions. - **Campaign shows high conversions but finance says no real sales increase.** Attribution counts people who would have bought anyway; measure **incrementality** with holdout groups or geo experiments; uplift modeling. - **A new ad never gets shown (cold start).** Exploration budget (bandits), priors from similar ads/advertiser, content features of the creative. - **Click fraud from bots.** Bot detection (behavior patterns, IP/device reputation), filter invalid traffic before billing and training.

Vocabulary: CTR, CVR, CPM, CPC, CPA, ROAS, RTB, DSP/SSP, second-price vs first-price auction, bid shading, pacing, calibration, attribution (last-click, multi-touch), incrementality, uplift, lookalike audiences, MMM, invalid traffic.

---

5. Social media & content platforms

Use cases: feed ranking, friend/follow suggestions, content moderation, spam and fake account detection, notification ranking, trending topics, creator recommendations, ad integration, misinformation detection, AI-generated content labeling.

Data & constraints: user graph, posts (text, images, video), engagement events at massive scale. Latency: feed < 200 ms. Integrity and safety are critical; regulation (EU Digital Services Act), elections, child safety laws. Adversaries: spammers, bot networks, coordinated campaigns.

Business metrics: DAU/MAU, time spent, sessions per user, retention, meaningful interactions, creator retention, reports per 1k views, prevalence of violating content, appeal overturn rate.

Case study: fake account detection

```
Signup / activity events → real-time rules (rate limits, known bad IPs) → block obvious bots
                         → feature store (device fingerprint, signup velocity, behavior
sequences, graph features)
                         → models: account-level classifier + graph model (clusters of linked
accounts)
                         → actions by confidence: allow / challenge (CAPTCHA, phone verify) /
restrict / remove
                         → appeals + human review → labels → retraining (weekly or faster)
```

- Deep dives: adversarial adaptation (attackers probe your thresholds), graph-based cluster takedowns, false positives on real users (appeals), measuring **prevalence** with random sampling rather than only counting removals.

What-ifs: - **Engagement-optimized feed is spreading outrage content.** Proxy metric problem; add integrity signals, downrank borderline content, optimize for long-term satisfaction surveys and retention, not raw engagement. - **Moderation model removes posts in one language much more often.** Bias from training data; per-language evaluation, more native-speaker labelers, recalibrated thresholds, human review. - **Bot network evades the classifier within days.** Move to behavior and graph signals that are costly to fake, fast retraining, honeypots, rate limits; treat as ongoing adversarial game. - **Viral misinformation during an election.** Fast-path human review, reduce distribution while reviewing, fact-checker labels, provenance signals; trade-off: speed vs accuracy vs free expression.

Vocabulary: DAU/MAU, engagement, retention cohorts, social graph, feed ranking, integrity, prevalence, coordinated inauthentic behavior, content policy, appeals, DSA, trust & safety.

---

6. Search engines

Use cases: web/document retrieval, query understanding (spelling, intent, entities), learning to rank, query autocomplete, snippets and answer extraction, AI-generated answers with citations, spam/SEO-spam detection, freshness for news, personalization, image and voice search.

Data & constraints: web-scale index (billions of documents), query logs, click logs (biased by position). Latency: < 200-500 ms end to end. Freshness matters for news. Quality judged by human raters plus clicks.

Business metrics: nDCG on rated queries, click-through and long-click rate, abandonment (no click), query reformulation rate, time to success, ad revenue, freshness lag.

Case study: web search with AI answers

```
Crawler (politeness, priority by importance & freshness) → parse & dedupe → indexer (inverted
index + vector index), sharded
```

```
Query → spell/intent/entity understanding → retrieval (BM25 + dense) from all shards → merge
     → learning-to-rank (hundreds of features: relevance, quality/authority, freshness, click
signals)
     → top results + optional LLM answer generated only from top results with citations
     → logging (clicks, dwell time) → training data (with position-bias correction)
```

- Deep dives: sharding by document and fan-out to all shards, tiered index (hot fresh index vs main index), position bias in click data (randomization or click models), LLM answer must cite sources and avoid hallucination.

What-ifs: - **Spam sites start ranking high.** New SEO trick; add spam classifiers, link analysis (link farms), quality signals, human rater review, manual actions. - **Results for breaking news are stale.** Freshness tier with fast crawling of news sources, freshness feature in ranker, detect "trending query" intent. - **AI answer quotes a satire website as fact.** Source quality filters, cross-source agreement, don't show AI answer for low-confidence or sensitive queries. - **Click-trained ranker keeps promoting the first result.** Position bias; use inverse propensity weighting, randomized interleaving, human relevance labels.

Vocabulary: inverted index, BM25, dense retrieval, learning to rank (pointwise/pairwise/listwise), nDCG, position bias, click models, query understanding, crawl budget, PageRank, freshness, interleaving, SERP.

---

7. Autonomous vehicles & robotics

Use cases: perception (object detection, segmentation, lidar point clouds), sensor fusion, tracking and prediction of other agents, planning and control, mapping and localization, simulation, data mining for rare events, robot grasping and manipulation, warehouse robots.

Data & constraints: camera, lidar, radar, IMU, GPS at high frequency; petabytes of driving logs. Latency: hard real-time (tens of milliseconds) **on the vehicle** with limited compute and power. Safety-critical: functional safety standards (ISO 26262, SOTIF). Long tail of rare events matters most.

Business metrics: miles per disengagement, collisions per million miles, comfort metrics (hard braking), mission success rate, simulation pass rate, robot pick success rate, throughput per robot.

Case study: data engine for perception improvement

```
Fleet vehicles → on-board triggers (model uncertainty, disagreement, driver takeover) → upload
interesting clips
   → data lake → auto-labeling (big offline models) + human labeling for hard cases
   → training (multi-task perception) → evaluation on scenario suites + simulation (replay &
synthetic edge cases)
   → safety review → model compression (quantization, TensorRT) → over-the-air deployment in
shadow mode → full rollout
```

- Deep dives: **long tail** (rare events like a child chasing a ball), active data mining instead of random sampling, offline metrics by scenario category, simulation for things you can't test on roads, shadow mode comparison, redundancy (multiple sensors and fallback behavior).

What-ifs: - **Detection fails in heavy rain/fog.** Under-represented conditions; mine fleet data for weather, augmentation, sensor fusion (radar works in fog), conservative fallback behavior (slow down). - **New model has better mAP but more hard-braking events.** Offline metric doesn't capture temporal stability;

measure tracking consistency, flicker, false positives at short range; evaluate in closed-loop simulation. - **A new city has different road signs.** Domain shift; collect & label local data, map updates, staged rollout by geography. - **Robot arm works in the lab but fails in the warehouse.** Lighting, object variety, clutter (sim-to-real gap); domain randomization in simulation, real-world fine-tuning, failure logging and retry policies.

Vocabulary: perception, sensor fusion, lidar point cloud, BEV (bird's-eye view), tracking, motion prediction, planning, SLAM, localization, disengagement, ODD (operational design domain), long tail, shadow mode, sim-to-real, ISO 26262, functional safety.

---

8. Manufacturing & IoT

Use cases: predictive maintenance, visual defect inspection, anomaly detection on sensor streams, process optimization (yield), digital twins, energy optimization, quality prediction, robotics on assembly lines, supply forecasting.

Data & constraints: high-frequency sensor time series (vibration, temperature, pressure), images from line cameras, maintenance logs (often handwritten). Edge deployment (factory network, low latency, offline). **Very few failure labels**. Cost of errors: missed defect → recall or safety issue; false stop → stopped production line (expensive).

Business metrics: unplanned downtime, OEE (overall equipment effectiveness), defect escape rate, false reject rate, scrap rate, maintenance cost, mean time between failures (MTBF).

Case study: predictive maintenance for 2,000 machines

```
Sensors (vibration, temp, current) → edge gateway (filtering, FFT features, buffering if
offline)
   → stream to cloud (MQTT → Kafka) → time-series DB + feature store
   → models: anomaly detection (no labels needed) + remaining useful life (RUL) model where
failure history exists
   → alerts with confidence & expected time to failure → maintenance work order system
   → technician feedback (real fault? what was replaced?) → labels → retraining
```

- Deep dives: few failures → start with **unsupervised anomaly detection** and physics-based thresholds, label from maintenance logs (NLP on technician notes), per-machine baselines (each machine is different), edge vs cloud split, cost-based alert threshold.

What-ifs: - **Inspection model rejects too many good parts after a lighting change.** Input drift; monitor image statistics, retrain with new lighting, fixed lighting enclosures, normalization. - **Only 10 examples of a rare defect.** Anomaly detection trained on good parts only (autoencoder, PatchCore), synthetic defects, few-shot learning, human-in-the-loop. - **Sensor stops sending data and the model says "healthy".** Missing data treated as normal; detect missing/flatlined sensors explicitly and alert on data quality. - **Technicians don't trust alerts.** Show which sensor and pattern triggered the alert, track precision, start with high-confidence alerts only, include their feedback.

Vocabulary: OEE, predictive vs preventive maintenance, RUL, MTBF, SCADA, PLC, MQTT, edge gateway, digital twin, FFT, anomaly detection, yield, scrap rate, false reject, defect escape.

---

9. Telecom

Use cases: churn prediction, network anomaly and fault detection, capacity planning and traffic forecasting, network optimization (5G resource allocation), customer support chatbots, fraud (SIM swap, subscription fraud), next-best-offer, call center analytics.

Data & constraints: call detail records (CDRs), network telemetry at very high volume, customer data. Regulation on customer data and lawful use. Network issues affect millions quickly. Many legacy systems.

Business metrics: churn rate, ARPU (average revenue per user), network uptime, dropped call rate, mean time to detect/resolve incidents, NPS, retention campaign ROI.

Case study: churn prediction + retention

```
Usage (CDRs), billing, complaints, network quality per customer, contract end date
   → monthly features → churn model (GBDT) → **uplift model** (who will stay *because* of an
offer)
   → targeting: offer to customers with high uplift and high value → campaign → measured with
control group
```

- Deep dives: predicting churn is not enough; target people who are **persuadable** (uplift), not "sure to leave" or "sure to stay"; always keep a holdout to measure the campaign effect.

What-ifs: - **Retention campaign increased churn.** Contacting some customers reminds them to leave ("sleeping dogs"); uplift modeling fixes this. - **Network anomaly alerts flood engineers during an outage.** Alert correlation (root-cause grouping by topology), deduplication, severity ranking. - **SIM-swap fraud rising.** Real-time risk scoring on swap requests (recent password changes, unusual location), step-up verification, block high-risk transactions after a swap.

Vocabulary: CDR, ARPU, churn, uplift, NPS, OSS/BSS, RAN, 5G slicing, KPI degradation, SIM swap, next-best-action.

---

10. Logistics, supply chain & delivery

Use cases: ETA prediction, route optimization, dispatch and matching (drivers ↔ orders, riders ↔ drivers), surge/dynamic pricing, demand forecasting, warehouse slotting and picking, fraud, address correction, delivery time promises.

Data & constraints: GPS traces, maps/road graph, orders, traffic, weather. Real-time decisions (seconds). Two-sided marketplace effects. Combination of ML (predictions) and optimization (OR solvers).

Business metrics: on-time delivery rate, ETA error (MAE, % late beyond promise), cost per delivery, driver utilization, wait time, cancellation rate, marketplace balance.

Case study: ETA prediction for food delivery

```
Order placed → components: restaurant prep time model + driver pickup travel time + drop-off
travel time
   features: restaurant load, time of day, weather, live traffic, driver location, historical
```

```
speeds on road segments
   → ETA model (GBDT or deep model on route segments) → quantile predictions → promise shown to
user (conservative quantile)
   → actual times logged → monitoring per city/restaurant → retraining
```

- Deep dives: **asymmetric cost** (being late is worse than early → predict a higher quantile), break ETA into stages, live updates during delivery, feedback loop (ETA promise affects demand).

What-ifs: - **ETAs are accurate on average but very wrong at dinner rush.** Model lacks real-time load features; add live restaurant/driver supply features, segment evaluation by hour. - **Surge pricing removes demand but driver supply doesn't increase.** Pricing model ignores supply elasticity; measure both sides in experiments; use switchback experiments. - **A/B test on dispatch algorithm gives weird results.** Marketplace interference (treatment and control share drivers); use switchback (time-based) or geo-based randomization. - **Hurricane disrupts the supply chain.** Forecast models break; switch to scenario planning, human overrides, rapid re-forecasting with recent data only.

Vocabulary: ETA, quantile regression, VRP (vehicle routing problem), dispatch/matching, surge pricing, supply-demand balance, switchback experiments, last mile, OR (operations research), slotting.

---

11. Energy & utilities

Use cases: load forecasting, renewable generation forecasting (solar, wind), grid anomaly and outage prediction, predictive maintenance of assets (transformers, turbines), energy trading, smart meter analytics, energy theft detection, battery optimization.

Data & constraints: smart meter time series, weather data, SCADA, satellite images. Regulated industry, critical infrastructure (security!), forecast errors cost money in balancing markets.

Business metrics: forecast error (MAPE, pinball loss), outage minutes (SAIDI/SAIFI), imbalance costs, asset failure rate, renewable curtailment.

Case study: day-ahead solar generation forecast

```
Weather forecasts (irradiance, clouds, temperature) + satellite cloud images + historical plant
output
   → features per plant & hour → probabilistic model (GBDT quantiles / deep model) → P10/P50/P90
per hour
   → trading & grid balancing decisions → actual output → error monitoring
```

- Deep dives: weather forecast is the biggest error source (use ensembles), probabilistic outputs for risk decisions, new plants (transfer from similar plants).

What-ifs: - **Load forecast fails on a holiday / heat wave.** Missing calendar or extreme-weather data; holiday features, extreme-event training data, human override. - **Smart meter data has gaps.** Impute carefully, flag estimated values, don't train on imputed values as truth. - **Theft-detection model flags poor neighborhoods disproportionately.** Fairness review, check proxies, require human inspection before action.

Vocabulary: load forecasting, day-ahead market, pinball loss, SCADA, smart meter, SAIDI/SAIFI, curtailment, demand response, grid balancing.

---

12. Education (edtech)

Use cases: personalized learning paths, knowledge tracing (predict mastery), automated grading and feedback, AI tutors, dropout prediction, content recommendation, plagiarism and AI-writing detection, question generation.

Data & constraints: student interaction logs, answers, essays, video engagement. **Minors' data** (COPPA, FERPA, GDPR-K): strict privacy and consent. Fairness across students is critical. Pedagogical correctness matters more than engagement.

Business metrics: learning outcomes (test score gain), course completion, retention, time to mastery, teacher time saved, student satisfaction.

Case study: AI tutor for math

```
Student question / attempt → guardrails (age-appropriate, safety) → tutor LLM with pedagogy
prompt (hint, don't give answer)
   → math verification tool (symbolic solver checks steps) → response
Knowledge tracing model (student mastery per skill) → chooses next problem difficulty
Teacher dashboard (progress, struggling students, flagged conversations)
```

- Deep dives: **don't just give answers** (learning vs convenience), verify math with tools (LLMs make arithmetic mistakes), measure learning gain (pre/post tests, A/B), privacy for children.

What-ifs: - **Students use the tutor to get answers and test scores drop.** Change policy to hints and Socratic questions, limit full solutions, measure learning outcomes not usage. - **Automated essay grader gives lower scores to non-native writers.** Bias analysis by group, train on diverse essays, human review for borderline, grade content separately from grammar. - **AI-writing detector falsely accuses students.** Detectors are unreliable; never use as sole evidence; prefer process-based assessment.

Vocabulary: knowledge tracing (BKT, DKT), mastery learning, adaptive learning, item response theory (IRT), learning gain, COPPA, FERPA, formative vs summative assessment.

---

13. HR tech & recruiting

Use cases: resume parsing and matching, candidate ranking, job recommendations, interview scheduling, attrition prediction, skills extraction, salary benchmarking, internal mobility, HR chatbots.

Data & constraints: resumes and job descriptions (text), application outcomes, employee data. **High fairness and legal risk**: equal employment laws, NYC bias-audit law for automated hiring tools, EU AI Act classes hiring as high-risk. Historical hiring data contains past bias.

Business metrics: time to hire, quality of hire (performance, retention), recruiter productivity, candidate experience, diversity of pipeline, application completion rate.

Case study: resume-job matching

```
Job description → skill & requirement extraction (LLM / NER) ─┐
Resume → parsing (layout + NER) → skills, experience, education ─┴→ matching model (embeddings +
structured features)
   → ranked candidates with explanations ("matches 7 of 9 required skills")
   → recruiter decision (human always decides) → outcomes → bias audits by group (adverse impact
ratio)
```

- Deep dives: remove protected attributes **and proxies** (names, photos, certain schools, gaps), **don't train only on past hiring decisions** (they encode bias), explainable skill-based matching, regular bias audits.

What-ifs: - **Model ranks women lower for engineering roles.** Learned from biased history; audit, remove proxy features, train on skill-match labels instead of past hires, apply fairness constraints, document. - **Candidates stuff resumes with hidden keywords.** Parse visible text only, semantic matching, keyword-stuffing detection. - **Attrition model is used to deny promotions.** Misuse of model; define allowed uses, governance, human decision-making, transparency.

Vocabulary: ATS (applicant tracking system), adverse impact / four-fifths rule, disparate impact, bias audit, skills taxonomy, quality of hire, EEOC, high-risk AI system.

---

14. Legal & compliance

Use cases: contract review and clause extraction, legal research (RAG over case law), e-discovery (find relevant documents in litigation), compliance monitoring (communications surveillance), regulatory change tracking, document drafting assistants, due diligence.

Data & constraints: long documents, precise language, confidentiality (attorney-client privilege), citations must be **exactly** correct. Hallucinated case law has led to real court sanctions. Explainability and traceability are required.

Business metrics: hours saved per review, recall in e-discovery (missing a document is costly), precision of clause extraction, citation accuracy, lawyer adoption.

Case study: contract review assistant

```
Contract upload → OCR/layout parsing → clause segmentation → clause classification (indemnity,
termination, liability cap…)
   → extraction of key terms (dates, amounts, parties) with structured output → comparison
against company playbook
   → risk flags with exact quoted text & location → lawyer review UI (accept / edit) → feedback
improves models
```

- Deep dives: **always quote the source text** (no unsupported claims), long-context handling, on-prem or private deployment for confidentiality, high recall for risky clauses, versioned playbooks.

What-ifs: - **Legal research tool cites a case that doesn't exist.** Retrieval-only citations: the model may only cite documents returned by retrieval; verify every citation against the database before showing. - **E-discovery misses relevant documents.** Measure recall with statistical sampling (elusion test), technology-

assisted review with active learning, document the process for court. - **Different jurisdictions have different rules.** Metadata filter by jurisdiction and date, show which law applies.

Vocabulary: e-discovery, TAR (technology-assisted review), privilege, clause extraction, playbook, redlining, jurisdiction, precedent, elusion rate, chain of custody.

---

15. Cybersecurity

Use cases: intrusion and anomaly detection, malware classification, phishing detection, user behavior analytics (insider threats), alert triage and prioritization, vulnerability prioritization, security copilots (LLM summarizing incidents), bot detection.

Data & constraints: logs (network flows, endpoint, authentication), emails, binaries. Huge volume, very few true attacks (extreme imbalance). **Adversarial**: attackers actively evade models. Analyst time is the bottleneck. Model itself is an attack target (poisoning, evasion, prompt injection on security copilots).

Business metrics: mean time to detect (MTTD) and respond (MTTR), true-positive rate of alerts, alerts per analyst per day, missed incidents, phishing click rate.

Case study: alert triage system for a SOC

```
SIEM alerts (thousands/day) → enrichment (asset criticality, user risk, threat intel)
   → triage model (probability alert is real + severity) → grouping related alerts into
incidents (graph)
   → prioritized queue for analysts + LLM summary of incident (with evidence links)
   → analyst verdicts → labels → retraining; automated playbooks for low-risk well-known cases
```

- Deep dives: alert fatigue, labels from analyst verdicts (biased toward what was investigated), adversarial robustness, explainability for analysts, never let the LLM take destructive actions automatically.

What-ifs: - **Attackers bypass the malware classifier with small changes.** Adversarial evasion; ensemble of static + behavioral detection, adversarial training, frequent updates. - **Anomaly detector flags every new software deployment.** Learn normal change patterns, whitelist known deploy events, context from change management system. - **Phishing model performance drops when attackers use LLM-written emails.** Shift to sender/infrastructure signals (domain age, authentication SPF/DKIM), URL analysis, not just writing style. - **Security copilot is prompt-injected via a malicious log line.** Treat log content as untrusted data, read-only tools, human approval for actions.

Vocabulary: SIEM, SOC, EDR, IDS/IPS, threat intelligence, IOC, MITRE ATT&CK, MTTD/MTTR, alert fatigue, phishing, zero-day, adversarial evasion, data poisoning, UEBA.

---

16. Gaming

Use cases: player churn prediction, matchmaking, cheat detection, in-game economy and pricing (offers), content/level recommendation, toxicity detection in chat/voice, game AI (NPC behavior, RL agents for testing), procedural and generative content, LTV prediction.

Data & constraints: telemetry events at huge scale, real-time matchmaking (seconds), multiplayer fairness, young players (privacy, safety). Players dislike unfair monetization.

Business metrics: retention (D1, D7, D30), DAU, session length, ARPDAU, payer conversion, LTV, match quality, cheat reports, toxicity rate.

Case study: skill-based matchmaking

```
Players enter queue → skill rating (Elo / TrueSkill-style, updated after each match) + latency
region + party size
   → matcher balances: skill difference, wait time, ping → match formed
   → match outcome → rating update → monitor: win-rate balance (~50%), quit rate, wait time
```

- Deep dives: trade-off between wait time and match fairness (relax constraints over time), new players' uncertain rating, smurf detection (experienced players on new accounts).

What-ifs: - **New cheat spreads and reports spike.** Server-side behavior anomaly detection (impossible aim/speed), rapid rules + model update, delayed ban waves so cheaters can't learn what triggered detection. - **Personalized offers increase revenue but reviews say "pay to win".** Guardrail metrics on retention and sentiment; fairness rules on offers. - **Matchmaking wait times are too long at night.** Wider skill window over time, merge regions, bots for low-population periods.

Vocabulary: retention D1/D7/D30, ARPDAU, LTV, whales, Elo/TrueSkill/MMR, matchmaking, smurfing, ban wave, telemetry, live ops.

---

17. Travel & hospitality

Use cases: dynamic pricing / revenue management (hotel rooms, flights), demand forecasting, search ranking for listings, personalized recommendations, fraud (fake listings, stolen cards), review analysis, customer service chatbots, cancellation prediction, overbooking.

Data & constraints: booking data (strong seasonality, long lead times), search logs, listing content. External shocks (pandemics, events). Inventory is perishable (an empty room tonight is lost forever).

Business metrics: RevPAR (revenue per available room), occupancy, ADR (average daily rate), booking conversion, cancellation rate, customer satisfaction.

Case study: hotel dynamic pricing

```
Historical bookings + current bookings on the books (pace) + competitor prices + events +
seasonality
   → demand forecast per room type per future date (with uncertainty)
   → price elasticity model → optimizer chooses price to maximize expected revenue (with
business rules: min/max, brand)
   → prices published → booking response → model update
```

- Deep dives: booking pace curves, elasticity estimation (you only see bookings at prices you set → needs experiments), overbooking with cancellation probability.

What-ifs: - **A pandemic hits and the model keeps forecasting normal demand.** Regime change; use recent data only, scenario inputs, human overrides, faster retraining. - **Listing ranking favors big hotels; small hosts complain.** Two-sided marketplace fairness; add exposure/diversity constraints, new-listing boosts. - **Fake listings scam guests.** Verification, image reverse-search, behavior signals, payment held until check-in.

Vocabulary: RevPAR, ADR, occupancy, revenue management, booking pace, lead time, overbooking, price elasticity, OTA, GDS.

---

18. Media, entertainment & streaming

Use cases: content recommendation, artwork/thumbnail personalization, content tagging (genres, moods, scenes), subtitle generation and translation, dubbing, content demand forecasting (what to license/produce), video quality optimization (encoding), churn prediction, search.

Data & constraints: viewing history (implicit feedback), video/audio content, metadata. Large catalog with long tail. Personalization at scale with low latency. Licensing differs by country.

Business metrics: hours watched, retention, churn, take rate on recommendations, content discovery (share of long-tail viewed), cost per viewing hour of content, streaming quality (rebuffering rate).

Case study: home page personalization

```
Candidate rows (Continue Watching, Trending, Because you watched X, genre rows)
   → per-row candidate titles (collaborative filtering, content embeddings, trending)
   → ranking of titles within rows + ranking of rows on the page (page-level optimization)
   → artwork personalization (bandit selects thumbnail per user)
   → impressions/plays → offline evaluation + A/B tests on retention
```

- Deep dives: page-level optimization (diversity across rows), country licensing filters, evaluation on long-term retention rather than clicks, bandits for artwork.

What-ifs: - **New original show gets low views despite high quality.** Cold start; content-based embeddings, editorial boosts, exploration, trailers. - **Recommendations become a filter bubble.** Diversity and novelty in ranking, explicit exploration, measure catalog coverage. - **Auto-generated subtitles have errors in names and slang.** Custom vocabulary, human review for top content, quality metrics (WER) by language.

Vocabulary: implicit feedback, take rate, catalog coverage, filter bubble, artwork personalization, contextual bandits, ASR/WER, adaptive bitrate, rebuffering.

---

19. Agriculture

Use cases: crop yield prediction, plant disease detection from photos, precision spraying (weed detection on machines), irrigation optimization, satellite/drone imagery analysis, livestock monitoring, commodity price forecasting, soil analysis.

Data & constraints: satellite and drone images, weather, soil sensors, farm records. Seasonal (one data point per season per field → slow learning). Poor connectivity in fields (edge/offline). Huge geographic variation.

Business metrics: yield per hectare, input cost (water, fertilizer, pesticide) reduction, disease detection accuracy, farmer adoption, prediction error at harvest.

Case study: plant disease detection app for farmers

```
Farmer takes a photo (mobile, often offline) → on-device small CNN (quantized) → disease + confidence
   → if low confidence: queue photo to upload later → cloud model / agronomist review
   → recommendation in local language (treatment, when to consult expert) → feedback → retraining
```

- Deep dives: offline-first mobile model, local languages and low literacy (voice, icons), regional differences in crops and diseases, photos taken in bad conditions.

What-ifs: - **Model trained on lab leaf photos fails on field photos.** Domain shift (background, light, multiple leaves); collect field photos, augmentation, segmentation of the leaf first. - **Yield model trained in one country fails in another.** Different crops, climate, practices; local data, transfer learning, per-region calibration. - **Cloud cover blocks satellite images during key weeks.** Radar satellites (SAR) see through clouds, temporal interpolation, fusion with weather data.

Vocabulary: NDVI, remote sensing, multispectral imagery, SAR, precision agriculture, variable-rate application, phenology, yield gap.

---

20. Government & public sector

Use cases: document processing (forms, permits), citizen service chatbots, tax fraud detection, benefits eligibility, public health surveillance, traffic management, policy analysis with NLP, translation services, defense and intelligence applications.

Data & constraints: sensitive citizen data, strict procurement and security requirements (on-prem, sovereign cloud, certifications), **high accountability**: decisions affect people's rights, must be explainable and appealable, public scrutiny. EU AI Act classifies many public uses as high-risk. Legacy systems and paper records.

Business metrics: processing time per application, backlog size, error rate, citizen satisfaction, fraud recovered, cost savings, equity of outcomes across groups.

Case study: benefits application processing

```
Application (web or scanned paper) → document AI (OCR + field extraction) → validation against registries
   → eligibility rules engine (law encoded as rules, not ML) + ML risk score for fraud review (only to prioritize review)
   → human caseworker makes the decision → decision letter with reasons → appeal process
   → audits: fairness by group, error rates, published transparency report
```

- Deep dives: **ML prioritizes, humans decide**, rules for legal eligibility, transparency and appeal, bias audits, data minimization.

What-ifs: - **Fraud model wrongly flags thousands of families** (this happened in real scandals). Severity: stop automated actions, human review, compensate; root cause: biased features (nationality, income proxies), no human oversight; prevent: impact assessments, audits, appeal rights. - **Chatbot gives wrong legal information to citizens.** Ground answers only in official sources with links, disclaimers, easy handoff to a human, monitor and correct. - **Data cannot leave the country.** Sovereign cloud or on-prem deployment, open-weight models hosted locally.

Vocabulary: EU AI Act (high-risk systems), algorithmic impact assessment, transparency register, due process, appeal, data minimization, sovereign cloud, FedRAMP, accessibility (WCAG).

---

21. Which chapters to study for each industry

Industry	Most relevant chapters
Healthcare & life sciences	07, 10, 09, 11, 13
Banking / finance / insurance	07, 05 #5, 04 C, 11
E-commerce & retail	05 #4, #6, 04 F, 11
Advertising & marketing tech	07 §7, 11, 02
Social media & content	05 #4, #7, 04 F, 09
Search engines	05 #6, 02 §7, 09
Autonomous vehicles & robotics	10, 08, 13
Manufacturing & IoT	07 §6, 10, 13
Telecom	07, 11, 01
Logistics & delivery	07, 11, 01
Energy & utilities	07, 13
Education	09, 02, 11
HR tech & recruiting	09, 07 §9, 04 #29
Legal & compliance	02 §7, 04 A, 05 #1
Cybersecurity	07, 04 #15, 01 §7
Gaming	11, 07, 01
Travel & hospitality	05 #4, #6, 11
Media & streaming	05 #4, 04 F, 11
Agriculture	10, 13
Government & public sector	02 §11, 05 #8, 04 #29

Final tip: before any interview, read this company's industry section, then prepare **one sentence** for each of these: the cost of errors, the main regulation, the latency need, and where labels come from. Use them

in your first two minutes.

24. Classic System Design Cases (For Any Engineer)

These are the classic "design X" questions asked in backend, full-stack, platform and ML engineer interviews. Basics (load balancers, caching, sharding, queues) are in 01-fundamentals.md. URL shortener, chat app and rate limiter are in 05-case-studies.md.

Every case uses the same 7 steps:

> **1. Requirements → 2. Estimates → 3. API → 4. High-level design → 5. Data model → 6. Deep dives → 7. What-ifs & trade-offs**

Contents

1. News feed (social network)
2. Video streaming platform
3. Ride-hailing (geo-indexing)
4. Collaborative document editing
5. File sync and storage
6. Payment system
7. Notification system
8. Web crawler
9. Search autocomplete (typeahead)
10. Distributed key-value store
11. Distributed message queue
12. Ticket booking (seat locking)
13. Hotel / home rental booking
14. Metrics & monitoring system
15. Distributed job scheduler
16. Real-time leaderboard
17. Proximity search (nearby places)
18. Stock exchange / order matching
19. Email service
20. CDN design
21. Patterns that repeat across cases

---

1. News feed (social network)

> "Design the home timeline of a social network like Twitter/X."

1. Requirements - Functional: post (text + media), follow users, see a home feed of people you follow (newest or ranked), like/reply. - Non-functional: feed loads < 200 ms, highly available (stale by a few seconds is OK), read-heavy (~100 reads per write).

2. Estimates: 300M DAU, each opens feed 10×/day → 3B reads/day ≈ 35k QPS (peak ~100k). Posts: 300M × 0.5 = 150M/day ≈ 1.7k writes/s. Post metadata 1 KB → 150 GB/day.

3. API

```
POST /v1/posts                 { text, media_ids }
GET  /v1/feed?cursor=...       → list of posts (cursor pagination)
POST /v1/users/{id}/follow
```

4. High-level design

```
Client → LB → Post service → Posts DB (sharded by post_id) + Media → object storage/CDN
                  │
                  └→ Queue → Fan-out workers ──→ Feed cache (Redis: user_id → list of
post_ids)
Client → LB → Feed service → read Feed cache → hydrate posts (Post cache) → (ranking) → client
Follow service → Social graph DB (followers / following)
```

5. Data model: `posts(post_id, author_id, text, media, created_at)`, `follows(follower_id, followee_id)`, feed cache: `feed:{user_id} → [post_id, ...]` (keep last ~800).

6. Deep dives - **Fan-out on write (push):** when a user posts, write the post_id into every follower's feed cache. Fast reads, slow writes. - **Fan-out on read (pull):** at read time, fetch recent posts from everyone you follow and merge. Slow reads, cheap writes. - **Hybrid (the usual answer):** push for normal users; for **celebrities** (millions of followers) don't fan out. Merge their posts at read time instead. - Ranking: candidate posts → ranking model (see 05 #4). - IDs: time-sortable Snowflake IDs so feeds sort by ID.

7. What-ifs & trade-offs - *Celebrity posts → fan-out storm:* hybrid model above; rate-limit fan-out workers. - *Feed cache lost (Redis failure):* rebuild lazily from Posts DB via pull model; replicas for cache. - *User sees deleted post:* soft delete flag checked at hydration time. - Trade-off: freshness vs cost. A fan-out lag of a few seconds is acceptable.

---

2. Video streaming platform

> "Design a video platform like YouTube: upload and watch videos."

1. Requirements: upload, transcode, watch on any device/network, search, view counts. Smooth playback, global, very read-heavy.

2. Estimates: 500 hours of video uploaded per minute. 1B views/day ≈ 12k QPS for video starts. Storage huge: each video stored in many resolutions (~5× original). Bandwidth is the biggest cost → **you need a CDN**.

3. API

```
POST /v1/videos/upload-url     → pre-signed URL for direct upload to object storage
POST /v1/videos                { title, description, upload_id }
GET  /v1/videos/{id}           → metadata + manifest URL
```

4. High-level design

```
Upload:  Client ──(pre-signed URL)──► Object storage (raw)
                    │ event
                    ▼
              Queue → Transcoding workers (split into chunks, encode in parallel:
                      240p…4K, codecs H.264/VP9/AV1) → Object storage (segments + HLS/DASH
manifest)
                    → Thumbnails, content moderation, captions (ML)
                    → Metadata DB status = READY → CDN pre-warm for popular channels
Watch:   Client → API → metadata DB/cache → manifest URL
         Client player ──► CDN edge (segments) ──miss──► origin storage
```

5. Data model: `videos(video_id, owner_id, title, status, duration, created_at)`, `video_files(video_id, resolution, codec, path)`, view counts in a separate counter store.

6. Deep dives - **Adaptive bitrate streaming (HLS/DASH):** video cut into 2-10 s segments at multiple bitrates; player switches quality based on bandwidth. - **Parallel transcoding:** split video into chunks → DAG of tasks → merge. Resumable uploads (chunked upload). - **View counts:** don't write to DB per view; count in stream (Kafka) → aggregate → batch update. Approximate is OK. - **Cost:** popular videos on CDN; long-tail videos served from fewer locations / cheaper storage tier.

7. What-ifs: *Transcoding backlog:* autoscale workers, prioritize by channel popularity. *Viral video overloads origin:* CDN request coalescing, pre-warm. *Upload fails at 90%:* resumable chunked uploads. *Copyrighted content:* fingerprinting at upload.

---

3. Ride-hailing (geo-indexing)

> "Design a ride-hailing app like Uber."

1. Requirements: rider requests ride, match with nearby driver, live driver location, trip tracking, pricing, payment. Matching < few seconds, location updates every ~4 s.

2. Estimates: 5M active drivers sending location every 4 s → **~1.25M location writes/s** (this is the hard part). Ride requests: ~1k/s.

3. API

```
POST /v1/drivers/location      { lat, lng }   (or via WebSocket)
POST /v1/rides                 { pickup, dropoff } → ride_id
GET  /v1/rides/{id}            → status, driver location
```

4. High-level design

```
Driver app ──WebSocket──► Location service ──► Geo index (in-memory, Redis GEO / custom grid)
                                            └──► Kafka (location stream → analytics, ETA models, trip
history)
Rider app → Ride service → Matching service: query geo index for nearby available drivers
                         → rank by ETA → send offer to driver (timeout → next driver)
                         → Trip service (state machine) → Payment service
Pricing service (surge = demand/supply per area)   ETA service (routing graph + ML)
```

5. Data model: `drivers(driver_id, status, vehicle)`, live location only in memory (`geo cell → set of driver_ids`), `trips(trip_id, rider_id, driver_id, state, route, fare)`.

6. Deep dives - **Geo-indexing:** split the map into cells: **geohash** (string prefix = area), **quadtree** (split dense areas more), or **H3 hexagons** (equal-size neighbours). Query: my cell + neighbour cells. - Location data is **ephemeral**. Keep it in memory, sharded by region/city; don't write every update to a DB. - **Matching consistency:** a driver must not get two rides → lock driver (atomic status change `AVAILABLE → OFFERED` with compare-and-set). - Trip as a **state machine**: requested → matched → arriving → in_progress → completed → paid.

7. What-ifs: *Driver app loses network:* mark stale after N missed heartbeats. *Geo index node crashes:* rebuild in seconds from next location updates. *Hotspot (concert ends):* smaller cells in dense areas, surge pricing. *Two riders matched to same driver:* atomic CAS on driver status.

---

4. Collaborative document editing

> "Design a real-time collaborative editor like Google Docs."

1. Requirements: many users edit the same doc at once, see each other's changes and cursors in < 100 ms, offline edits, version history, permissions.

2. Estimates: most docs have 1-5 concurrent editors; few have 100+. Edit operations small (few bytes) but frequent.

3. API: `GET /docs/{id}` (snapshot + version), WebSocket `/docs/{id}/session` for sending/receiving operations.

4. High-level design

```
Clients ──WebSocket──► Collaboration servers (one "owner" server per document, chosen by
consistent hashing)
                         │  apply OT/CRDT, assign order, broadcast to other clients
                         ├──► Operation log (append-only, per doc)
                         └──► Periodic snapshots → Document store
Presence (cursors) via the same socket, not persisted.
```

5. Data model: `documents(doc_id, owner, acl)`, `operations(doc_id, version, op, user_id, ts)`, `snapshots(doc_id, version, content)`.

6. Deep dives - **The core problem:** two users edit the same place at the same time. Need all copies to end up identical. - **OT (Operational Transformation):** a central server orders operations and *transforms* each incoming op against concurrent ones (e.g., shift an insert position). Used by Google Docs. Needs a central server. - **CRDT (Conflict-free Replicated Data Types):** each character has a unique ID and position; merges are automatic and order-independent. Works peer-to-peer and offline. More memory. Used by Figma-style/local-first apps (Yjs, Automerge). - Version history = snapshots + op log replay.

7. What-ifs: *User offline for an hour:* queue ops locally, rebase/merge on reconnect (CRDT easier). *Owner server dies:* another server takes the doc, rebuilds from snapshot + log. *Huge doc slow to load:* snapshot + lazy loading.

---

5. File sync and storage

> "Design a file sync service like Dropbox / Google Drive."

1. Requirements: upload/download, sync across devices, share, version history, works with large files and bad networks.

2. Estimates: 50M users × 10 GB = 500 PB. Most files rarely change.

3. API: `POST /files/chunks/check {hashes}` → which chunks are missing; `PUT /chunks/{hash}`; `POST /files/commit {path, chunk_list, base_version}`; long-poll/WebSocket `/changes?cursor=`.

4. High-level design

```
Client (watcher + chunker) ──► Metadata service ──► Metadata DB (files, versions, chunk lists), strongly consistent
                          ──► Block service ──► Object storage (chunks addressed by hash)
Notification service ──► other devices pull changes
```

5. Data model: `files(file_id, owner, path, latest_version)`, `versions(file_id, version, chunk_hashes[], modified_by)`, `chunks(hash, size, location, ref_count)`.

6. Deep dives - **Chunking (4 MB) + content hashing:** only upload changed chunks (delta sync), resumable, **deduplication** (same chunk stored once across users). - Metadata needs **strong consistency** (SQL); blocks need durability (object storage, replicated). - **Conflicts:** two devices edit offline → commit with `base_version`; if stale, keep both ("conflicted copy").

7. What-ifs: *Huge file on mobile network:* chunked, resumable, pause on metered networks. *Ransomware encrypts files and syncs:* version history + bulk-restore. *Deleting a deduplicated chunk:* reference counting + delayed garbage collection.

---

6. Payment system

> "Design a payment system for an e-commerce site."

1. Requirements: charge customers via card/wallet (external payment providers), refunds, pay out sellers, never lose or double-charge money, audit trail.

2. Estimates: 1M orders/day ≈ 12 TPS. Throughput is small. **Correctness is what matters.**

3. API

```
POST /v1/payments   { order_id, amount, currency, method }   Header: Idempotency-Key
GET  /v1/payments/{id}
POST /v1/refunds    { payment_id, amount }                    Header: Idempotency-Key
Webhook from provider: /v1/provider-events
```

4. High-level design

```
Checkout → Payment service ──► Payments DB (state machine)  ──► Payment provider (card network)
                │                          ▲                              │
                │                          └──── webhook / poll status ◄──┘
                └──► Ledger service (double-entry, append-only)
                └──► Outbox → Kafka → order service, notifications, payouts
Nightly reconciliation job: our ledger vs provider settlement files → mismatches → alerts /
manual review
```

5. Data model: `payments(payment_id, order_id, idempotency_key UNIQUE, amount, currency, status, provider_ref)`, `ledger_entries(entry_id, txn_id, account_id, debit, credit, created_at)`. Every transaction has entries whose debits = credits.

6. Deep dives - **Idempotency:** client sends a unique key; server stores it with the result; a retry returns the stored result instead of charging again. - **State machine:** created → pending → succeeded / failed → refunded. Only valid transitions allowed. - **Double-entry ledger:** money moves between accounts (customer → platform → seller); append-only, never update. Easy auditing. - **Unknown outcome** (timeout from provider): mark *pending*, query provider status / wait for the webhook. Never blindly retry with a new key. - **Exactly-once effects** via idempotency + **transactional outbox** (write event in the same DB transaction). - Money as integers (cents) + currency, never floats. PCI: don't store card numbers. Use provider tokens.

7. What-ifs: *Provider timed out, did the charge happen?* → pending + reconcile. *Webhook arrives twice:* dedupe by event ID. *Ledger and provider disagree:* reconciliation flags it, human review. *Provider down:* failover to second provider for new payments.

---

7. Notification system

> "Design a system that sends push, SMS and email notifications."

1. Requirements: many services send notifications; channels push/SMS/email/in-app; user preferences & opt-out; templates; scheduling; rate limits; no spam/duplicates.

2. Estimates: 10M push + 1M SMS + 5M email per day; spikes during campaigns.

3. API: `POST /v1/notifications { user_id, template_id, params, channels, priority, dedupe_key }`.

4. High-level design

```
Services → Notification API → validate, check preferences & rate limits, render template
        → Priority queues per channel (Kafka/SQS: high = OTP, low = marketing)
        → Channel workers → providers (APNs/FCM, SMS gateway, email provider)
        → Delivery status / retries / DLQ → status DB → analytics
```

6. Deep dives: separate queues so marketing floods don't delay OTP codes; retries with backoff; dedupe by `dedupe_key`; user quiet hours & time zones; provider failover; tracking (delivered, opened).

7. What-ifs: *Campaign of 50M emails slows OTPs:* priority queues. *Provider down:* failover provider. *User receives same notification 3×:* idempotent workers + dedupe key. *Device tokens expired:* remove on provider feedback.

---

8. Web crawler

> "Design a crawler that downloads 1 billion pages per month."

1. Requirements: crawl the web, respect robots.txt and politeness, avoid duplicates, refresh pages, store content for indexing.

2. Estimates: 1B pages / month ≈ 400 pages/s; 500 KB/page → 500 TB/month.

4. High-level design

```
Seed URLs → URL frontier (priority + per-host queues for politeness)
          → Fetchers (DNS cache, robots.txt cache) → content
          → Dedupe (content hash / SimHash for near-duplicates) → Object storage
          → Parser → extract links → URL filter (seen? Bloom filter) → back to frontier
```

6. Deep dives: politeness: one queue per host, delay between requests; **URL seen check** with Bloom filter + DB; **priority** by PageRank/freshness; **recrawl** frequency by change rate; distributed by hashing host → crawler node.

7. What-ifs: *Crawler trap (infinite calendar URLs):* max depth, URL pattern limits. *Site blocks us:* slow down, respect robots. *Duplicate content on many URLs:* canonical URLs + SimHash.

---

9. Search autocomplete (typeahead)

> "Show top 5 suggestions as the user types."

1. Requirements: suggestions in < 100 ms per keystroke, ranked by popularity (+ personalization), updated daily or near-real-time, filter offensive terms.

2. Estimates: 10M DAU × 20 searches × 6 keystrokes = 1.2B requests/day ≈ 14k QPS.

4. High-level design

```
Offline: search logs → aggregate counts (Spark, daily) → build Trie with top-k precomputed at each node → snapshot
Online:  Client (debounce 100 ms, browser cache) → CDN/edge cache → Autocomplete service (trie in memory, sharded by prefix)
```

6. Deep dives: store **top-k at each trie node** (no traversal at query time); shard by first letters (watch for skew); real-time trending via stream counts merged with daily trie; personalization re-ranks a few results.

7. What-ifs: *Breaking news term missing:* streaming trending layer. *Offensive suggestion:* blocklist filter before serving. *Hot prefix "a" overloads shard:* cache common prefixes at edge.

---

10. Distributed key-value store

> "Design a key-value store like DynamoDB/Cassandra."

1. Requirements: `put(key, value)`, `get(key)`; huge scale; highly available; tunable consistency.

4. Design (key ideas)

```
Client → any node (coordinator) → consistent hashing ring → N replicas (e.g., N=3)
Write path: commit log (WAL) → memtable (memory) → flush to SSTables on disk → compaction
```

- **Partitioning:** consistent hashing with **virtual nodes** (even load, easy scaling).
- **Replication:** N copies on next nodes of the ring, across zones.
- **Quorum:** write to W, read from R; **W + R > N** → strong-ish consistency. Lower W/R → faster, more available.
- **Conflicts:** last-write-wins (timestamps) or vector clocks (detect concurrent writes).
- **Failure handling: hinted handoff** (another node holds writes for a dead node), **read repair**, **anti-entropy with Merkle trees**, gossip for membership.

- Storage: an **LSM tree** for fast writes; Bloom filters to skip SSTables on reads.

7. What-ifs: *Node dies:* hinted handoff, then replace and stream data. *Network partition:* choose AP (accept writes, resolve later) or CP (reject) per CAP. *Hot key:* add caching / key splitting.

---

11. Distributed message queue

> "Design a message queue / log like Kafka."

Key ideas

```
Producers → Broker cluster: topic split into partitions (append-only log on disk)
            each partition: 1 leader + followers (replication, ISR = in-sync replicas)
Consumers in a consumer group: each partition read by one consumer in the group; consumer tracks
offset
Controller (Raft/ZooKeeper) handles leader election and metadata
```

- **Ordering** guaranteed only within a partition → choose partition key (e.g., user_id) for per-entity order.
- **Durability:** `acks=all` waits for all in-sync replicas.
- **Retention** by time/size; consumers can **replay** from an offset.
- **Throughput:** sequential disk writes, batching, compression, zero-copy.
- **Delivery semantics:** at-least-once by default (commit offset after processing) → idempotent consumers; exactly-once via idempotent producers + transactions.

What-ifs: *Consumer lag grows:* add partitions + consumers, speed up processing, check for slow/poison messages (DLQ). *Broker dies:* follower in ISR becomes leader. *Hot partition:* better key, add salt.

---

12. Ticket booking (seat locking)

> "Design a concert/movie ticket booking system."

1. Requirements: browse events, pick seats, hold seats for ~10 min while paying, no double booking, survive huge spikes (big concert sale).

2. Estimates: normal: low QPS. Big sale: 1M users in the first minutes for 50k seats.

4. High-level design

```
Users → CDN (static event pages) → Virtual waiting room (queue, admit N users/min)
      → Booking service → Seat hold (Redis: SET seat_id user NX EX 600) → Payment → confirm in
DB (transaction)
Seats DB: seats(event_id, seat_id, status, booking_id, version)
```

6. Deep dives - **No double booking:** hold with atomic `SET NX` + expiry, and confirm in DB with **optimistic locking** (`UPDATE seats SET status='BOOKED' WHERE seat_id=? AND status='HELD' AND held_by=?`) or a DB unique constraint on (event_id, seat_id). - Holds expire automatically if payment not completed. - **Waiting room** protects the system and is fair (FIFO). - Bots: CAPTCHA, purchase limits per account.

7. What-ifs: *Payment succeeds after hold expired:* refund or re-check seat; extend hold during payment. *Redis down:* fall back to DB row locks (slower). *Spike 100× expected:* waiting room, static pages from CDN, autoscale.

---

13. Hotel / home rental booking

> "Design a booking system for hotels or home rentals (like Airbnb)."

- **Requirements:** search listings by location/dates/price, view availability, book without double booking, reviews, host calendar.
- **Design:** Search service (Elasticsearch with geo + filters; availability as a filter from a calendar index) → Listing service → Booking service with **availability table** `(listing_id, date, status)` → reserve dates in a DB transaction with row locks or unique constraint per (listing_id, date) → payment → confirmation. Read-heavy search vs strongly consistent booking → separate stores.
- **Trade-offs:** search index can be slightly stale (a listing might show available but fail at booking → re-check at booking, tell user).
- **What-ifs:** *Two guests book same dates:* constraint wins, second gets "no longer available". *Host changes price during checkout:* lock price quote with expiry. *Overbooking in hotels (intentional):* business rule with compensation flow.

---

14. Metrics & monitoring system

> "Design a system like Prometheus/Datadog that collects metrics from 100k servers."

- **Requirements:** collect metrics (CPU, latency, custom), store, query dashboards, alerting.
- **Estimates:** 100k hosts × 1,000 metrics every 10 s = **10M data points/s**.
- **Design:**

```
Agents on hosts → collectors (push) or scrapers (pull) → Kafka → ingesters → Time-series DB
  (compression: delta-of-delta timestamps, XOR floats; partition by metric + time)
Query service → dashboards;  Alert evaluator (rules every minute) → notification system
Downsampling: raw 10 s for 7 days → 1 min for 30 days → 1 h for 1 year
```

- **Deep dives: cardinality explosion** (labels like user_id create millions of series, so limit them), retention & downsampling, alert deduplication and routing.

- **What-ifs:** *Monitoring system down during an incident:* run it separately from what it monitors, multi-region. *Alert storm:* group alerts, dependency-aware alerting.

---

15. Distributed job scheduler

> "Design a system to run millions of scheduled/cron jobs."

- **Requirements:** schedule one-time and recurring jobs, run on time (±seconds), retries, no duplicate runs, visibility of status.
- **Design:**

```
API → Jobs DB (job_id, schedule, next_run_at, status)
Scheduler (leader-elected or partitioned by job_id hash): poll jobs where next_run_at <= now →
push to queue
   → Workers execute → report status → update next_run_at
```

- **Deep dives:** avoid double runs with **leases** (`UPDATE ... SET owner=?, lease_until=? WHERE status='READY'`), idempotent jobs, time-bucketed index on `next_run_at`, priorities, retries with backoff, DLQ, timezone/DST handling.
- **What-ifs:** *Scheduler node dies:* lease expires, another picks the job. *Thundering herd at midnight:* jitter, spread start times. *Job runs longer than its interval:* skip or queue policy.

---

16. Real-time leaderboard

> "Top 100 players for a game with 50M players, updated in real time."

- **Design:** Redis **sorted set** (`ZADD leaderboard score user`, `ZREVRANGE 0 99`, `ZREVRANK user`) gives O(log n) updates. Persist scores in DB as source of truth.
- **Scale:** shard by score range or by region; global top-k = merge top-k of each shard. Daily/weekly boards = separate keys with TTL.
- **What-ifs:** *Redis lost:* rebuild from DB. *Cheating:* validate scores server-side. *Exact rank for millions:* approximate rank for low-ranked players (percentile buckets).

---

17. Proximity search (nearby places)

> "Find restaurants near me (like Yelp)."

- **Requirements:** search businesses within radius, filter by category/rating, read-heavy, business data changes rarely.

- **Design:** geo index (**geohash** prefixes, **quadtree**, or PostGIS / Elasticsearch geo queries) → fetch businesses in my cell + neighbours → filter by exact distance → rank (distance, rating, popularity). Read replicas + cache popular areas.
- **Difference from ride-hailing:** places don't move → index can be rebuilt daily and heavily cached; drivers move → in-memory, constantly updated.
- **What-ifs:** *Dense city returns too many results:* smaller cells / quadtree split. *Rural area returns none:* expand radius gradually.

---

18. Stock exchange / order matching

> (Brief. Asked mostly at fintech and trading firms.)

- **Core: order book** per symbol: buy orders sorted by price desc, sell orders by price asc, then by time (price-time priority). Matching engine is **single-threaded per symbol** for determinism and very low latency (microseconds), in memory.
- **Durability:** every incoming order written to a sequenced, replicated log before matching; engine state can be rebuilt by replaying the log; hot standby replicas consume the same log.
- **Around it:** risk checks before orders reach the engine, market data publishing (multicast), clearing/settlement after.
- **What-ifs:** *Engine crashes:* standby takes over from same log position. *Flash crash:* circuit breakers halt trading.

---

19. Email service

> "Design an email service like Gmail."

- **Requirements:** send/receive (SMTP), store mail, search, labels, attachments, spam filtering, huge storage.
- **Design:** inbound SMTP servers → spam/virus filtering (ML) → mail storage (metadata in wide-column DB sharded by user; bodies/attachments in object storage) → search index per user → IMAP/HTTP APIs → clients; outbound queue → SMTP to other servers with retries; reputation management (SPF, DKIM, DMARC).
- **What-ifs:** *Spam wave:* rate limit senders, update models, reputation blocks. *Receiving server down:* other MTAs retry (SMTP has built-in retries). *Big attachment:* store once, reference by ID.

---

20. CDN design

> "Design a content delivery network."

- **Components:** many edge **PoPs** worldwide with caches → regional/mid-tier caches → origin. Routing users to nearest healthy PoP via **anycast** or **GeoDNS**.
- **Caching:** cache key (URL + selected headers), TTL from `Cache-Control`, **purge/invalidation** API, versioned file names (`app.3f2a.js`) to avoid purges.
- **Deep dives:** request coalescing (one origin fetch for many concurrent misses), tiered caching to protect origin, TLS at edge, DDoS absorption, edge compute.
- **What-ifs:** *Bad file cached everywhere:* purge + versioned URLs. *Origin overloaded on cache expiry:* stale-while-revalidate, shielding tier. *One PoP down:* anycast reroutes automatically.

---

21. Patterns that repeat across cases

Problem	Pattern	Seen in
Don't do the same thing twice	Idempotency keys, dedupe IDs	Payments, notifications, queues
Two users want the same thing	Atomic CAS / `SET NX` / unique constraint / optimistic locking	Tickets, rentals, ride matching
Huge read volume	Precompute + cache + CDN	Feed, autocomplete, video
Huge write volume	Append-only logs, LSM trees, batching, streaming aggregation	Metrics, KV store, view counts
Location queries	Geohash / quadtree / H3	Ride-hailing, proximity
Spiky traffic	Queue + waiting room + autoscaling	Tickets, notifications
Slow work	Async workers via queue	Video transcoding, crawling
Must not lose money/data	Strong consistency, transactions, ledger, reconciliation	Payments, file metadata
Real-time two-way	WebSocket + per-entity server	Chat, docs, ride tracking
Event to many services reliably	Transactional outbox + Kafka	Payments, orders

> **Interview tip:** after the basic design, always say: "*The hardest part of this system is ___, so let me go deeper there.*" For feed it's fan-out, for tickets it's concurrency, for payments it's correctness, for ride-hailing it's location writes.

25. Data Engineering & SQL

Almost every ML, data science and AI engineer loop has a SQL round or data pipeline questions. Models are only as good as the data pipelines feeding them.

Contents

1. Data modeling
2. Storage: warehouse, lake, lakehouse
3. File and table formats
4. Pipelines: batch, streaming, ETL vs ELT
5. Orchestration, CDC, partitioning, data quality
6. Spark essentials
7. SQL interview: concepts
8. SQL interview: 24 practice questions
9. What-if scenarios (data engineering)

---

1. Data modeling

Q: OLTP vs OLAP? | | OLTP | OLAP | |---|---|---| | Purpose | Run the app (orders, users) | Analytics, reporting, ML training data | | Queries | Many small reads/writes by key | Few large scans and aggregations | | Design | Normalized (3NF), row storage | Denormalized (star schema), columnar storage | | Examples | Postgres, MySQL | BigQuery, Snowflake, Redshift, Databricks |

Q: Normalization vs denormalization? Normalization removes duplication (less storage, easy updates, more joins). Denormalization duplicates data for faster reads (analytics, NoSQL).

Q: Star schema? One **fact table** (events/measurements: `fact_orders(order_id, date_key, customer_key, product_key, amount)`) in the middle, surrounded by **dimension tables** (descriptions: `dim_customer`, `dim_product`, `dim_date`). Simple joins, fast aggregations. **Snowflake schema:** dimensions are normalized further (`dim_product → dim_category`). Less duplication, more joins.

```
              dim_date
                 |
dim_customer ── fact_orders ── dim_product
                 |
              dim_store
```

Q: Fact table grain? The meaning of one row ("one row per order line"). Always define grain first. A wrong grain causes double counting.

Q: Slowly Changing Dimensions (SCD)? How to handle dimension values that change (customer moves city). | Type | How | History? | |---|---|---| | SCD 0 | Never change | n/a | | SCD 1 | Overwrite old value | No | | SCD 2 | New row with `valid_from`, `valid_to`, `is_current` | **Yes (most asked)** | | SCD 3 | Extra column `previous_city` | Only last change |

Why ML engineers care: SCD2 gives **point-in-time correct** features (what the customer's city was *at the time* of the order) → no leakage.

2. Storage: warehouse, lake, lakehouse

	Data warehouse	Data lake	Lakehouse
Data	Structured, cleaned	Any: raw files, images, logs	Any, with table features on top
Schema	On write	On read	Both
Cost	Higher	Cheap (object storage)	Cheap storage + compute
Examples	Snowflake, BigQuery, Redshift	S3/GCS/ADLS + files	Databricks (Delta), Iceberg/Hudi tables
Good for	BI, SQL analytics	ML raw data, archives	Both BI and ML on one copy

Medallion architecture: bronze (raw, as ingested) → **silver** (cleaned, deduped, joined) → **gold** (business-level aggregates, features).

3. File and table formats

Format	Type	Use
CSV / JSON	Row, text	Exchange, small data; slow, no types
Parquet	Columnar, compressed	Analytics/ML: read only needed columns, predicate pushdown
ORC	Columnar	Hive ecosystem
Avro	Row, schema embedded	Streaming/Kafka messages, schema evolution
Delta Lake / Apache Iceberg / Hudi	Table format over Parquet	ACID transactions, time travel, schema evolution, upserts (MERGE) on data lakes

Q: Why columnar for analytics? Queries read few columns of many rows → read less data; same-type values compress well.

4. Pipelines: batch, streaming, ETL vs ELT

- **Batch:** process data in chunks on a schedule (hourly/daily). Simple, cheap, high latency. Spark, dbt, SQL.
- **Streaming:** process events continuously (seconds). Kafka + Flink / Spark Structured Streaming / Kafka Streams. Needed for fraud, real-time features, live dashboards.
- **Lambda architecture:** batch layer (accurate) + speed layer (fast) merged. **Kappa:** stream only, reprocess by replaying the log.
- **ETL:** transform before loading into the warehouse. **ELT:** load raw first, transform inside the warehouse with SQL (dbt). ELT is the modern default (cheap storage, fast warehouses).

Streaming concepts to name: - **Event time vs processing time:** when it happened vs when we saw it. - **Windows:** tumbling (fixed, non-overlapping), sliding (overlapping), session (gap-based). - **Watermarks:** how long to wait for late events before closing a window. - **Exactly-once:** checkpoints + idempotent/transactional sinks.

5. Orchestration, CDC, partitioning, data quality

- **Orchestration:** Airflow, Dagster, Prefect. They run DAGs of tasks, retries, schedules, backfills, dependencies, SLAs.
- **Idempotent pipelines:** re-running a day gives the same result (overwrite partition `date=2026-11-01`, or MERGE by key). You need this for retries and backfills.
- **CDC (Change Data Capture):** read the database's change log (e.g., Debezium on Postgres WAL/MySQL binlog) → Kafka → lake/warehouse. Near-real-time copies without heavy queries on production DB.
- **Partitioning:** split tables by a column (usually date) so queries skip data. **Clustering / bucketing / Z-ordering:** sort data inside partitions for faster filters. Avoid over-partitioning (tiny files).
- **Data quality checks:** freshness, row counts, nulls, uniqueness, accepted values, referential integrity, distribution checks. Tools: dbt tests, Great Expectations, Soda. **Data contracts** with producers.
- **Lineage & catalog:** know where data comes from and who uses it (DataHub, Unity Catalog, OpenLineage).

6. Spark essentials

- **Driver** plans, **executors** run tasks on **partitions**. Transformations are **lazy**; actions trigger execution.
- **Narrow** transformations (map, filter) don't move data. **Wide** ones (groupBy, join) cause a **shuffle** across the network, which is expensive.
- **Joins: broadcast join** when one side is small (send it to every executor, no shuffle); sort-merge join for two big tables.
- **Data skew:** one key has most rows → one task runs forever. Fixes: salting the key, broadcast join, AQE (adaptive query execution) skew handling.
- `cache()/persist()` reused DataFrames; avoid `collect()` on big data; prefer built-in functions over Python UDFs.
- **Repartition vs coalesce:** repartition shuffles to any number; coalesce only reduces partitions without full shuffle.

---

7. SQL interview: concepts

Q: Order of execution of a SQL query? `FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT`. That's why you can't use a SELECT alias in WHERE.

Q: WHERE vs HAVING? WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q: Join types? INNER (matches only), LEFT (all left rows + matches, NULL otherwise), RIGHT, FULL OUTER, CROSS (every combination), SELF join (table with itself), anti-join (`LEFT JOIN ... WHERE right.id IS NULL` or `NOT EXISTS`), semi-join (`EXISTS`).

Q: UNION vs UNION ALL? UNION removes duplicates (slower); UNION ALL keeps all.

Q: ROW_NUMBER vs RANK vs DENSE_RANK? For scores 100, 90, 90, 80: ROW_NUMBER 1,2,3,4; RANK 1,2,2,4; DENSE_RANK 1,2,2,3.

Q: COUNT() vs COUNT(col)? COUNT() counts rows; COUNT(col) skips NULLs.

Q: NULL gotchas? `NULL = NULL` is not true → use `IS NULL`. `NOT IN (subquery with NULL)` returns no rows → use `NOT EXISTS`. Aggregates ignore NULLs.

Q: When does an index help? Selective filters, joins, ORDER BY on indexed columns. Doesn't help with functions on the column (`WHERE YEAR(created_at)=2026` → rewrite as a range), leading wildcards (`LIKE '%abc'`), or low-selectivity columns.

Q: CTE vs subquery? A CTE (`WITH x AS (...)`) has the same power, reads better, and can be recursive.

Q: Window function? Computes over a set of rows related to the current row **without collapsing rows**: `SUM(amount) OVER (PARTITION BY user_id ORDER BY created_at)`.

---

8. SQL interview: 24 practice questions

Tables used:

```
users(user_id, signup_date, country)
orders(order_id, user_id, order_date, amount, status)
events(user_id, event_time, event_name)          -- e.g. 'view', 'add_to_cart', 'purchase'
employees(emp_id, name, dept_id, salary, manager_id)
departments(dept_id, dept_name)
logins(user_id, login_date)
```

(Syntax is standard/Postgres-style; small differences across databases.)

1. Second highest salary.

```sql
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
-- or: SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

2. N-th highest salary (generic).

```sql
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 3 LIMIT 1;
```

3. Top 3 earners per department (top-N per group).

```sql
SELECT dept_id, name, salary FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees e
) t WHERE rnk <= 3;
```

4. Employees earning more than their manager (self join).

```sql
SELECT e.name
FROM employees e JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

5. Departments with no employees (anti-join).

```sql
SELECT d.dept_name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

6. Users who never ordered.

```sql
SELECT u.user_id FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

7. Find duplicate rows.

```sql
SELECT user_id, order_date, amount, COUNT(*) AS cnt
FROM orders GROUP BY user_id, order_date, amount
HAVING COUNT(*) > 1;
```

8. Delete duplicates, keep the first (dedupe).

```sql
DELETE FROM orders WHERE order_id IN (
  SELECT order_id FROM (
    SELECT order_id, ROW_NUMBER() OVER (
      PARTITION BY user_id, order_date, amount ORDER BY order_id) AS rn
    FROM orders) t
  WHERE rn > 1);
```

9. Latest order per user.

```sql
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date DESC) AS rn
  FROM orders o) t
WHERE rn = 1;
```

10. Running total of revenue by day.

```sql
SELECT order_date, daily_rev,
       SUM(daily_rev) OVER (ORDER BY order_date) AS running_total
FROM (SELECT order_date, SUM(amount) AS daily_rev FROM orders GROUP BY order_date) d;
```

11. 7-day moving average of revenue.

```sql
SELECT order_date,
       AVG(daily_rev) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM (SELECT order_date, SUM(amount) AS daily_rev FROM orders GROUP BY order_date) d;
-- note: assumes one row per day; fill missing days with a calendar table for exact results
```

12. Month-over-month growth %.

```sql
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS rev
  FROM orders GROUP BY 1)
SELECT month, rev,
       ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY month)) / LAG(rev) OVER (ORDER BY month), 2)
AS mom_pct
FROM m;
```

13. Percentage of total per country.

```sql
SELECT u.country, SUM(o.amount) AS rev,
       100.0 * SUM(o.amount) / SUM(SUM(o.amount)) OVER () AS pct_of_total
FROM orders o JOIN users u USING (user_id)
GROUP BY u.country;
```

14. Median order amount.

```sql
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median FROM orders;
-- Without PERCENTILE_CONT:
SELECT AVG(amount) FROM (
  SELECT amount, ROW_NUMBER() OVER (ORDER BY amount) AS rn, COUNT(*) OVER () AS n FROM orders) t
WHERE rn IN ((n + 1) / 2, (n + 2) / 2);
```

15. Day-1 retention (users who came back the day after signup).

```sql
SELECT COUNT(DISTINCT l.user_id) * 1.0 / COUNT(DISTINCT u.user_id) AS d1_retention
FROM users u
LEFT JOIN logins l ON l.user_id = u.user_id AND l.login_date = u.signup_date + INTERVAL '1 day';
```

16. Monthly cohort retention table.

```sql
WITH cohort AS (
  SELECT user_id, DATE_TRUNC('month', signup_date) AS cohort_month FROM users),
activity AS (
  SELECT DISTINCT user_id, DATE_TRUNC('month', login_date) AS active_month FROM logins)
SELECT c.cohort_month,
       (EXTRACT(YEAR FROM a.active_month) - EXTRACT(YEAR FROM c.cohort_month)) * 12
       + EXTRACT(MONTH FROM a.active_month) - EXTRACT(MONTH FROM c.cohort_month) AS
month_number,
       COUNT(DISTINCT a.user_id) AS active_users
FROM cohort c JOIN activity a USING (user_id)
GROUP BY 1, 2 ORDER BY 1, 2;
```

17. Conversion funnel (view → add_to_cart → purchase).

```sql
SELECT
  COUNT(DISTINCT CASE WHEN event_name = 'view' THEN user_id END)        AS viewed,
  COUNT(DISTINCT CASE WHEN event_name = 'add_to_cart' THEN user_id END) AS added,
  COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END)    AS purchased
FROM events
```

```sql
WHERE event_time >= DATE '2026-11-01';
-- Strict funnel (steps in order) → compare MIN(event_time) per step per user.
```

18. Users who logged in 3+ consecutive days (gaps and islands).

```sql
WITH d AS (
  SELECT DISTINCT user_id, login_date FROM logins),
g AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS grp
  FROM d)
SELECT user_id, MIN(login_date) AS streak_start, COUNT(*) AS streak_len
FROM g GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
-- Trick: date minus row number is constant within a consecutive streak.
```

19. Sessionize events (new session after 30 min of inactivity).

```sql
WITH e AS (
  SELECT user_id, event_time,
         CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time)
                   > INTERVAL '30 minutes'
                OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
              THEN 1 ELSE 0 END AS new_session
  FROM events)
SELECT user_id, event_time,
       SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM e;
```

20. Time between a user's consecutive orders.

```sql
SELECT user_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date) AS
days_since_prev
FROM orders;
```

21. Users whose first order was over 100.

```sql
SELECT user_id FROM (
  SELECT user_id, amount, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date, order_id)
AS rn
  FROM orders) t
WHERE rn = 1 AND amount > 100;
```

22. Pivot: revenue per status as columns.

```sql
SELECT order_date,
       SUM(CASE WHEN status = 'completed' THEN amount ELSE 0 END) AS completed,
       SUM(CASE WHEN status = 'refunded'  THEN amount ELSE 0 END) AS refunded
FROM orders GROUP BY order_date;
```

23. Daily active users (DAU) and 7-day rolling distinct users.

```sql
SELECT login_date, COUNT(DISTINCT user_id) AS dau FROM logins GROUP BY login_date;

SELECT d.login_date, COUNT(DISTINCT l.user_id) AS wau_rolling
FROM (SELECT DISTINCT login_date FROM logins) d
JOIN logins l ON l.login_date BETWEEN d.login_date - INTERVAL '6 days' AND d.login_date
GROUP BY d.login_date;
-- (COUNT DISTINCT is not allowed as a window function in most databases → self-join)
```

24. Recursive CTE: all reports under a manager (org chart).

```sql
WITH RECURSIVE reports AS (
  SELECT emp_id, name, manager_id, 1 AS level FROM employees WHERE emp_id = 1
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, r.level + 1
  FROM employees e JOIN reports r ON e.manager_id = r.emp_id)
SELECT * FROM reports;
```

How to answer SQL questions in interviews: restate the output (columns, one row per what?), ask about duplicates/NULLs/ties, build step by step with CTEs, then mention performance (indexes, partitions).

---

9. What-if scenarios (data engineering)

The daily pipeline is late and the 9 AM dashboard/model is stale

Causes (ranked): upstream source late; data volume spike; resource contention (shared cluster); a slow step (skew, bad join); failed task stuck in retries. **Isolate:** look at task timeline in the orchestrator. Which task got slower? Compare input size vs normal. **Fix now:** rerun the failed part, add resources, publish yesterday's data with a "stale" banner. **Long-term:** SLA alerts on upstream freshness, sensors waiting for data, optimize the slow step, separate compute for critical pipelines. **Prevent:** freshness SLOs + alerts *before* consumers notice.

Duplicate rows appeared in a table

Causes: at-least-once delivery from Kafka; pipeline re-ran and appended instead of overwriting; join fan-out (joining on a non-unique key); source sent the same records twice. **Isolate:** check counts by key and by load batch; check whether duplicates share the same `ingested_at`. **Fix:** make loads idempotent (overwrite partition or `MERGE` on business key), dedupe with `ROW_NUMBER()`, fix join keys. **Prevent:** uniqueness tests on primary keys in every run.

A query that took 10 seconds now takes 10 minutes

Causes: data grew; missing partition filter (full scan); stats out of date → bad plan; new join causing explosion; skew; lock contention (OLTP). **Isolate:** `EXPLAIN / EXPLAIN ANALYZE`; check bytes scanned; compare plans. **Fix:** add partition filter/index, rewrite (filter early, avoid functions on indexed columns, avoid `SELECT *`), update statistics, pre-aggregate into a summary table.

Upstream team changed a column's type / renamed a column

Fix now: pin to the previous schema version or map the column, backfill. **Long-term: data contracts** + schema registry + CI checks that fail producer changes that break consumers; schema evolution rules (add columns, never rename/remove without notice).

You need to backfill 2 years of data with new logic

Plan: idempotent job parameterized by date; run in chunks (per month), in parallel within limits; write to a new table/version, validate against old, then swap; don't overload production sources (read from snapshots/replicas); track progress.

A Spark join runs forever at 99% (one task stuck)

Cause: data skew. One key (e.g., `user_id = NULL` or a huge customer) has most rows. **Fix:** filter/handle NULL keys separately, **salt** the skewed key, broadcast the small table, enable AQE skew join.

The data lake has millions of tiny files, queries are slow

Cause: streaming writes or over-partitioning produce small files → overhead per file. **Fix:** compaction jobs (`OPTIMIZE` in Delta, rewrite in Iceberg), larger write batches, fewer partitions, auto-compaction.

Numbers in the dashboard don't match the finance report

Isolate: compare definitions (gross vs net, time zone, refunds included?, dedupe rules), compare at the lowest grain for one day. **Fix:** one shared **metrics layer / semantic layer** with agreed definitions, documented.

Training data has future information (leakage) from a pipeline

Cause: joined latest dimension values instead of values at event time. **Fix:** SCD2 dimensions + point-in-time joins (feature store). See 07.

Streaming job shows wrong counts because events arrive late

Fix: use event time with watermarks, allow late data updates, and correct with a daily batch reconciliation.

26. Cloud & Infrastructure for ML

You don't need to be a cloud architect, but ML/AI engineers are expected to know how models run in the cloud, what GPUs cost, and what to do when infrastructure breaks. MLOps practices are in 13-mlops.md; serving patterns in 02-ml-and-llm-systems.md.

Contents

1. Cloud basics
2. AWS vs GCP vs Azure service map
3. Containers & Kubernetes for ML
4. GPUs for training and inference
5. Ways to serve models in the cloud
6. Networking & security basics
7. Infrastructure as code & CI/CD
8. Observability
9. Cost optimization
10. Interview Q&A
11. What-if scenarios (cloud & infra)

---

1. Cloud basics

Building block	What it is	ML use
Compute	VMs, containers, serverless functions	training jobs, model servers, batch jobs
Storage	Object (S3-like), block (disks), file (NFS)	datasets, model weights, checkpoints
Databases	Managed SQL/NoSQL, warehouses	metadata, features, analytics
Networking	VPC, subnets, load balancers, DNS	private, secure model endpoints
IAM	Identities, roles, policies	who/what can read data, deploy models

- **Region** = geographic area; **Availability Zone (AZ)** = separate data center inside a region. Deploy across 2-3 AZs for high availability.
- **Shared responsibility:** the cloud secures the hardware; you secure your data, access, and configuration.
- **IaaS → PaaS → SaaS:** more managed = less control, less operations work.

2. AWS vs GCP vs Azure service map

Need	AWS	GCP	Azure
VMs	EC2	Compute Engine	Virtual Machines
Object storage	S3	Cloud Storage (GCS)	Blob Storage / ADLS Gen2

Need	AWS	GCP	Azure
Managed Kubernetes	EKS	GKE	AKS
Serverless functions	Lambda	Cloud Functions / Cloud Run	Azure Functions / Container Apps
ML platform	SageMaker	Vertex AI	Azure Machine Learning
Managed LLM APIs	Bedrock	Vertex AI (Gemini + model garden)	Azure OpenAI / AI Foundry
Data warehouse	Redshift	BigQuery	Synapse / Fabric
Streaming	Kinesis / MSK (Kafka)	Pub/Sub / Dataflow	Event Hubs / Stream Analytics
Batch big data	EMR / Glue	Dataproc / Dataflow	HDInsight / Databricks
Orchestration	MWAA (Airflow) / Step Functions	Cloud Composer (Airflow) / Workflows	Data Factory
Container registry	ECR	Artifact Registry	ACR
Secrets	Secrets Manager	Secret Manager	Key Vault
Monitoring	CloudWatch	Cloud Monitoring / Logging	Azure Monitor
IAM	IAM roles/policies	IAM + service accounts	Entra ID + RBAC / managed identities
Vector search	OpenSearch, Aurora pgvector	Vertex AI Vector Search, AlloyDB pgvector	AI Search

> **Interview tip:** say the concept first ("managed Kubernetes"), then the name for the cloud the company uses. Concepts transfer; names don't matter much.

3. Containers & Kubernetes for ML

- **Docker image** = code + dependencies + (sometimes) model. Pin versions; use slim base images; CUDA base images for GPU.
- **Kubernetes (K8s) basics: Pod** (running container(s)), **Deployment** (keeps N pods running, rolling updates), **Service** (stable network address), **Ingress** (HTTP routing), **ConfigMap/Secret**, **Job/CronJob** (batch), **Namespace** (isolation), **node pools** (groups of machines, e.g., GPU pool).
- **GPU scheduling:** NVIDIA device plugin exposes `nvidia.com/gpu`; request it in pod spec; use **taints/tolerations** so only GPU workloads land on expensive GPU nodes; **MIG / time-slicing** to share one GPU between small models.
- **Autoscaling:**
- **HPA:** more pods based on CPU/GPU/custom metrics (QPS, queue length, latency).
- **Cluster autoscaler / Karpenter:** more nodes when pods can't be scheduled.
- **KEDA:** scale on events (queue depth), including **scale to zero**.
- **ML serving on K8s: KServe**, Seldon, Ray Serve, BentoML, Triton, vLLM/TGI containers. Features: canary rollouts, autoscaling, model versioning, request batching.

- **Training on K8s:** Kubeflow Training Operator, Ray, Volcano/Kueue (gang scheduling, so all GPUs for a distributed job start together or not at all).
- **Health probes:** *readiness* (ready for traffic after model loaded) vs *liveness* (restart if stuck). A big model needs a long startup/readiness time or it gets killed in a loop.

4. GPUs for training and inference

Topic	Key points
Why GPUs	Thousands of cores for parallel matrix math; high memory bandwidth
Families (examples)	Data-center: A100, H100/H200, B200 (training + big LLM inference); L4, A10G, T4 (cheaper inference)
Memory is usually the limit	Model weights + activations + optimizer states (training) or KV cache (LLM inference)
Training memory rule	Full fine-tune with Adam in mixed precision ≈ 16-20 bytes/parameter → 7B model ≈ 120-140 GB → multiple GPUs or LoRA/QLoRA
Inference memory rule	FP16 = 2 bytes/param (7B ≈ 14 GB) + KV cache; INT4 ≈ 0.5 byte/param
Multi-GPU	Data parallel (DDP), FSDP/ZeRO, tensor & pipeline parallel; fast interconnect (NVLink, InfiniBand) matters
Utilization	Watch GPU util %, memory, SM occupancy; low util usually = data loading bottleneck or small batches
Alternatives	TPUs (GCP), AWS Trainium/Inferentia, CPUs for small models
Cost	GPUs are the biggest ML cost line, and idle GPUs are the #1 waste

5. Ways to serve models in the cloud

Option	Good for	Trade-off
Managed endpoint (SageMaker / Vertex / Azure ML)	Fast start, built-in autoscaling & monitoring	Less control, platform cost
Kubernetes (KServe, vLLM)	Many models, custom needs, cost control at scale	You operate it
Serverless containers (Cloud Run, Lambda, Container Apps)	Spiky/low traffic, small CPU models, scale to zero	Cold starts, limited GPU support, size limits
Serverless GPU providers / managed LLM APIs	LLMs without running GPUs	Per-token cost, vendor dependency
Batch jobs (Spark, Batch services)	Precompute predictions	Not real-time
Edge / on-device	Privacy, offline, very low latency	Small models only

6. Networking & security basics

- **VPC:** your private network in the cloud. **Public subnet** (internet-facing load balancers) vs **private subnet** (model servers, databases).
- **Private endpoints / PrivateLink / Private Service Connect:** reach cloud services (storage, LLM APIs) without going over the public internet. Often required for sensitive data.
- **Security groups / firewall rules:** allow only needed ports and sources.
- **API gateway** in front of models: auth, rate limits, quotas, logging.
- **IAM least privilege:** training job role can read only its dataset bucket; serving role can read only the model registry. Use **workload identity** (no long-lived keys in code).
- **Encryption:** in transit (TLS) and at rest (KMS-managed keys, customer-managed keys for regulated industries).
- **Secrets:** in a secrets manager, injected at runtime. Never put them in images, code, notebooks, or prompts.
- **Data residency:** some data must stay in a country/region → choose region and LLM endpoints accordingly.

7. Infrastructure as code & CI/CD

- **IaC:** Terraform / OpenTofu, Pulumi, CloudFormation, Bicep. Infra is reviewed in pull requests, reproducible, versioned; avoid manual console changes ("click-ops") which cause drift.
- **CI:** lint, unit tests, data/model tests, build image, security scan of image and dependencies.
- **CD:** deploy to staging → automated checks → canary in production → full rollout; GitOps (Argo CD / Flux) keeps cluster state equal to git.
- **Environments:** dev / staging / prod in separate accounts/projects for blast-radius control.

8. Observability

- **Metrics:** latency p50/p95/p99, QPS, error rate, GPU util/memory, queue depth, tokens/s, cost.
- **Logs:** structured JSON with request IDs; redact PII.
- **Traces:** OpenTelemetry across gateway → service → model → vector DB.
- **Dashboards & alerts** on SLOs (e.g., p95 < 500 ms for 99% of 5-min windows), not on every metric.
- ML-specific monitoring (drift, quality) in 13 and 02 §6.

9. Cost optimization

Lever	How
Spot / preemptible instances	60-90% cheaper; for training with checkpointing, batch jobs; not for critical online serving alone
Reserved instances / savings plans / committed use	Discount for steady baseline load
Autoscaling & scale to zero	No idle GPUs at night; KEDA, serverless
Right-sizing	Smaller GPU (L4 vs H100) for inference; CPU for small models

Lever	How
Model optimization	Quantization, distillation, batching, caching (see 02 §8)
Storage tiers & lifecycle rules	Move old datasets/checkpoints to cold storage, delete temp data
Data transfer	Keep compute next to data (same region); egress costs money
Visibility	Tags/labels per team/project, budgets, alerts, cost dashboards (FinOps)

10. Interview Q&A

Q: How would you deploy a PyTorch model as a scalable API? Package with a model server (FastAPI/TorchServe/Triton) in a Docker image → push to registry → deploy on managed endpoint or K8s (Deployment + Service + HPA) behind a load balancer/API gateway → readiness probe after model load → canary rollout → metrics, logs, alerts.

Q: How do you train a model that needs 8 GPUs for 3 days without wasting money? Spot instances with frequent checkpointing to object storage and automatic resume; gang scheduling; profile first on a small run; data in same region; mixed precision; shut everything down automatically at the end.

Q: Managed ML platform vs Kubernetes? Managed: faster, less ops, good for small teams. K8s: flexibility, many models, cheaper at scale, needs platform engineers. Many companies mix: managed training, K8s serving.

Q: How do you keep secrets out of code? Secrets manager + workload identity; scan repos for secrets in CI; rotate keys.

Q: How do you make a model endpoint highly available? Multiple replicas across AZs, health checks, autoscaling with minimum replicas, graceful degradation/fallback model, multi-region for critical systems.

Q: Cold start problem? Container start + model download/load can take minutes for big models. Fixes: min replicas warm, bake weights in image or cache on node-local disk, faster formats (safetensors), lazy load, smaller models.

11. What-if scenarios (cloud & infra)

GPU quota or capacity is not available in our region

Mitigate now: use another GPU type or region (if data rules allow), queue the training job, use smaller model/LoRA to fit smaller GPUs. **Long-term:** request quota early, reserved capacity/commitments for critical needs, multi-region/multi-cloud capability, portable containers.

Pods are stuck in "Pending"

Causes (ranked): not enough nodes/GPUs (cluster autoscaler at max or no capacity), resource requests too big for any node, taints without matching tolerations, node selector/affinity mismatch, quota limits per namespace, PVC can't attach (volume in another zone). **Isolate:** `kubectl describe pod` → Events section tells the reason. **Fix:** adjust requests, add node pool, fix tolerations/selectors, raise quota.

Cloud bill doubled this month

Isolate: cost explorer by service → by tag/team → by resource. **Usual causes:** forgotten GPU instances/endpoints, autoscaling min too high, egress (cross-region traffic), logs volume explosion, storage of old checkpoints, runaway LLM API usage. **Fix:** stop idle resources, lifecycle rules, budgets + alerts, auto-shutdown for notebooks, per-team quotas. **Prevent:** cost dashboards reviewed weekly, tagging enforced by policy.

The whole region has an outage

Depends on design: single-region → service down (state the RTO/RPO honestly). Better: multi-AZ always; for critical systems, warm standby in a second region with replicated data, DNS failover, models and images replicated across regions, tested runbooks (game days).

New pods take 8 minutes to start because the model downloads every time

Fix: cache weights on node-local SSD or a shared volume, pre-pull images (DaemonSet), bake model into image for small models, use faster storage and parallel download, keep min warm replicas, readiness probe set correctly so traffic waits.

An API key / cloud secret was pushed to a public GitHub repo

This is an incident. Act in minutes: revoke/rotate the key immediately, check access logs for misuse, remove from git history (but assume it's already copied), check billing for abuse (crypto mining is common). **Prevent:** secret scanning in CI and pre-commit hooks, short-lived credentials via workload identity, no long-lived keys.

Latency is fine in one AZ but bad in another

Cross-AZ calls (model in AZ-a, vector DB in AZ-b), noisy neighbours, uneven load balancing. Fix: zone-aware routing, colocate dependent services, check LB health checks.

Training job keeps getting killed on spot instances

This is expected, so design for it: checkpoint every N minutes, auto-resume, mix of spot + on-demand for the coordinator, diversify instance types.

27. Security, Privacy & Responsible AI

Senior AI/ML interviews almost always include a question like "*How would you make this safe?*", "*What could an attacker do?*" or "*How do you make sure the model is fair?*". This chapter gives you the vocabulary and the structured answers. Related: 04 #7, 04 #15, 04 #20, 22 §8.

> Note: laws differ by country and change often. In interviews, describe principles and say you would work with legal/compliance teams. Never claim a design is "fully compliant" on your own.

Contents

1. Threat modeling for ML and LLM systems
2. Attacks on LLM applications
3. Attacks on ML models and data
4. Supply chain security
5. Defenses checklist
6. Privacy
7. Fairness
8. Explainability and transparency
9. Regulation and risk tiers
10. AI governance inside a company
11. What-if scenarios

---

1. Threat modeling for ML and LLM systems

Q: What is threat modeling? Systematically asking: *What are we protecting (assets)? Who might attack (actors)? How (attack surface)? What happens if they succeed (impact)? What stops them (controls)?*

Q: Assets in an AI system? Training data, user data and conversations, model weights, system prompts and business logic, tool credentials, the model's decisions (loans, moderation), compute budget.

Q: Attack surface of an LLM app? User input, uploaded files, retrieved documents, web pages and emails the agent reads, tool outputs, third-party models/plugins, training data sources, the model API itself, logs.

Q: A simple framework to use in interviews? Walk the pipeline (same layer map as 03) and for each layer ask about **C-I-A**: Confidentiality (can data leak?), Integrity (can someone change data or behavior?), Availability (can someone make it slow/expensive/down?).

Q: Known reference lists? There are public community lists of top LLM application risks (prompt injection, sensitive information disclosure, supply chain, data and model poisoning, improper output handling, excessive agency, system prompt leakage, vector/embedding weaknesses, misinformation, unbounded consumption) and knowledge bases of adversarial ML tactics. Mentioning that you use such checklists sounds senior.

2. Attacks on LLM applications

Attack	What happens	Main defenses
Direct prompt injection / jailbreak	user tricks model into ignoring rules	layered guardrails, safe model, never rely on the prompt for security
Indirect prompt injection	hidden instructions in documents, web pages, emails, tool results	treat retrieved content as untrusted, approval for actions, limit outbound channels
Sensitive data disclosure	model reveals PII, secrets, other tenants' data	data minimization, ACL pre-filtering, output PII filters, no secrets in prompts
System prompt leakage	attacker extracts instructions	assume prompts are public; keep secrets and auth logic out
Insecure output handling	model output used directly in SQL, shell, HTML → injection/XSS	treat output as untrusted input: escape, parameterize, validate
Excessive agency	agent has more permissions than needed	least privilege, approvals, scoped tokens (22 §8)
Denial of wallet / resource abuse	huge prompts, loops, bots burn your budget	rate limits, token quotas, max steps, abuse detection
Vector store attacks	poisoned documents planted to be retrieved; cross-tenant leakage	ingestion controls, provenance, tenant isolation
Data exfiltration via rendering	model outputs an image/link URL containing secrets that the client auto-loads	block or proxy external URLs in rendered output

Q: Can prompt injection be fully solved today? No. Say: "*I treat it like phishing. I reduce the chance with classifiers and good prompting, but design the system so a successful injection has limited impact.*"

3. Attacks on ML models and data

Q: Data poisoning? Attacker inserts bad training examples to degrade the model or plant a **backdoor** (a trigger pattern that causes a chosen output, e.g., a small sticker makes a stop sign "speed limit"). Risk sources: scraped web data, user feedback loops, crowd labels, public datasets. **Defenses:** data provenance and trusted sources, anomaly/outlier detection on training data, label audits, limiting the weight of any single user's feedback, robust training, testing for backdoor triggers, keeping clean holdout sets.

Q: Adversarial examples (evasion)? Tiny, often invisible input changes that flip a prediction (image noise, character swaps in text, feature manipulation in fraud). **Defenses:** adversarial training, input preprocessing/denoising, ensembles, monitoring confidence, rate-limiting probing, not exposing raw scores, combining ML with rules and human review for high-stakes decisions.

Q: Model extraction (stealing)? Attacker queries your API many times and trains a copy. **Defenses:** rate limits and quotas, detect systematic query patterns, return labels not full probabilities, watermark model outputs, legal terms.

Q: Membership inference? Attacker determines whether a specific record was in the training data (a privacy leak, e.g. "this patient was in the cancer dataset"). **Defenses:** regularization (overfit models leak more), differential privacy, limit output detail.

Q: Model inversion / training data extraction? Reconstruct training data from the model (LLMs can repeat memorized text like emails or keys). **Defenses:** deduplicate and scrub PII/secrets from training data, differential privacy, output filters, test for memorization.

Q: Fraud and abuse adversaries adapt. How is that different? Attackers change behavior after you deploy (concept drift by design). Use frequent retraining, rules for quick response, red-teaming, and features that are costly for attackers to fake.

4. Supply chain security

Q: Risks? Downloaded model files can contain **malicious code** (Python pickle files execute code when loaded), compromised open-source packages, poisoned public datasets, third-party model APIs changing behavior, malicious plugins/tool servers. **Defenses:** - Prefer safe weight formats (e.g., safetensors) over pickle; never load untrusted pickles. - Pin versions and hashes of models, datasets and packages; scan dependencies; private mirrors. - Model and dataset provenance records (who trained, on what, when). People call this an "AI bill of materials". - Sandboxed evaluation of new third-party models before use. - Vendor reviews for model providers (data retention, training on your data, regions).

5. Defenses checklist

Use this as a closing list in any design interview: 1. **Identity & access:** authN for users, authZ enforced in backend, tenant isolation, least privilege for tools and services. 2. **Data:** minimization, classification (public/internal/confidential/PII), encryption in transit and at rest, retention limits, access logging. 3. **Input controls:** validation, size limits, injection/abuse classifiers, file scanning. 4. **Output controls:** PII/secret filters, policy classifiers, output treated as untrusted in downstream systems. 5. **Action controls:** risk tiers, approvals, limits, idempotency, undo. 6. **Model controls:** safe formats, version pinning, evals for safety, red-teaming before launch. 7. **Monitoring:** abuse patterns, cost spikes, unusual queries (extraction), drift, safety-violation rates, audit logs. 8. **Incident response:** kill switch / feature flag, rollback, credential revocation, user notification process, post-mortem.

6. Privacy

Q: What is PII and how do you handle it in ML pipelines? Personally identifiable information: names, emails, phone numbers, IDs, addresses, plus quasi-identifiers (ZIP + birth date + gender can identify most people). Handle by: detect (regex + NER models), minimize (don't collect what you don't need), mask/tokenize before training and logging, restrict access, set retention, and keep a data inventory.

Q: Anonymization vs pseudonymization? Pseudonymization replaces identifiers with tokens but can be reversed with a key → still personal data under many laws. Anonymization makes re-identification practically impossible. That is hard to achieve, and simple removal of names is usually not enough.

Q: k-anonymity? Each record is indistinguishable from at least $k-1$ others on quasi-identifiers. Weak against attackers with background knowledge; extended by l-diversity and t-closeness.

Q: Differential privacy (DP)? A mathematical guarantee that the output (statistic or model) changes very little whether or not any single person's data is included. Achieved by adding calibrated noise. **ε (epsilon)** = privacy budget: smaller ε → stronger privacy, lower accuracy. **DP-SGD** clips per-example gradients and adds noise during training.

Q: Federated learning? Train on devices or separate organizations without moving raw data: clients compute updates locally, a server aggregates them (FedAvg). Used for keyboards, hospitals. Challenges: non-IID data across clients, communication cost, updates can still leak information → combine with **secure aggregation** and DP.

Q: Other privacy-enhancing technologies? Secure multi-party computation, homomorphic encryption (compute on encrypted data; slow), trusted execution environments (confidential computing), synthetic data (check it doesn't memorize real records).

Q: Data protection principles you should mention (GDPR-style)? Lawful basis/consent, purpose limitation (data collected for X not reused for Y without basis), data minimization, storage limitation, accuracy, security, and individual rights: access, correction, deletion ("right to be forgotten"), objection, and rights around **solely automated decisions with significant effects** (explanation and human review).

Q: Why is the right to be forgotten hard in ML? Data may be in raw storage, feature store, training snapshots, vector indexes, caches, logs, backups, **and inside trained model weights**. Plan: data lineage to find every copy; delete from stores and indexes; retrain on schedule without the data (or use **machine unlearning** techniques, which are still an active research area); prefer RAG over fine-tuning for personal data so deletion = delete from index.

Q: Data residency? Some data must stay in a specific country/region. Design: region-specific deployments and storage, region-pinned model endpoints, no cross-region logging/replication of that data, vendor contracts covering regions.

Q: What do you check before sending customer data to a third-party LLM API? Data retention, whether data is used for training, encryption, regions, certifications, contractual terms, zero-retention options; redact PII before sending when possible.

7. Fairness

Q: Where does bias come from? Historical bias in labels (past decisions were biased), representation bias (group under-sampled), measurement bias (proxy labels like "arrests" for "crime"), aggregation bias (one model for very different groups), deployment bias (used in a different context than designed), feedback loops.

Q: Fairness metrics? | Metric | Meaning | |---|---| | **Demographic parity** | same positive-prediction rate across groups | | **Equal opportunity** | same true positive rate (recall) across groups | | **Equalized odds** | same TPR **and** FPR across groups | | **Predictive parity** | same precision across groups | | **Calibration within groups** | a score of 0.7 means 70% for every group | | **Individual fairness** | similar individuals get similar outcomes |

Q: Can you satisfy all of them? No. When base rates differ between groups, several of these are mathematically incompatible (impossibility results). You must **choose based on context** with stakeholders and document the choice.

Q: How do you mitigate bias? - **Pre-processing:** collect more representative data, reweight/resample, fix biased labels, remove proxy features (carefully: removing the protected attribute alone doesn't remove bias). - **In-processing:** fairness constraints or adversarial debiasing during training. - **Post-processing:** group-specific thresholds (check legal acceptability), calibration. - **Process:** slice-based evaluation before every launch, human review for high-stakes cases, monitoring by group after launch.

Q: Fairness in LLMs? Stereotypes in generations, different quality for different languages/dialects, refusing more for some groups. Evaluate with counterfactual prompts (swap names/genders/dialects and compare outputs), red-teaming, per-language eval sets.

8. Explainability and transparency

Q: Global vs local explanations? Global: how the model behaves overall (feature importance, partial dependence). Local: why this one prediction (SHAP values for this applicant).

Q: Methods? | Method | Notes | |---|---| | Interpretable models (linear, small trees, GAMs) | best when regulation requires clear reasons | | **SHAP** | game-theory based feature attributions; consistent; can be slow; correlated features complicate | | **LIME** | fits a simple local model around one prediction; unstable | | Permutation importance | global, model-agnostic | | Partial dependence / ICE plots | effect of one feature | | Counterfactual explanations | "If income were $5k higher, the loan would be approved." Users can act on it | | Grad-CAM / saliency | which image regions mattered | | Attention visualization | not a reliable explanation by itself | | LLM citations / reasoning summaries | show sources; reasoning text may not reflect true internal process |

Q: Model cards and datasheets? Short documents describing a model's intended use, out-of-scope uses, training data, evaluation results (including by group), limitations, and ethical considerations. Datasheets do the same for datasets. Standard practice for responsible releases.

Q: Transparency to users? Tell users when they are interacting with AI, label AI-generated content, give reasons for important automated decisions, and offer a way to reach a human or appeal.

9. Regulation and risk tiers

Many AI regulations (for example, the EU's AI Act) use a **risk-based approach**. Described generally:

Tier	Examples	Typical obligations
Unacceptable / prohibited	manipulative systems exploiting vulnerable people, certain social scoring	not allowed
High risk	credit scoring, hiring, education access, medical devices, critical infrastructure, law enforcement uses	risk management, data governance, documentation, logging, human oversight, accuracy/robustness, conformity checks
Limited risk / transparency	chatbots, deepfakes/generated content	tell users it's AI, label generated content
Minimal risk	spam filters, game AI	voluntary codes

General-purpose / foundation model providers often have extra duties (technical documentation, copyright policies, and more for very capable models). Sector rules also apply: finance (fair lending, model risk management), health (medical device and health data rules), employment (anti-discrimination). **Interview answer:** identify the risk level of the use case early, and build documentation, logging, human oversight and evaluation into the design from day one.

10. AI governance inside a company

- **AI use policy:** which tools/models are approved, what data may be used with them.
- **Model inventory / registry:** every production model with owner, purpose, risk tier, data used, evaluation results.
- **Review process:** risk assessment before launch (privacy, security, fairness, legal); higher risk → more review.
- **Red-teaming:** structured adversarial testing before release and regularly after.
- **Human oversight design:** who can override, how, and are they trained not to rubber-stamp (automation bias).
- **Incident management:** process for AI incidents (harmful output, data leak, biased decisions), with post-mortems.
- **Monitoring and audits:** periodic re-evaluation, especially after model or data changes.
- **Roles:** model owner, data steward, security, legal/privacy, responsible-AI reviewers.

---

11. What-if scenarios

A user tricked the support chatbot into showing another customer's order details (data breach).

Mitigate now: disable the affected tool or feature, preserve logs, start the incident process (security plus legal/privacy, since notification duties may apply). **Causes (ranked):** order-lookup tool trusted an order ID chosen by the model instead of checking the logged-in user; shared cache/memory across users; retrieval without tenant/ACL filters; prompt injection. **Isolate:** replay the conversation trace; check which tool call returned the data and what authorization it used. **Fix:** authorization in the backend (tool only returns orders owned by the authenticated user), user-scoped caches/memory, ACL pre-filtering, output PII checks. **Prevent:** authorization tests ("user A cannot read B"), red-team prompts in CI, audit logs and alerts on cross-account access patterns.

You discover poisoned data in the training set (spam reviews designed to fool the classifier).

Isolate: find the source (one data feed? specific accounts? time window?), measure how much and which model versions trained on it. **Fix now:** roll back to a model trained before the poisoning, block the source. **Long-term:** retrain on cleaned data, provenance tracking, anomaly detection on incoming training data, cap influence per user/source, backdoor trigger tests. **Prevent:** data validation gates in the training pipeline, trusted-source policies.

A regulator (or customer) asks: "Why was this loan application denied?"

Answer structure: the decision record must be reproducible: model version, input features at decision time, score, threshold, and policy rules applied (log all of these). Provide **reason codes** from local explanations (SHAP or an interpretable model), plus counterfactual guidance ("higher income or lower debt ratio would change the outcome"). Show the model's documentation (model card, validation, fairness tests) and the human-review/appeal process. **If you can't explain it:** that is a design flaw for a high-risk use case → use interpretable models or constrained models, and log decisions properly.

A user asks you to delete their data, but it was used to fine-tune a model.

Steps: delete from raw data, features, vector indexes, caches, logs (per retention policy) using data lineage; for the model: assess risk (was the data memorizable? test by probing), retrain/fine-tune again without the data on the next cycle or sooner if needed, or apply unlearning methods with validation. **Prevent:** don't fine-tune on personal data unless necessary; prefer RAG for personal facts; record which datasets each model used.

Your hiring/credit model shows lower approval rates for one group.

Clarify: which fairness metric matters here and what is legally required (with legal team). **Isolate:** slice metrics (TPR, FPR, calibration by group), check representation in data, look for proxy features (ZIP code, school), check label bias. **Fix:** better data, remove/transform proxies, fairness-aware training, threshold review, human review for borderline cases. Pause the model if harm is serious. **Prevent:** fairness tests as a launch gate, monitoring by group, documentation of trade-offs. See 04 #29.

Attackers fool your vision model with stickers on products (adversarial attack).

Fix now: add rules/human checks for the affected flow, collect the attack examples. **Long-term:** adversarial training with physical-style augmentations, ensembles of different models, multi-sensor/multi-view checks, confidence monitoring, rate-limit probing. **Prevent:** red-team the model before launch with known attack types.

Someone is cloning your model through your public API.

Signs: high-volume, systematic, diverse queries from few accounts. **Fix:** rate limits and quotas, return less detail (labels instead of probabilities), anomaly detection on query patterns, output watermarking, terms of service enforcement.

An engineer downloaded a community model file and loaded it in production.

Supply-chain risk (pickle files can run code). **Fix now:** check the environment for compromise, rotate credentials if needed. **Long-term:** only safe formats (safetensors), approved model registry, scanning, sandboxed evaluation, pinned hashes.

The LLM's answers are noticeably worse for users writing in a regional dialect.

Fairness/quality gap. Build per-language/dialect eval sets, counterfactual tests, add representative data or translation steps, choose models that perform better on those languages, monitor quality by language.

Your team wants to send customer support transcripts to an external LLM provider.

Check data processing terms (retention, training use, regions), redact PII before sending, get privacy/legal approval, data minimization (only needed fields), logging and access controls, option for a self-hosted model for the most sensitive data.

28. AI Product Sense

Senior AI/ML roles, AI product manager roles and many startup interviews include questions like **"How would you measure success of this AI feature?"** or **"Should we build our own LLM?"** They test whether you can connect models to **users, business value and cost**, not just accuracy.

Contents

1. The product-sense framework
2. Framing a business problem as ML
3. Do we even need ML or an LLM?
4. Success metrics: north star and guardrails
5. Build vs buy
6. MVP and iteration
7. Cost and ROI estimation for AI features
8. UX for AI features
9. Launch and rollout
10. Stakeholder communication
11. Prioritization
12. Practice questions with model answers
13. What-if scenarios (product)

---

1. The product-sense framework

Use this order for any open product question. Memory trick: **"U-P-S-M-R"** (Users, Problem, Solution, Metrics, Risks).

Step	What to say
1. Users	Who uses it? Segments (new vs power users, consumers vs businesses). What do they need?
2. Problem	What pain are we solving? How do we know it's real (data, complaints, research)? How big is it?
3. Solution	Options (non-AI first, then AI). Pick one with reasons. Define the MVP.
4. Metrics	North-star metric, supporting metrics, guardrail metrics. How we'd test it (A/B).
5. Risks	Quality failures, safety, privacy, cost, trust. Mitigation for each.

A sentence you can reuse:

> "I'd start from the user problem, not the model. The model is only useful if it changes a user outcome we can measure."

---

2. Framing a business problem as ML

Business goal	Possible ML framing	Label / signal
Reduce customer churn	Classify who will churn; better: **uplift** (who will stay because of an offer)	Churned within 60 days; response to offer
Reduce support costs	Classify & route tickets; auto-answer FAQs with RAG	Resolution without human, CSAT
Increase sales	Recommend products; rank search results	Purchases, add-to-cart
Reduce fraud losses	Score transactions	Chargebacks, confirmed fraud
Save employee time	Summarize, draft, extract	Time per task, edit distance of AI draft vs final

Questions to ask while framing: - What **decision** will the prediction drive? (A prediction no one acts on has zero value.) - What is the **proxy** we optimize, and how could it diverge from the real goal? (Clicks vs satisfaction.) - What **error is more costly**: false positive or false negative? - Where do **labels** come from, and how delayed are they? - What's the **baseline** today (manual process, rules, nothing)?

---

3. Do we even need ML or an LLM?

Strong candidates are willing to say "no ML needed". Use this check:

Use plain software / rules when…	Use classic ML when…	Use an LLM when…
Logic is clear and stable	Patterns are complex but the task is narrow (score, classify, rank)	Input or output is open-ended language, images, or code
Errors are not acceptable and rules can guarantee correctness	You have labeled data or behavior logs	You need flexibility across many tasks with little labeled data
Small volume, a human can do it	Low latency and low cost per prediction matter	Reasoning over text, summarizing, drafting, conversation
Example: tax calculation	Example: fraud score, demand forecast	Example: support assistant, document Q&A

Rule of thumb: "Start with the simplest thing that could work, measure it, then add complexity only where the data shows a gap."

---

4. Success metrics: north star and guardrails

Metric type	Meaning	Example (AI support assistant)
North star	The one metric that shows real user/business value	% of tickets resolved without a human, with good CSAT

Metric type	Meaning	Example (AI support assistant)
Supporting / driver	Explains why the north star moves	Answer acceptance rate, deflection rate, time to answer
Model quality	Offline and online quality of the AI	Faithfulness, retrieval recall@k, thumbs-up rate
Guardrails	Must NOT get worse	CSAT, escalation complaints, wrong refunds, latency, cost per ticket, safety incidents
Adoption	Are people using it?	Weekly active users of the feature, repeat usage

Common traps to mention: - **Usage ≠ value:** many people trying the feature once is not success; look at retention of the feature. - **Proxy gaming:** optimizing "answers given" can increase confident wrong answers. - **Survivorship:** only happy users give feedback; sample conversations for human review. - **Novelty effect:** usage spikes then drops; measure over weeks.

---

5. Build vs buy

Option	Pros	Cons	Choose when
Use an API model (buy)	Best quality, fast to launch, no infra	Per-token cost, vendor dependency, data leaves your system (check contract), rate limits	Most new AI features; validating demand
Open-weight model, self-hosted	Control, data stays inside, cost-efficient at high volume, can fine-tune	GPU operations, scaling, lower top quality, team skills needed	High volume, strict privacy, narrow task, latency needs
Fine-tune (API or open)	Better for specific format/style/domain, smaller cheaper model	Needs good data, maintenance, evals	Prompting + RAG not enough, or cost reduction at scale
Train your own foundation model	Full control	Extremely expensive (millions+), slow, needs rare expertise	Almost never; only if the model *is* the product and you have unique data & budget
Buy a full SaaS product	Fastest	Least customization, shared roadmap	Commodity need (e.g., meeting transcription)

Decision factors to list: quality needed, data privacy, volume & cost curve, latency, time to market, team skills, strategic differentiation ("is this our core advantage?").

---

6. MVP and iteration

1. **Wizard of Oz / manual first:** humans do the task behind the scenes to test if users want it.
2. **Prompt + API prototype** in days; test with internal users.
3. **Eval set** from real examples before scaling (50-200 cases is enough to start).
4. **Limited beta** with clear scope ("only answers HR policy questions").

5. **Measure, collect failures, improve** (prompt → retrieval → model choice → fine-tune, in that order).
6. **Expand scope** gradually.

Scope cuts that make a good MVP: one user segment, one language, one task, human-in-the-loop instead of full automation, read-only actions before write actions.

---

7. Cost and ROI estimation for AI features

Token cost math (practice this, interviewers love it)

```
cost per request = (input_tokens × input_price + output_tokens × output_price) / 1,000,000
monthly cost     = cost per request × requests per day × 30
```

Example: support assistant, 20,000 conversations/day, 4 turns each. - Per turn: 3,000 input tokens (system prompt + retrieved context + history), 300 output tokens. - Assume price: $3 per 1M input tokens, $15 per 1M output tokens (use the real prices of the model you'd pick). - Per turn: 3,000 × 3/1M + 300 × 15/1M = $0.009 + $0.0045 = **$0.0135** - Per day: 20,000 × 4 × $0.0135 = **$1,080** → about **$32,400 / month**.

ROI: if each conversation resolved by AI saves a $4 human ticket and 40% are resolved: 20,000 × 0.4 × $4 × 30 = **$960,000 / month saved** vs ~$32k LLM cost (+ infra, people). Clear win.

Levers to cut cost (say these after the math)

Lever	Typical effect
Shorter context (better retrieval, fewer chunks, summarize history)	Input tokens down 30-60%
Prompt / prefix caching for the fixed system prompt	Big discount on repeated tokens
Model routing (small model for easy questions)	50-80% cost cut on routed traffic
Semantic cache for repeated questions	Removes some calls fully
Limit output length	Output tokens are the most expensive
Batch API for non-urgent work	Often ~50% cheaper
Self-hosting at very high volume	Lower marginal cost, higher fixed cost

Also count **hidden costs**: embeddings, vector DB, evaluation runs (LLM judges cost tokens too), human review, engineering time.

---

8. UX for AI features

AI is probabilistic, so the product must handle being wrong gracefully.

Principle	How
Set expectations	Say what it can and can't do; examples of good questions
Show sources	Citations / links so users can verify

Principle	How
Show confidence carefully	Use "I'm not sure, here's what I found" or route to human; avoid fake precise percentages
Easy correction	Edit, regenerate, thumbs down with reason
Human handoff	One click to a human, with conversation context passed along
Fallbacks	When the model fails or times out: cached answer, search results, or clear error
Keep humans in control of risky actions	Preview + confirm before sending emails, payments, deletions
Latency perception	Stream tokens, show progress steps ("Searching documents...")
Privacy transparency	Tell users what data is used and stored
Consistency	Same question should get the same core answer (see 03)

---

9. Launch and rollout

1. **Offline evals pass** (quality, safety, latency, cost thresholds).
2. **Internal dogfooding** → collect failures.
3. **Shadow mode** where possible (AI suggests, human decides, compare).
4. **Staged rollout:** 1% → 10% → 50% → 100% with an A/B test and guardrail alerts.
5. **Kill switch / feature flag** ready.
6. **Monitoring dashboard:** quality (thumbs, escalations), safety incidents, latency, cost per request.
7. **Feedback loop:** weekly review of failures → eval set grows → improvements.

See also 02 §5 and 13.

---

10. Stakeholder communication

Audience	They care about	How to talk
Executives	Business impact, risk, cost, timeline	"This reduces support cost ~30%, costs ~$30k/month, main risk is wrong answers, mitigated by citations and human handoff."
Product managers	User value, scope, trade-offs	Options with pros/cons, what the MVP includes
Legal / compliance	Data use, regulation, liability	Data flow diagram, retention, vendor terms, audit logs
Engineers	Architecture, effort, dependencies	Design docs, APIs, SLAs
End users	"Can I trust it?"	Clear UI, sources, limits, how to get a human

Explaining model uncertainty to non-technical people:

> "The model is right about 9 times out of 10 on our test questions. For the 1 in 10 it gets wrong, we show sources so people can check, and we send risky cases to a human."

---

11. Prioritization

Use **impact × confidence ÷ effort** (ICE/RICE) and add AI-specific factors: - **Data readiness:** do we already have the data and labels? - **Error tolerance:** can the use case survive mistakes? (Drafting emails: yes. Medical dosing: no.) - **Feedback loop:** will usage generate data to improve it? - **Cost at scale:** is unit economics positive?

Good first AI use cases: high volume, repetitive, tolerant of errors with human review, clear success metric (e.g., ticket triage, document extraction, internal search).

---

12. Practice questions with model answers

1. How would you measure success of an AI writing assistant (like in an email or docs product)?

Users: people writing emails/documents. North star: **% of AI suggestions kept** (accepted with little editing) by weekly active users, or time saved per document. Supporting: suggestion acceptance rate, edit distance between AI draft and final text, repeat usage (D7/D30 retention of the feature). Quality: human ratings on samples, tone/grammar eval. Guardrails: complaints, harmful/off-tone outputs, latency (suggestions must be fast), cost per active user, no drop in overall product retention. Test: A/B test feature on vs off, measure time-to-send and satisfaction.

2. Should we build our own LLM?

Almost always no for the base model. Ask: Is the model our core product? Do we have unique data at huge scale and millions in budget? Usually better: API model + RAG + prompt engineering, then fine-tune an open model if cost, privacy or latency demand it. I'd list decision factors (quality, privacy, cost at volume, latency, skills, strategic value) and propose: start with API, build eval set, re-evaluate at a cost/volume threshold.

3. Our AI feature has low adoption. Why, and what would you do?

Clarify: low discovery (users don't see it), low trial (they see but don't try), or low retention (try once, don't return)? Look at the **funnel**. Causes: hidden entry point, unclear value, bad first experience (wrong answer), too slow, users don't trust it, doesn't fit workflow. Actions: interview users and read sampled sessions, fix the first-run experience, show example prompts, integrate into existing workflow (suggest, don't make them ask), measure each funnel step after changes.

4. How would you decide between a rules-based system and ML for fraud?

Use both: rules for known patterns and hard policies (fast, explainable), ML for complex patterns and ranking risk. Compare on precision/recall at the business cost, maintenance effort, adaptability to new fraud, and explainability. Start with rules if no labels, add ML once chargeback labels accumulate.

5. A PM wants to add a chatbot to the app "because everyone has one". How do you respond?

Ask what user problem it solves and what success looks like. Check support logs and search queries for real needs. If the need is narrow (track order), a button may beat a chatbot. If needs are varied and language-heavy, a scoped assistant can help. Propose a small experiment with a clear metric before a big build.

6. How do you estimate the cost of an LLM feature before building it?

Estimate requests/day × tokens in/out × price (see §7), add embeddings, vector DB, eval and review costs. Compare with value created per request. Identify cost levers and the break-even point for self-hosting.

7. How would you improve a recommendation feature whose clicks are high but users say it's "repetitive"?

Clicks are a weak proxy. Add diversity/novelty constraints, measure long-term retention and satisfaction surveys, penalize near-duplicates, explore new items. Guardrail: don't let CTR drop too much while increasing diversity.

8. What metrics would you use for an AI customer support agent?

North star: resolution rate without human **and** CSAT ≥ human baseline. Drivers: deflection rate, first-contact resolution, time to resolve. Quality: correctness on audits, policy compliance, hallucination rate. Guardrails: escalation complaints, wrong actions (refunds), latency, cost per resolved ticket, repeat contacts within 7 days (a sign it didn't really solve the problem).

9. Leadership wants 100% accuracy before launch. How do you handle it?

Explain that probabilistic systems are never 100%, and that humans aren't either. Compare with the **current baseline** (human error rate, current process). Agree on an acceptable error rate per error type, and design for mistakes: human review for high-risk cases, citations, easy correction, monitoring. Propose a staged launch on low-risk scope.

10. How would you prioritize between improving model accuracy by 2% and reducing latency by 50%?

Depends on what limits user value. Check data: does latency hurt usage (drop-off while waiting)? Do errors cause complaints or escalations? Run small experiments if possible. Often latency wins for interactive features; accuracy wins for high-stakes decisions.

11. How do you know if users trust the AI feature?

Behavioral signals: acceptance without edits, users verifying sources less over time (but not blindly), repeat usage, handoff rate, override rate. Surveys. Watch for **over-trust** too (users accept wrong answers) in high-risk domains, and audit samples.

12. Design the metric dashboard for a new AI search feature.

Top: north star (successful sessions: click/answer accepted without reformulation). Funnel: queries → results/answers shown → clicks/accepts → task done. Quality: thumbs, no-answer rate, citation click rate, human-rated relevance sample. System: p50/p95 latency, error rate, cost per query. Guardrails: complaints, safety flags. Slice by user segment, language, query type.

13. How would you price an AI feature?

Understand cost per use and value per use. Options: included in plan (drives retention), add-on subscription, usage-based (credits). Protect margins with limits for heavy users. Test willingness to pay; watch cost of power users (the top 5% can drive most cost).

14. When would you choose a smaller, cheaper model over the best model?

When the task is narrow (classification, extraction), latency matters, volume is high, and evals show the small model meets the quality bar. Use routing: small model by default, escalate hard cases to the big model.

15. How would you run an experiment for an LLM feature?

Randomize by user; primary metric tied to value (task completion, retention); guardrails (cost, latency, complaints); enough duration to pass novelty; log model/prompt versions; segment results; check for sample ratio mismatch. See 11.

16. An exec asks: "Why can't the AI just do the whole job instead of assisting?"

Explain the error rate and the cost of errors in that job. Show a maturity path: suggest → draft for approval → automate low-risk cases → expand automation as measured accuracy and trust grow.

17. How would you choose the first AI use case for a company new to AI?

High volume, repetitive, measurable, low risk if wrong, data available, a motivated business owner. Examples: internal knowledge search, ticket triage, document extraction. Avoid starting with high-stakes autonomous decisions.

---

13. What-if scenarios (product)

Usage of the AI feature spiked at launch, then dropped 70% after 3 weeks.

Novelty effect, or first experiences were disappointing. Isolate: retention cohorts, satisfaction of returning vs churned users, sample their sessions. Fix: improve quality on top use cases, make value obvious in

workflow, remove friction (speed). Prevent: judge launch success on 4-8 week retention, not week 1.

AI cost is growing faster than revenue.

Find the cost per active user and the top 5% heaviest users; break down tokens by feature. Fix now: limits for heavy usage, caching, shorter contexts. Long-term: model routing, smaller fine-tuned model, pricing tiers aligned with usage.

Users say the AI is "confidently wrong" and complaints are going viral on social media.

Mitigate now: narrow the scope, add disclaimers and citations, route sensitive topics to humans, fix the shared examples. Root cause: hallucination or retrieval failures (see 04 #3). Prevent: red-team before launch, eval set of sensitive/edge questions, monitoring social and support channels.

A big customer refuses to use the AI because of data privacy.

Offer: no training on their data, data retention controls, region/tenant isolation, private deployment or bring-your-own-key, audit logs, clear documentation of data flow. Legal + security review.

The A/B test shows the AI feature improves engagement but reduces revenue.

Maybe users find answers without buying, or the AI displaces ads/upsells. Analyze segments and funnel; decide with leadership which goal matters; redesign so AI helps toward purchase (product suggestions) while keeping user value.

The model vendor raises prices 3× or deprecates the model you use.

Keep an abstraction layer (LLM gateway), maintain evals so you can switch models in days, have a second provider tested, consider open-weight models for core tasks.

Two teams want the same GPUs/budget for different AI projects.

Compare expected impact, confidence, cost and strategic value with numbers; consider shared platform investments (gateway, eval tooling) that help both; escalate with a clear recommendation, not just the conflict.

29. Senior, Staff & Leadership Rounds (AI/ML)

For senior engineer, staff/principal engineer, tech lead and engineering manager roles in AI/ML. At this level, interviewers care less about *whether* you can build a model and more about **judgment**: what to build, what not to build, how to reduce risk, and how to get people aligned.

Basic behavioral prep (STAR, project deep dive) is in 15-behavioral-and-project-deep-dive.md. This chapter builds on it.

Contents

1. What changes at senior levels
2. Technical strategy & roadmaps
3. Prioritization & build vs buy
4. Managing ML project risk and uncertainty
5. Estimating ML projects
6. Technical debt in ML systems
7. Org design for ML teams
8. Leading people: influence, mentoring, hiring
9. Cross-functional communication
10. Incidents & postmortems
11. Practice questions with model answers
12. What-if leadership scenarios

---

1. What changes at senior levels

Level	Scope	Interviewers look for
Mid-level	A feature / a model	Delivers well-defined work, good technical depth
Senior	A system or project end to end	Owns ambiguity, makes good trade-offs, mentors others
Staff / Principal	Multiple teams, a technical area	Sets direction, influences without authority, prevents big mistakes, multiplies others
Tech lead / Manager	A team	Delivery, people growth, hiring, alignment with business

Signals that you are senior: - You start from the **business problem**, not the model. - You talk about **trade-offs, risks, and what you chose *not* to do**. - Your stories include **other people** (you unblocked, aligned, coached). - You have **numbers** for impact. - You admit mistakes and show what changed afterwards.

2. Technical strategy & roadmaps

Q: How do you create a technical strategy for an AI/ML area? 1. **Understand the goal:** company objectives, user pain points, constraints (data, people, budget, regulation). 2. **Assess today:** current systems, data quality, tooling, skills, tech debt. 3. **Define where to be in 12-24 months:** a few clear bets (e.g., "shared feature store", "LLM gateway for all teams", "evaluation platform"). 4. **Path:** phased milestones, each delivering value on its own (no "big bang in 18 months"). 5. **Write it down** (1-3 page doc), get feedback early, revise, communicate widely. 6. **Measure:** metrics that show progress (time to deploy a model, % models monitored, cost per prediction).

Q: What makes a good roadmap? Outcome-based (problems solved), not a list of features; has explicit priorities and non-goals; reviewed regularly; balances new features, reliability, and tech debt (e.g., 70/20/10).

3. Prioritization & build vs buy

Prioritization frameworks to name: impact vs effort matrix, RICE (Reach × Impact × Confidence / Effort), cost of delay, "one-way vs two-way door" decisions (reversible decisions → move fast; irreversible → be careful).

Build vs buy (very common for AI: vector DB, LLM API vs self-host, labeling tools, MLOps platforms): | Consider | Questions | |---|---| | Core differentiation | Is this our competitive advantage? If not, prefer buy. | | Total cost | License/API cost vs engineers to build **and maintain** for years | | Time to value | How soon do we need it? | | Fit & control | Does the vendor meet requirements (latency, data residency, security)? Lock-in risk? | | Team skills | Can we operate it at 3 AM? | | Exit plan | Can we switch later? Use abstraction layers (e.g., an LLM gateway). |

Model answer pattern: "*I'd buy (or use managed) for undifferentiated infrastructure and build where it's our core advantage, with an abstraction layer so we can switch later. Then I'd revisit when the scale or cost changes.*"

4. Managing ML project risk and uncertainty

ML projects are uncertain: you don't know if the data supports the goal until you try.

Techniques: - **Feasibility spike first (1-3 weeks):** baseline + quick data analysis + rough model → go / no-go decision with stakeholders. - **Define success and "kill criteria" upfront:** "If we can't reach 80% precision at 50% recall by week 6, we stop or change scope." - **Start with the simplest thing:** rules/heuristics, then classic ML, then deep learning/LLMs. - **Ship thin end-to-end slice early:** a simple model in production with monitoring beats a perfect model in a notebook. - **Human-in-the-loop** for early versions to reduce risk. - **Risk register:** data availability, label quality, latency, compliance, adoption, dependencies. - **Communicate uncertainty honestly:** ranges and confidence, not single dates.

5. Estimating ML projects

- Break into phases: **data (access, cleaning, labels)** → baseline → iteration → productionization → monitoring → launch. Data work is usually **50%+ of the time**. Say this out loud.
- Estimate phases separately; give **ranges** (best/likely/worst); re-estimate at milestones.
- Time-box research phases instead of promising results.
- Include non-modeling work: reviews (security, legal, privacy), integration, A/B test duration, documentation, on-call.

6. Technical debt in ML systems

Common ML-specific debt (from "Hidden Technical Debt in ML Systems" ideas): - **Glue code & pipeline jungles:** ad-hoc scripts connecting everything. - **Entanglement (CACE):** "Changing Anything Changes Everything". One feature change shifts the whole model. - **Undeclared consumers:** other teams silently use your model's outputs. - **Data dependencies** stronger than code dependencies, with no contracts. - **Dead experimental code paths** and unused features. - **Feedback loops:** the model affects its own training data. - **Config debt:** thousands of untracked settings, prompts in random places. - **No monitoring / no evals:** the most dangerous debt.

How to handle debt as a leader: make it visible (debt register with cost/risk), tie fixes to business impact ("each deploy takes 2 days → 1 hour saves X"), reserve capacity each quarter, fix debt while building related features, set standards (paved road) to stop new debt.

7. Org design for ML teams

Model	How	Pros	Cons
Centralized ML team	One team serves the whole company	Shared standards, strong ML community	Far from product, becomes a bottleneck
Embedded	ML engineers inside product teams	Close to users and business	Duplicate infrastructure, isolated MLEs
Hub and spoke (hybrid)	Central **ML platform** team + embedded applied teams	Shared infra + product focus	Needs clear ownership boundaries

Platform team rule of thumb: treat internal teams as customers; build the "paved road" (standard way to train, deploy, monitor, evaluate) that is easier than doing it yourself; measure adoption and time-to-production.

8. Leading people: influence, mentoring, hiring

Leading without authority (staff engineers): understand others' goals first; use data and prototypes instead of opinions; write proposals (RFC/design docs) and invite critique; find allies; disagree and commit when the decision is made.

Mentoring: regular 1:1s; give stretch tasks with safety nets; review design docs, not just code; teach debugging methods (like 03); give credit publicly; give specific, kind, direct feedback.

Hiring for ML: define the role clearly (research vs applied vs platform); structured interviews with rubrics; test practical skills (debugging, system design, data sense), not trivia; look for curiosity and engineering rigor; diverse panels; fast, respectful process.

Performance issues: clarify expectations, give specific feedback early with examples, agree on a plan with support, follow up, document; escalate if no improvement.

9. Cross-functional communication

Audience	They care about	How to talk
Executives	Business impact, risk, cost, timeline	1-page summary, decision needed, options with trade-offs, recommendation first
Product managers	User value, scope, launch dates	What the model can/can't do, confidence ranges, MVP options
Legal / compliance / privacy	Data use, consent, regulation, liability	Data flows, retention, explainability, human oversight, audit logs
Engineers on other teams	APIs, dependencies, reliability	Contracts, SLAs, versioning, migration plans
Customers / support	Why the AI did something	Plain language, limitations, how to give feedback

Explaining ML to non-technical people: use analogies, focus on outcomes and error types ("it will miss about 1 in 20 fraud cases and wrongly block about 1 in 1,000 good customers"), never hide limitations.

10. Incidents & postmortems

During an incident (as leader): declare it, assign roles (incident commander, communications, investigators), **mitigate first** (rollback, disable feature, fallback), communicate regularly to stakeholders, keep a timeline. **After: blameless postmortem** covering what happened, impact, timeline, root causes (often several), what went well, action items with owners and dates, and follow-up until done. Focus on systems and processes, not individuals. AI-specific incidents: harmful/incorrect outputs at scale, data leakage between users, model drift causing business loss, runaway costs.

---

11. Practice questions with model answers

1. Tell me about the most complex technical decision you made. STAR: *Situation:* three teams each built their own LLM integrations; costs and incidents were growing. *Task:* I proposed a shared approach. *Action:* compared options (each team continues / buy a vendor gateway / build a thin internal gateway), wrote an RFC with cost and risk analysis, ran a 2-week prototype with one team, and chose the thin internal gateway with a provider abstraction. *Result:* all teams migrated in a quarter, LLM cost dropped 35% with caching and routing, and provider outages no longer took features down. Lesson: prototype before arguing.

2. How do you decide whether to use ML at all? Ask: is there a clear decision to improve, enough data with labels or feedback, a tolerance for errors, and value above the cost of building and maintaining it?

Always build a non-ML baseline first. If rules achieve 90% of the value, ship rules.

3. How do you balance research exploration and delivery? Time-boxed exploration with clear questions and decision points; keep a production baseline improving in parallel; share findings even when negative; portfolio approach (most effort on likely wins, some on bets).

4. A stakeholder wants 99% accuracy. What do you do? Understand why (what decision depends on it, what error type hurts), show the trade-off curve (precision vs recall), propose options: high-confidence automation + human review for the rest; agree on metrics that reflect business cost.

5. How do you set the technical direction for a team that's new to LLMs? Start with use cases with clear value and low risk; create shared foundations (evaluation set, prompt/version management, gateway, guardrails); train the team; set standards (no launch without evals and monitoring); review progress monthly.

6. Describe a time you disagreed with your manager. Show respectful disagreement with data, listening to their constraints, finding a middle ground or disagreeing and committing, and the result. Avoid stories where you were "right and they were dumb".

7. How do you handle a project that is failing? Detect early via milestones and metrics; be honest with stakeholders quickly; diagnose (wrong problem, bad data, unrealistic scope, team issues); options: rescope, pivot, add help, or stop. Stopping a project early is a success if it saves resources. Say this.

8. How do you measure the success of an ML platform team? Adoption (teams/models using it), time from idea to production, number of incidents, % models with monitoring and evals, cost per model/prediction, internal user satisfaction.

9. How do you make sure ML models are fair and safe? Define risks early with legal/ethics; slice-based evaluation; red-teaming for LLMs; human oversight for high-stakes decisions; documentation (model cards); monitoring; clear escalation path and authority to stop launches.

10. How do you grow junior engineers? Clear expectations, stretch projects with support, pair on debugging and design, feedback in 1:1s, let them present their work, sponsor them for opportunities.

11. How do you deal with tech debt when product wants only features? Translate debt into business terms (slower delivery, incidents, costs), propose a fixed capacity share, bundle debt fixes into feature work, show results to build trust.

12. Describe a time you influenced a team you didn't manage. Example: convinced another team to adopt data contracts by showing how 4 of the last 6 model incidents came from their schema changes, proposed a lightweight process, offered to build the CI check myself. Result: zero schema-related incidents next quarter.

13. How do you choose between two strong technical proposals from your team? Agree on decision criteria first (cost, risk, time, reversibility); time-boxed prototypes if needed; decide, document the reasoning (ADR), and support the chosen path; give credit to both.

14. How do you evaluate a vendor's AI product? Run it on *our* data with *our* eval set, check latency/cost at our scale, security & data handling (training on our data? retention? region?), SLAs, lock-in and exit plan, references. See 05 #3 for abstraction.

15. What would you do in your first 90 days as ML tech lead? 30 days: listen. Do 1:1s with team and stakeholders, understand systems, data, pain points, metrics. 60 days: pick 1-2 quick wins, write a draft strategy. 90 days: agreed roadmap, team processes (design reviews, eval standards, on-call), first improvements shipped.

16. How do you handle an underperforming team member? See §8: specific feedback early, understand causes (skills, clarity, personal issues), a support plan with clear goals, regular follow-up, fair documentation, escalate with HR if needed.

17. How do you communicate a delay to executives? Early, with the reason, impact, new realistic range, options (cut scope / add people / move date) and your recommendation. Never surprise them at the deadline.

18. How do you keep ML systems reliable at scale? SLOs, monitoring of data/model/system, canary deployments, rollback, runbooks, on-call, postmortems, regular game days, and evals in CI.

19. Tell me about a mistake that had real impact. Own it fully, explain impact with numbers, what you did immediately, what systemic change prevented recurrence, what you learned. (See failure example in 15.)

20. How do you decide build vs buy for a vector database? Scale (millions vs billions of vectors), existing stack (Postgres → pgvector is often enough), latency/filtering needs, ops skills, cost, data residency. Start simple, abstract the interface, migrate if limits appear.

21. How do you prioritize between many AI feature requests? Value × confidence / effort, strategic alignment, data readiness, risk; group into themes; say no clearly with reasons; review quarterly.

22. How do you ensure knowledge isn't stuck in one person? Documentation, design docs, pairing, rotating on-call, code reviews by multiple people, runbooks, recorded demos.

23. What's your approach to design reviews? Written doc before code for significant changes; reviewers from affected teams; focus on goals, alternatives, risks, rollout, monitoring; time-boxed; decision recorded.

24. How do you think about AI regulation and compliance? Know the main ideas (risk-based rules like the EU AI Act, privacy laws like GDPR, sector rules in finance/health), involve legal early, build documentation, data lineage, human oversight, and the ability to explain and delete data into the system design from the start.

25. Why do ML projects fail, in your experience? Wrong problem or no clear business owner, data not available or poor quality, success not defined, no path to production, no monitoring after launch, users don't trust or adopt the output. Mention how you prevent each.

26. How do you build trust with a skeptical business team? Start small with a pilot they choose, show results in their metrics, be transparent about failures, involve them in error analysis, deliver consistently.

---

12. What-if leadership scenarios

(Same idea as 03: clarify → options → decide → communicate → prevent.)

Your model fails publicly right after launch (bad outputs go viral / big customer complains)

1. **Mitigate first:** rollback or disable the feature, or add a stricter filter; inform stakeholders and support within the hour.
2. **Assess impact:** how many users, what harm, legal/PR risk.
3. **Communicate:** honest internal update, help comms team with external message.
4. **Root cause** via postmortem (eval gaps? launch too fast? missing guardrails?).
5. **Prevent:** stronger launch checklist (red-teaming, slice evals, staged rollout, kill switch).

Your team strongly disagrees on the approach (fine-tune vs RAG, PyTorch vs JAX, build vs buy)

Make the disagreement productive: agree on criteria, let each side write a short proposal, run a time-boxed experiment on the same eval set, decide (or escalate with a clear recommendation), document the decision, and ask everyone to commit. Thank the "losing" side. Their concerns become risks to monitor.

An executive says "Put GenAI in every product this quarter"

Don't say no; channel the energy: list candidate use cases, score by value, feasibility, and risk, pick 2-3 high-value pilots with clear metrics, build shared foundations (gateway, evals, guardrails) to make later launches cheap and safe, report results honestly. Explain risks (cost, hallucinations, data privacy) in business terms.

The deadline is impossible

Clarify what's really fixed (date? scope? quality?). Offer options: reduced scope (MVP), phased launch, more people (with ramp-up cost), or a new date. Give your recommendation with confidence ranges. Don't silently burn the team out or cut evaluation and safety.

Your key engineer resigns in the middle of a critical project

Short-term: knowledge transfer plan for the notice period (docs, recorded walkthroughs, pairing), reassign ownership, re-plan timeline with stakeholders. Long-term: reduce bus factor (see Q22), understand why they left (stay interviews with others), backfill hiring.

An engineer raises an ethical concern about a model (bias, privacy, misuse)

Take it seriously and thank them. People need to feel safe raising concerns. Pause launch if the risk is real; investigate with data (slice metrics, privacy review); involve legal/ethics/responsible-AI team; decide transparently and document; fix or stop. Never punish the person who raised it.

Product wants to launch, but evaluations show the model isn't ready

Show the evidence in business terms (error rate, affected users, risk), propose alternatives (limited beta, human review, narrower scope, feature flag with staged rollout), agree on launch criteria together. If overruled, document the risk and ensure monitoring + kill switch.

Two teams built overlapping ML platforms

Map both (users, features, costs), meet both leads, agree on a target architecture and a migration path, avoid "winner and loser" framing, merge the best parts, set timeline and ownership.

Your team's model costs more to run than the revenue it creates

Measure unit economics (cost per prediction vs value), then: optimize (smaller models, caching, batching, routing), reprice or limit usage, or retire the feature. Present options with numbers to leadership.

A regulator or big customer asks "how does your AI make decisions?"

Provide documentation you should already have: model cards, data sources and lineage, evaluation results by segment, human oversight process, explanation methods, incident history and controls. If you don't have them, that's a priority action item.